Qissago

Project Team

Dania Waseem 23I-2622
Munaza Tariq 23i-2545
Wajeeha Khalid 23i-2610

Session 2023-2027

Supervised by

Mr. Sohail Abbas



FAST School of Computing

National University of Computer and Emerging Sciences
Islamabad, Pakistan

October, 2026

Contents

Students' Submission i

Anti-Plagiarism Statement iii

Authors' Declaration v

Abstract vii

Acknowledgments ix

Executive Summary xi

List of Abbreviations xv

1 Introduction and Project Vision 1

1.1 Project Overview . . . 1
1.2 Problem Statement . . . 2
1.3 Problem Context and Motivation . . . 2
1.4 Project Objectives . . . 2
1.5 Project Scope . . . 4
1.6 Project Modules . . . 4
1.7 Stakeholders and User Classes . . . 5
1.8 Success Criteria . . . 6
1.9 Constraints and Assumptions . . . 7
1.10 Ethical, Legal and DEI Considerations . . . 8

2 Research and Existing Solutions 9

2.1 Existing Solutions and Products . . . 9
2.2 Comparison of Existing Solutions . . . 10
2.3 Limitations and Gaps . . . 10
2.4 Proposed Solution . . . 11
2.5 Complex Computing Problem . . . 12
2.6 Technical Challenges . . . 12
2.7 Research Findings and Design Decisions . . . 13

3 Proposal Feedback and Iteration Plan **15**
3.1 Proposal Feedback Summary . . . 15
3.2 Proposal Action Register . . . 16
3.3 Iteration Strategy . . . 17
3.4 Iteration 1 Plan . . . 18
3.5 Iteration 2 Plan . . . 19
3.6 Iteration 3 Plan . . . 20
3.7 Iteration 4 Plan . . . 22
3.8 Work Breakdown and Responsibilities . . . 23
3.9 Progress Against Plan . . . 23
3.10 Delays, Deviations and Recovery Plan . . . 24

4 Software Requirements **25**
4.1 User Interactions / Use Cases . . . 25
4.1.1 Actors . . . 25
4.1.2 Use case list . . . 26
4.1.3 Use-case / event response table . . . 27
4.2 Use Case Diagram . . . 29
4.3 Detailed Use Cases . . . 29
4.4 Functional Requirements . . . 35
4.4.1 Module 1: Authentication . . . 35
4.4.2 Module 2: Chatbot (Intake) . . . 35
4.4.3 Module 3: Quick-select Form . . . 36
4.4.4 Module 4: Multi-agent Planner . . . 36
4.4.5 Module 5: Itinerary Viewing and Management . . . 37
4.4.6 Module 6: Content and Data . . . 37
4.4.7 Module 7: Replanning . . . 38
4.4.8 Module 8: Narration . . . 38
4.4.9 Module 9: Video . . . 39
4.5 Non-Functional Requirements . . . 39
4.5.1 Reliability . . . 39
4.5.2 Usability . . . 40
4.5.3 Performance . . . 40
4.5.4 Security . . . 40
4.5.5 Scalability . . . 41
4.5.6 Compatibility . . . 41
4.5.7 Maintainability . . . 41
4.6 Acceptance Criteria . . . 41
4.7 Requirements Traceability . . . 43

5 System Design **45**

5.1 System Architecture . . . 45
5.1.1 Box and Line Diagram . . . 45
5.1.2 Four-Tier Architecture . . . 46
5.1.2.1 Presentation Layer . . . 46
5.1.2.2 Application Layer . . . 48
5.1.2.3 Integration Layer . . . 48
5.1.2.4 Data Layer . . . 48
5.1.3 Architecture Benefits . . . 49
5.2 Architectural Components . . . 49
5.3 Interfaces and Dependencies . . . 51
5.3.1 REST Interface . . . 51
5.3.2 External Dependencies . . . 52
5.4 Architecture Decisions and Rationale . . . 52
5.5 Design Evolution . . . 53
5.6 Domain Model . . . 54
5.7 Data Design . . . 55
5.7.1 Users Table . . . 55
5.7.2 Cities Table . . . 58
5.7.3 Interests Table . . . 58
5.7.4 Places (POIs) Table . . . 58
5.7.5 Place Interests Table . . . 59
5.7.6 Chat Sessions and Chat Messages Tables . . . 60
5.7.7 Itineraries Table . . . 60
5.7.8 Agent Runs Table . . . 61
5.7.9 Itinerary Versions and Replan Requests Tables (Iteration 2) . . . 61
5.7.10 Source, Persona and Narration Tables (Iteration 3) . . . 62
5.7.11 Video Tables (Iteration 4) . . . 63
5.7.12 Relational Integrity . . . 64
5.8 Process / Behavior Design . . . 65
5.8.1 Activity Diagram . . . 65
5.8.1.1 Trip Description Flow . . . 65
5.8.1.2 Itinerary Generation Flow . . . 65
5.8.1.3 Itinerary Use Flow . . . 67
5.8.1.4 Key Decision Points . . . 67
5.8.2 Data Flow Diagrams . . . 67
5.8.2.1 DFD Level 0 – Context Diagram . . . 67
5.8.2.2 DFD Level 1 . . . 69
5.8.2.3 DFD Level 2 – Collect Preferences . . . 72
5.8.2.4 DFD Level 2 – Generate Itineraries . . . 73
5.8.2.5 DFD Level 2 – Replan (Iteration 2) . . . 75

5.8.2.6 DFD Level 2 – Narrate (Iteration 3) . . . 77
5.8.2.7 DFD Level 2 – Generate Video (Iteration 4) . . . 81
5.8.3 System-level Sequence Diagrams . . . 82
5.8.4 State Diagrams . . . 82
5.8.5 Class Diagram . . . 82
5.8.6 Deployment Diagram . . . 90

6 Iteration 1: Design and Implementation 93
6.1 Iteration Objectives . . . 93
6.2 Selected Use Cases / Features . . . 93
6.3 Design . . . 94
6.3.1 Agent Pipeline . . . 94
6.3.2 Chatbot Design . . . 95
6.3.3 User Interface Design . . . 95
6.4 Implementation . . . 95
6.5 Completed Features / Module . . . 96
6.6 Technical Implementation Details . . . 97
6.7 Algorithms and Technical Decisions . . . 99
6.7.1 Travel Distance and Time . . . 99
6.7.2 Cost Model . . . 99
6.7.3 Place Scoring . . . 100
6.7.4 Daily Rhythm . . . 100
6.7.5 Budget-Constrained Selection and Scheduling . . . 101
6.7.6 Match Score . . . 101
6.7.7 Technical Decisions . . . 102
6.8 Tools, Frameworks and Technologies . . . 102
6.9 Demonstration Evidence . . . 103
6.10 Iteration Outcome . . . 104

7 Testing and Early Validation 105
7.1 Testing Strategy . . . 105
7.2 Test Levels and Techniques . . . 105
7.3 Test Cases . . . 106
7.3.1 Chatbot Test Cases . . . 106
7.3.2 Planner and Cost Test Cases . . . 108
7.3.3 API, Data and Ingestion Test Cases . . . 109
7.3.4 Application (Frontend) Test Cases . . . 109
7.4 Test Results . . . 110
7.4.1 Backend . . . 110
7.4.2 Application . . . 112
7.5 Acceptance Testing / Validation . . . 112

7.6 Edge and Failure Cases . . . 114
7.7 Defects and Corrective Actions . . . 114
7.8 Current Validation Status . . . 115

8 Project Management and Development Evidence 117
8.1 Work Completed . . . 117
8.2 Work Remaining . . . 118
8.3 Individual Responsibilities . . . 118
8.4 Contribution of Each Team Member . . . 119
8.5 Tools and Tool-Assisted Development . . . 120
8.6 External Components and Dependencies . . . 121
8.7 Risks and Mitigation . . . 121
8.8 Security and Privacy Considerations . . . 122
8.9 Current Limitations . . . 122

9 Remaining FYP Plan 125
9.1 Remaining Scope . . . 125
9.2 Remaining Modules and Features . . . 125
9.3 Remaining Iterations . . . 126
9.4 Implementation Plan . . . 126
9.4.1 Iteration 2 – Replanning . . . 126
9.4.2 Iteration 3 – Narration . . . 127
9.4.3 Iteration 4 – Video . . . 127
9.5 Testing and Validation Plan . . . 127
9.6 Deployment Plan . . . 128
9.7 Tentative Timeline . . . 128
9.8 Expected Final Deliverables . . . 129

10 Conclusion 131

List of Figures

3.1 Iteration roadmap of QissaGo 17
3.2 Iteration 1: personalized itinerary planning 18
3.3 Iteration 2: dynamic replanning 20
3.4 Iteration 3: cinematic narration 21
3.5 Iteration 4: personalized travel video 22

4.1 Use case diagram for QissaGo 34

5.1 Box and Line Diagram of QissaGo 47
5.2 Four-Tier Architecture of QissaGo 47
5.3 Component diagram of QissaGo 56
5.4 Multi-agent architecture of the complete system 56
5.5 Domain model of QissaGo 56
5.6 Entity relationship diagram of QissaGo (all iterations) 57
5.7 Activity diagram of QissaGo 66
5.8 Context diagram of QissaGo 68
5.9 DFD Level 1 diagram of QissaGo 68
5.10 DFD Level 2 diagram of Collect Preferences 71
5.11 DFD Level 2 diagram of Generate Itineraries 74
5.12 DFD Level 2 diagram of Replan 76
5.13 DFD Level 2 diagram of Narrate 78
5.14 DFD Level 2 diagram of Generate Video 80
5.15 System Sequence Diagram for Register function (Authentication Module) 83
5.16 System Sequence Diagram for Log In function (Authentication Module) . 83
5.17 System Sequence Diagram for Start Chat function (Chatbot Module) . . . 83
5.18 System Sequence Diagram for Send Chat Message function (Chatbot Module) 84
5.19 System Sequence Diagram for Get Form Options function (Quick-select Form Module) 84
5.20 System Sequence Diagram for Generate Itineraries function (Multi-agent Planner Module) 84
5.21 System Sequence Diagram for View Itinerary function (Itinerary Module) 85
5.22 System Sequence Diagram for View All Options function (Itinerary Module) 85

5.23 System Sequence Diagram for Save / Unsave Itinerary function (Itinerary Module) . . . 85
5.24 System Sequence Diagram for Get Saved Itineraries function (Itinerary Module) . . . 86
5.25 System Sequence Diagram for Export Itinerary PDF function (Itinerary Module) . . . 86
5.26 System Sequence Diagram for Edit Itinerary function (Replanning Module) 86
5.27 System Sequence Diagram for Weather Replanning function (Replanning Module) . . . 87
5.28 System Sequence Diagram for Get Stop Narration function (Narration Module) . . . 87
5.29 System Sequence Diagram for Ask About a Stop function (Narration Module) . . . 87
5.30 System Sequence Diagram for Generate Travel Video function (Video Module) . . . 88
5.31 System Sequence Diagram for Check Video Status function (Video Module) 88
5.32 State diagram of a chat session . . . 88
5.33 State diagram of an itinerary . . . 89
5.34 State diagram of a video job (Iteration 4) . . . 89
5.35 Class diagram of the agents and services . . . 89
5.36 Deployment diagram of QissaGo . . . 91

List of Tables

1.1 Project objectives . . . 3
1.2 Project modules . . . 5
1.3 Stakeholders and user classes . . . 5
1.4 Success criteria . . . 6

2.1 Existing travel applications . . . 9
2.2 Comparison of existing solutions . . . 10
2.3 Limitations of existing solutions . . . 11
2.4 Technical challenges . . . 12
2.5 Findings and the design decisions they led to . . . 13

3.1 Proposal action register . . . 16
3.3 Iteration 1 objectives . . . 18
3.4 Iteration 1 features and deliverables . . . 19
3.5 Iteration 2 objectives and deliverables . . . 20
3.6 Iteration 3 objectives and deliverables . . . 20
3.2 Iteration roadmap . . . 21
3.7 Iteration 4 objectives and deliverables . . . 22
3.8 Work breakdown and responsibilities . . . 23
3.9 Iteration 1 milestones . . . 23
3.10 Deviations from the proposal . . . 24

4.1 Actors of QissaGo . . . 25
4.2 Use cases . . . 26
4.3 Use-case / event response table . . . 27
4.4 UC-01 Register . . . 29
4.5 UC-04 Plan trip by chat . . . 30
4.6 UC-06 Plan trip by quick-select form . . . 30
4.7 UC-22 Generate itineraries . . . 31
4.8 UC-08 View itinerary details . . . 31
4.9 UC-13 Export itinerary PDF . . . 32
4.10 UC-14 Edit itinerary through chat . . . 32
4.11 UC-17 Listen to stop narration . . . 33
4.12 UC-20 Generate travel video . . . 33

4.13 Acceptance criteria . . . 42
4.14 Requirements traceability matrix . . . 43

5.1 Architectural components . . . 49
5.2 REST endpoints . . . 51
5.3 External dependencies . . . 52
5.4 Architecture decisions . . . 52
5.5 Design changes since the proposal . . . 53
5.6 Domain entities . . . 54

6.1 Use cases and features of Iteration 1 . . . 94
6.2 Completed modules of Iteration 1 . . . 96
6.3 Important files, classes and functions . . . 97
6.4 Variant weights . . . 100
6.5 Pace and scheduling rules . . . 101
6.6 Tools, frameworks and technologies . . . 102
6.7 Demonstration evidence . . . 103
6.8 Iteration 1 outcome . . . 104

7.1 Test levels and techniques . . . 106
7.2 Chatbot test cases . . . 107
7.3 Planner and cost test cases . . . 108
7.4 API, data and ingestion test cases . . . 109
7.5 Application test cases . . . 109
7.6 Backend test results by file . . . 110
7.7 Backend coverage by module . . . 111
7.8 Application test results . . . 112
7.9 Manual acceptance tests . . . 113
7.10 Acceptance criteria status . . . 113
7.11 Edge and failure cases covered . . . 114
7.12 Defects found and corrective actions . . . 115
7.13 Validation status at the end of Iteration 1 . . . 115

8.1 Work completed in Iteration 1 . . . 117
8.2 Work remaining . . . 118
8.3 Individual responsibilities . . . 119
8.4 Contribution of each team member in Iteration 1 . . . 119
8.5 Development tools . . . 120
8.6 External components and licences . . . 121
8.7 Risks and mitigation . . . 121

9.1 Remaining modules and features . . . 125

9.2 Remaining iterations 126
9.3 Testing and validation plan 127
9.4 Tentative timeline 128
9.5 Expected final deliverables 129

Students' Submission

We certify that this report is submitted as the FYP-1 mid-report for the project titled *Qissago*.

Student 1: Dania Waseem	Signature: _____
Student 2: Munaza Tariq	Signature: _____
Student 3: Wajeeha Khalid	Signature: _____

Anti-Plagiarism Statement

The students certify that this report is their own work and that all sources used in preparing the report have been appropriately acknowledged and referenced.

Authors' Declaration

We declare that the work documented in this report represents the work completed by the project team up to the FYP-1 mid-report stage. Contributions, tools, external components, and relevant sources are disclosed where applicable.

Abstract

Travellers in Pakistan who want a personalised trip still have to collect information from scattered websites, social media and word of mouth and then turn it into a day-by-day plan that respects their budget, time, group and interests. Existing travel applications mostly offer static lists of attractions or generic itineraries; they rarely understand free-form requests, rarely question missing or contradictory details and do not continue the experience during and after the trip. QissaGo addresses this gap with an AI-powered tourism application built around the travel experience. A user describes the desired trip through a conversational chatbot or a quick-selection interface; a multi-agent system then plans the trip and later iterations add cinematic narration at stops and an automatically generated travel video of the completed trip.

The project follows an iterative, product-oriented methodology with three iterations. Iteration 1, which is complete, delivers user accounts, a multi-turn chatbot with preference extraction and targeted clarification, a quick-selection planning form and a multi-agent planner in which an orchestrator coordinates intake, weather, destination, budget, scheduling, validation and enrichment agents to produce three ranked, budget-aware and weather-aware itineraries with travel times, prices, weather, maps and opening hours. Users can also save itineraries, view or hide per-stop details and export their plans as PDF files. The system maintains a place database populated from open sources to support destination selection and itinerary generation. Language understanding uses a hybrid of rule-based extraction and a locally hosted open-source large language model, while all cost, time and distance decisions are made by deterministic algorithms. The system is implemented with Flutter for Android and web, a FastAPI backend, PostgreSQL and Docker.

The remaining iterations will extend the experience with conversational itinerary editing, dynamic replanning and persona-based cinematic narration with audio at selected destinations. The final stage will introduce template-based automated travel video generation to transform completed trips into shareable visual stories. Through these capabilities, QissaGo aims to provide a more personalised, adaptive and engaging travel experience while promoting tourism in Pakistan.

Keywords: AI, Tourism, Chatbot, Travel Planning, Multi-Agent System

Acknowledgments

The project team acknowledges the guidance and support received from the project supervisor, faculty members, and other contributors.

Executive Summary

Project Overview and Motivation

QissaGo is an AI-powered tourism application that transforms a traveller's trip preferences into a personalised travel experience through a chatbot or a quick-selection interface. Planning a trip often requires gathering information from scattered sources and manually balancing budget, interests, group requirements, travel dates and pace. Existing tools largely provide static information and offer limited support for incomplete requests or changing travel needs. QissaGo addresses these challenges through a multi-agent system that generates personalised itineraries and aims to make travel more engaging and memorable through cinematic narration at destinations and automated travel memory videos. The application supports Android and web platforms through a single codebase.

Scope

The complete system consists of four modules: (1) a conversational chatbot that collects trip preferences and asks targeted follow-up questions; (2) a multi-agent itinerary planner that builds and later re-plans day-by-day itineraries; (3) cinematic narration with multiple personas and question answering about each stop; and (4) automated travel video generation from user-selected templates.

Research findings

A review of existing travel planning applications highlighted a lack of focus on preserving travel memories and creating meaningful experiences beyond itinerary planning. This finding shaped QissaGo's vision of making travel more memorable through cinematic narration at destinations and automatically generated travel memory videos. The system also combines a conversational chatbot with deterministic planning agents to provide

personalised itineraries that respect budget, time, weather and opening hours.

Current Implementation

Iteration 1 is complete. Users can register, log in and maintain sessions. The chatbot supports multi-turn conversations, extracts trip details including destinations, dates, budgets, interests and special needs, and asks targeted questions with suggested answers when information is missing or contradictory. A quick-select form provides the same planning process through buttons. On confirmation, an orchestrator coordinates seven specialised agents (Intake, Weather, Destination, Budget, Scheduling, Validator and Enrichment) to generate three alternative itineraries ranked by match score. Users can save, revisit, compare and export itineraries as PDFs. Place content from open sources, including Wikipedia, Wikidata, Wikimedia Commons and official tourism and heritage websites, is stored in the system's database. The system uses Flutter, FastAPI, PostgreSQL, Docker and Ollama.

Validation Status

Iteration 1 validation combines automated and manual testing. Automated tests cover chatbot extraction and clarification, planning agents, the complete planning pipeline, API integration, authentication, data isolation and the mobile and web interfaces. All automated tests pass and run through GitHub Actions with a code-coverage threshold. Manual acceptance testing covers the complete user journey from registration to saving and exporting itineraries. Defects identified during testing were corrected. Further details are provided in Chapter 7.

Risks and remaining work

The main risks are the restriction to free services, the reliability and speed of a locally hosted language model, changes in real-world information such as prices and opening hours, the limits of free hosting, factual accuracy of generated narration and the processing time of video rendering. These are mitigated respectively by open-source components, a rules-first design with validated model output and automatic fallback, repeatable data ingestion with recorded sources, openly licensed media with stored credits, a local demonstration setup, retrieval-grounded narration and background rendering jobs. The remaining work is Iteration 2 and 3 (replanning, narration with personas, text-to-speech,

and question answering about a stop) and Iteration 4 (video templates and automatic video generation), followed by user studies.

List of Abbreviations

API	Application Programming Interface
APK	Android Package
CI/CD	Continuous Integration / Continuous Delivery
CORS	Cross-Origin Resource Sharing
DEI	Diversity, Equity and Inclusion
ER	Entity Relationship
FR	Functional Requirement
FYP	Final Year Project
HTTPS	Hypertext Transfer Protocol Secure
JSON	JavaScript Object Notation
JWT	JSON Web Token
LLM	Large Language Model
NFR	Non-Functional Requirement
ORM	Object-Relational Mapping
PDF	Portable Document Format
PKR	Pakistani Rupee
RAG	Retrieval-Augmented Generation
REST	Representational State Transfer
SUS	System Usability Scale
TTS	Text-to-Speech
UI	User Interface
UX	User Experience

Chapter 1

Introduction and Project Vision

1.1 Project Overview

QissaGo is an AI-powered tourism application for Pakistan, built around the travel experience rather than only the travel plan. It runs as an Android app and as a web app from a single Flutter codebase. The user describes the trip through a chatbot (destination, dates, number of days, budget, group, interests and pace) or by selecting options with buttons. A multi-agent system then builds day-by-day itineraries around those preferences. The experience continues after planning: the user can change the plan by chatting, hear cinematic narration at the stops of the trip and receive an automatically assembled travel video of the finished trip.

The system is built in four iterations.

- **Iteration 1** is conversational trip planning. The chatbot collects preferences and the multi-agent planner produces three alternative itineraries with time slots, travel times, costs, weather and place details. This iteration is complete.
- **Iteration 2** adds dynamic replanning, where an itinerary is edited through chat and only the affected parts are rebuilt.
- **Iteration 3** adds cinematic narration at selected stops of the trip.
- **Iteration 4** adds the automated travel video, together with final testing, deployment and user evaluation.

This report describes the work up to the FYP-1 mid evaluation.

1.2 Problem Statement

A traveller who wants to visit a city in Pakistan must decide which places to see, how many days to stay, how to move between places and how much it will cost. These decisions depend on each other. A plan made without checking all of them can be over budget, can have too many stops in one day, can include places that are closed at the planned time, or can send the group outdoors on a rainy or very hot day.

Many applications and websites already help with trip planning, so the problem is not the absence of planners. The problem is that existing tools solve only one part of the journey. Most either list attractions and leave the user to combine them, or generate a plan once and do not check it against budget, opening hours, weather and pace. Very few are built for destinations in Pakistan, and, to the best of our knowledge, none connects the whole experience in one system: a natural conversation to describe the trip, a feasible plan, changes to that plan through chat, a story told about each place and a video that remembers the trip afterwards.

The problem addressed by this project is to build a single system that accepts a trip description in natural language, returns feasible itineraries that respect the constraints and then extends the same trip into an editable plan, a narrated experience and a travel video, without the user having to research and combine every detail by hand.

1.3 Problem Context and Motivation

Pakistan has many historical, cultural and natural places, but information about them, about prices, opening hours and weather is spread over different websites and applications. Tourists often see a fort or a museum without knowing its story, and after the trip making a video to remember it takes time and editing skill.

The project is therefore motivated by three ideas. First, a conversation is the natural way to describe and change a trip, while a computer is better than a person at the exact parts of planning: adding costs, estimating travel time and checking opening hours. Second, narration turns a list of places into a story. Third, an automatic video turns a finished trip into a memory without extra effort.

1.4 Project Objectives

The objectives of Iteration 1 are measurable through automated tests and manual acceptance scenarios. The objectives of Iterations 2, 3 and 4 are planned.

Table 1.1: Project objectives

ID	Objective	Iteration
O1	A user can describe a trip in natural English or with buttons.	1
O2	The system detects missing or contradictory details and asks targeted follow-up questions instead of guessing.	1
O3	Specialised agents produce three feasible, budget-respecting, weather-aware itineraries and rank them.	1
O4	Every stop shows the attributes defined in the proposal (route, time, distance, price, weather and review score) and the user can switch them on or off.	1
O5	Conversations and itineraries are stored persistently for each user.	1
O6	A user can change a generated itinerary through chat, for example by removing a stop or changing the start time.	2 (planned)
O7	After a change, the system rebuilds only the affected days and keeps the plan feasible within the budget, opening hours and pace.	2 (planned)
O8	The system detects a weather problem in a plan and suggests a replanning option to the user.	2 (planned)
O9	At selected stops, the system turns stored source material about the place into a narration script in a storytelling tone instead of a plain list of facts.	3 (planned)
O10	The user can choose between more than one narrator persona or tone.	3 (planned)
O11	Narration is shown or played at the relevant stop of the trip and is based only on stored source material.	3 (planned)
O12	A user can select a video template (for example a highlight reel or a day-by-day recap) with its own pacing and style.	4 (planned)
O13	The system automatically assembles a travel video from the images, clips and captions of the stops of the trip.	4 (planned)
O14	Captions for each stop are generated and stay consistent with the narration content.	4 (planned)
O15	Development follows industry practice: automated tests, CI/CD, a branch-based workflow and documentation.	4 (planned)
O16	The complete system is deployed and evaluated by real users.	4 (planned)

1.5 Project Scope

Included in Iteration 1 (implemented)

- User accounts: register, log in and log out.
- A chatbot that collects trip preferences in multi-turn English conversation and a quick-select form that shares the same preference model.
- A multi-agent planner that generates three alternative day-by-day itineraries with a match score.
- Per-stop details with show/hide options, day maps and route links.
- Saving, listing and reopening itineraries, viewing all generated options and PDF export.

Planned for Iterations 2, 3 and 4

- Iteration 2: editing an itinerary through chat, rebuilding only the affected days and weather-triggered replanning suggestions.
- Iteration 3: cinematic narration at selected stops, written in a storytelling tone from stored source material and offered in several personas.
- Iteration 4: video templates, automatic assembly of a travel video from the stops of a trip, captions, automated tests with a CI/CD pipeline, final deployment and user evaluation.

Not included

- Languages other than English.
- Destinations that are not in the project database.
- Live traffic data and live prices; travel times and prices are estimates.
- Travel to the destination city other than by road.

1.6 Project Modules

Table 1.2: Project modules

Module	**Responsibility**
Authentication	Registration, login, logout, persistent login, protection of user data.
Chatbot (Intake)	Conversation manager, preference extractor and a local LLM.
Quick-select form	Button-based alternative to chat that fills the same preference model.
Multi-agent planner	Orchestrator and the Weather, Destination, Budget, Scheduling, Validator and Enrichment agents; three ranked itineraries.
Itinerary viewing and management	Stop details with toggles, maps and route links, saved itineraries, all options, PDF export.
Content and data	Database, ingestion scripts and seed scripts for places, images and weather caching.
Replanning	Edit an itinerary through chat and rebuild only the affected days; weather-triggered suggestions.
Narration	Turns stored source content about a stop into a storytelling script in more than one persona, and presents it at the relevant point of the trip.
Video	Caption generation, template-based video assembly from the stops of the trip, and background rendering.

1.7 Stakeholders and User Classes

Table 1.3: Stakeholders and user classes

User class	**Description**
Solo travellers	Plan a trip alone on a phone or in a browser. Want flexibility, safe and simple plans, and are not expected to have technical knowledge.

User class	Description
Group travellers	Couples, families and groups of friends who travel together. Needs differ by group: families with children or elderly members need a relaxed pace, indoor options and short drives, couples look for scenic and cultural places, and friends prefer a fuller schedule with adventure. All of them share costs and need a realistic budget.
Budget travellers and students	Have a strict budget and need plans that cover meals, hotel, transport and entry fees without surprises.
Domestic and international tourists	Visitors who are unfamiliar with some or all destinations in Pakistan. They need clear plans and rely on stories and context at each place to understand what they see. They use the English interface.
Content creators and travel enthusiasts	Like to remember and share trips, and benefit from the narration and the automatic travel video.
Data and service providers	Open-Meteo, OpenStreetMap, Wikipedia, Wikidata and Wikimedia Commons provide weather, maps, text and images under open terms.

1.8 Success Criteria

Table 1.4: Success criteria

Criterion	How it is measured
Generated plans respect the constraints	Agent tests over several trip profiles check that three plans are produced, that each is within budget, has no overlapping stops, respects opening hours and the pace limit, and has no duplicate places.
Chatbot understands varied input	Extractor and chat tests with different phrasings, and an evaluation set of about 50 realistic messages.
Full user flow works	An automated test of the full flow (chat, generate, save, list, view options and unsave), and manual acceptance scenarios on the app.
Backend is well tested	All backend tests pass and test coverage stays above 70%.
Replanning is correct	After a chat edit, only the affected days change, and the new plan still respects the budget, opening hours and pace.

Criterion	How it is measured
Narration quality	Factual consistency of the narration with the stored source text, and user ratings of how engaging it is compared with a plain facts list.
Video is produced correctly	A video is generated for a finished trip in the chosen template, with the correct stops, images and captions, within an acceptable rendering time.
Usability	A study with 5 to 10 users measuring task success, task time and the System Usability Scale (SUS) score.

1.9 Constraints and Assumptions

Constraints

- Only free and open services and tools may be used; paid APIs and paid models are not allowed.
- The language model must run locally. A small model (Qwen 2.5, 3B) is used, so its language ability is limited and its output must be validated.
- The system works in English only.
- Destinations are limited to those stored in the project database.
- Place content depends on open sources; prices and opening hours change over time and are not live.
- The project is done by a team of three within the academic calendar of FYP-1 and FYP-2.

Assumptions

- The user has an internet connection to reach the backend. Weather has a fallback when the weather service cannot be reached.
- Costs are calculated in Pakistani Rupees (PKR) from stored estimates for meals, hotels, transport and entry fees.
- Travel between places is by road, by car.

1.10 Ethical, Legal and DEI Considerations

- **Privacy.** The system stores only a name, an email address and a password hash for each user, together with the user's own chats and itineraries. Passwords are stored only as bcrypt hashes. The language model runs locally, so user messages are not sent to third-party AI services.
- **Data isolation.** A user can access only their own chats and itineraries.
- **Content licences.** Place text and images come from open sources. Author and licence are stored with each image, and the credits are recorded. Only openly licensed sources are used.
- **Honest results.** Prices, travel times and opening hours are estimates and are not live. The language model never decides money, time or distance, and its output is validated against the database. In Iteration 3, narration will be based only on stored source text.
- **Inclusion.** The planner supports groups with children or elderly members and users who prefer indoor places and it offers three paces (relaxed, balanced, packed). A current limitation is that the system supports English only.

Chapter 2

Research and Existing Solutions

2.1 Existing Solutions and Products

Several applications help travellers plan trips. Some are international tools that can also be used for Pakistan, and some are built for Pakistan. Table 2.1 summarises the ones reviewed, based on their public descriptions.

Table 2.1: Existing travel applications

Application	**Description**
Wanderlog (1)	International trip planner. Users build an itinerary, budget costs, organise reservations, see places on a map and plan with friends. Plans are stored offline.
Layla (2)	International conversational AI travel agent that helps users discover destinations, build itineraries and plan and book trips. It was acquired by Expedia Group.
Wego (3)	Global booking platform for flights and hotels. It launched its full services in Pakistan with Urdu support and local payment methods.
GOPKZ (4)	AI-powered digital tourism platform for Pakistan, launched in Islamabad in 2026.
Hunza.ai (5)	Itinerary generator for Pakistan. A question-based form collects the destination, start location, trip length, accommodation and trip type, and the itinerary can be downloaded as a PDF.
Trango Trip (6)	AI travel planner for Pakistan that ranks places using reviews and popularity and generates itineraries from them.

Application	Description
Pakrism (7)	Pakistani tour booking app with curated group, private, family and honeymoon packages, each with a day-by-day itinerary and fixed pricing.
FindMyAdventure (8)	Pakistani platform that brings tour companies and agents together, so users can book existing group trips, stays and experiences or build their own trip.

2.2 Comparison of Existing Solutions

The applications are compared with QissaGo on criteria that matter for this project. “Not stated” means that the feature is not described in the sources used.

Table 2.2: Comparison of existing solutions

Application	Pakistan-focused	Conversational input	Itinerary generation	Budget handling	Narration or video
Wanderlog	No	Not stated	Yes	Yes	Not stated
Layla	No	Yes	Yes	Not stated	Not stated
Wego	Yes	Not stated	Not stated	Not stated	Not stated
GOPKZ	Yes	Not stated	Not stated	Not stated	Not stated
Hunza.ai	Yes	Form-based	Yes	Not stated	Not stated
Trango Trip	Yes	Not stated	Yes	Not stated	Not stated
Pakrism	Yes	Not stated	Fixed packages	Package pricing	Not stated
FindMyAdventure	Yes	Not stated	Existing trips or manual	Not stated	Not stated
QissaGo	**Yes**	**Yes**	**Yes**	**Yes**	**Planned**

2.3 Limitations and Gaps

Table 2.3: Limitations of existing solutions

Type of solution	**Limitation**
International planners (Wanderlog, Layla)	Not built for Pakistan. Wanderlog mainly helps users organise a plan they build themselves, and neither tool describes narration at places or a trip video.
Booking platforms (Wego, Pakrism, FindMyAdventure)	Focus on booking flights, hotels and ready-made tour packages. The plan is fixed or chosen by the user and is not built from a conversation about the user's own budget, interests and pace.
Pakistani AI planners (GOPKZ, Hunza.ai, Trango Trip)	Generate itineraries for Pakistan, but their descriptions do not mention editing the plan through chat, narration at stops or an automatic travel video. Hunza.ai collects input through a fixed form.

The gap is that the reviewed solutions cover only parts of the journey. To the best of our knowledge, none combines a conversation that asks for missing details, a feasible plan that respects budget, weather and opening hours, changes to the plan through chat, a narrated story for each place and an automatic travel video for Pakistan in one system.

2.4 Proposed Solution

QissaGo is proposed as a conversation-based travel planner for Pakistan. The user describes the trip in English or uses a quick-select form. A hybrid chatbot, which uses rules first and a small local language model second, turns the description into a structured preference object and asks for anything that is missing or contradictory. A team of specialised agents, coordinated by an orchestrator, then builds three itineraries with different strategies. The calculations are done by algorithms, so the plans respect the budget, the opening hours, the travel time and the weather. The plans are ranked with a match score and shown in a Flutter app with maps, photos and details that the user can show or hide.

The solution uses only free and open parts: Flutter, FastAPI, PostgreSQL, Ollama with an open-source model, Open-Meteo, OpenStreetMap and content from Wikipedia, Wikidata and Wikimedia Commons. The same architecture is extended later with replanning, narration and automatic video generation.

2.5 Complex Computing Problem

The complex computing problem is to turn a natural-language trip request into a feasible itinerary. Several difficulties have to be solved together:

- **Conflicting requirements.** Budget, days, interests, group size, pace and weather pull in different directions. A budget that is too low for meals and a hotel makes the trip impossible.
- **Unstructured input.** Users write dates, money and groups in many forms (“50k”, “1.5 lakh”, “next friday”, “2 adults and 3 kids”), and a small local model can return wrong values.
- **Constrained scheduling.** Places have opening hours, visit durations and locations. Travel time and cost depend on the order of the places and can push a plan over budget.
- **Cooperating components.** Intake, weather, budgeting, scheduling, validation and enrichment must work on one shared state.
- **Limited resources.** Only free and open services, local data and a small local model can be used.

2.6 Technical Challenges

Table 2.4: Technical challenges

Challenge	**Approach**
Reading dates, money, durations and groups from free text	A rule layer with ordered parsing; money is parsed before dates, and the context of the last question is remembered.
Unreliable or unavailable language model	Rules first; the model fills only what the rules miss; its values are validated against the database; if it times out, the system falls back to rules.
Realistic cost of a trip	A cost model for meals, stay, transport and entries computed in code; the minimum trip cost is used to detect budgets that are too low.

Challenge	Approach
Three plans that are different but all valid	Three scoring weightings, a novelty bonus in the Budget Agent, and a Validator Agent that removes invalid items.
Plans that go over budget after travel costs	The Scheduling Agent drops the weakest place when travel costs exceed the budget, and the Validator Agent recomputes the totals.
Place data and images without paid APIs	Ingestion scripts import data from open sources once; images are resized and stored locally with author and licence.
Changing one part of a plan without rebuilding all of it	Detect what changed and re-run only the agents for the affected days.
Narration that stays true to the sources	Ground the narration in stored source text.
Time to render a video	Background jobs from a queue, short templates and compressed media.

2.7 Research Findings and Design Decisions

Table 2.5: Findings and the design decisions they led to

Finding or constraint	Decision
Paid APIs are not allowed, but a language model is still useful for reading free text.	Hybrid chatbot: rules first, and a small local open-source model (Ollama, Qwen 2.5 3B) second.
Budget, time, distance and opening hours must be exact and repeatable.	The language model is used for language only; planning is done by deterministic algorithms.
Planning has several separate concerns.	Independent agents work on a shared state with a trace log.
Open sources provide place text, coordinates and images with licences.	Content is imported once and stored locally with author and licence.
One team has to deliver both a mobile app and a web app.	A single Flutter codebase for Android and web.

Finding or constraint	**Decision**
Users, places and itineraries are related, plans are flexible documents, and retrieval is needed later.	PostgreSQL with JSON columns, and pgvector later.
Video rendering is slow.	Long-running work runs as background jobs through a queue.

Chapter 3

Proposal Feedback and Iteration Plan

3.1 Proposal Feedback Summary

The proposal was approved by the evaluation panel with recommended modifications. In summary, the panel found the idea somewhat vague and the scope too large, and asked for the scope to be reduced. The panel also asked that the personalized itinerary and the personalized video making become the core features of the project, with the other features reduced or removed.

The modifications were grouped into four concerns, each of which was addressed before development began:

- **Unclear modules and vague idea.** The proposal listed many loosely connected modules. The scope was reduced to the core features: personalized conversational itinerary planning with dynamic replanning, and personalized automated travel video generation, supported by cinematic narration. Every remaining module directly supports one of these.
- **Insufficient technical contribution.** The system architecture and its diagrams were worked out properly, so that the role of every component and the way the components work together is clearly understood. The planning problem was formalised as a constrained selection and scheduling problem (budget, time windows, opening hours, travel time, pace and personal constraints) solved by a pipeline of specialised cooperating agents with a dedicated validation step and a quantitative match score. Planned technical work includes grounded narration using retrieval-augmented generation and template-driven video synthesis.
- **Core features.** The personalized itinerary became the whole of Iteration 1, and personalized video making was promoted to a full, dedicated iteration (Iteration 4) instead of a secondary feature.

- **Scope reduction and approval.** Modules not related to the core features were removed or reduced. These are the safety and alert module, predictive disruption forecasting and the personalized recommendation features. The revised scope was presented to and approved by the supervisor.

3.2 Proposal Action Register

Table 3.1 records each piece of feedback, the action taken and where it can be verified.

Table 3.1: Proposal action register

No.	Feedback / Required Change	Action Taken	Status	Verified in
1	The idea is vague and the scope is too large	Scope redefined into core modules, each mapped to an iteration with defined inputs, outputs and agents	Completed	Iteration strategy (Section 3.3)
2	Technical contribution needs to be stronger	Architecture and diagrams worked out; multi-agent constraint-based planner with validation and match scoring designed and implemented; grounded narration and template-driven video synthesis designed	Completed (Iteration 1); planned (Iterations 2, 3 and 4)	GitHub repository: agent code and automated tests
3	Personalized itinerary must be a core feature	Iteration 1 dedicated to personalized itinerary planning: chatbot, preference model, seven planning agents, three ranked variants	Completed	GitHub release v0.1.0; automated tests
4	Personalized video making must be a core feature	Video generation made a complete iteration (Iteration 4) with templates, captions and background rendering	Scheduled for Iteration 4	Iteration 4 plan (Section 3.7)

No.	Feedback / Required Change	Action Taken	Status	Verified in
5	Reduce the scope and improve the idea	Non-core modules removed or reduced: safety and alerts, predictive disruption forecasting and personalized recommendation features	Completed	Iteration roadmap (Table 3.2)
6	Go forward with supervisor's approval	Revised scope, iteration plan and architecture reviewed and approved by the supervisor	Completed	Supervisor review

3.3 Iteration Strategy

The project follows an iterative and incremental development model with four iterations. Each iteration delivers a complete, demonstrable vertical slice of the product, covering user interface, API, agents, database and automated tests, on a single shared codebase. Every iteration ends with a tagged release, passing automated checks and a demonstration to the supervisor. Feedback from one iteration is fed into the plan of the next.

The two core features requested by the panel are the personalized itinerary and the personalized video. The itinerary is built in Iteration 1 and made dynamic in Iteration 2. Cinematic narration is added in Iteration 3, and the video generation in Iteration 4 reuses the itinerary and the narration.

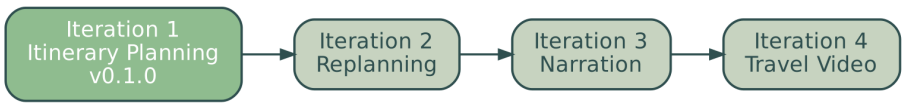


Figure 3.1: Iteration roadmap of QissaGo

Explanation of Figure 3.1:

- The four iterations follow each other: itinerary planning, replanning, narration and travel video.
- Iteration 1 is complete and released as v0.1.0.
- Each iteration builds on the previous one: replanning changes the itinerary, narration adds stories to its stops, and the video uses both the itinerary and the narration.

3.4 Iteration 1 Plan

Goal. A registered user can describe a trip in natural English or through buttons, and receives three feasible, budget-respecting, weather-aware and personalized itineraries that can be inspected, saved and exported.

Table 3.3: Iteration 1 objectives

ID	Objective	Success measure
O1	A user can describe a trip in natural English or with buttons	Destination, dates, duration, budget, group, interests, exclusions, pace and special needs are extracted from free text and from form selections into one shared preference model
O2	Missing or contradictory details are detected and clarified	The chatbot asks one targeted question at a time and rejects past dates, unsupported destinations, group-size conflicts and budgets below the minimum trip cost
O3	Specialised agents produce three ranked itineraries	Every plan stays within budget, has no overlapping visits, respects opening hours and pace, and carries a match score out of 100
O4	Each stop presents its details	Time slot, duration, travel time and distance, cost, weather note, photo, description, opening hours and map links are shown and can be switched on or off
O5	Data is stored persistently per user	Accounts, conversations, messages, itineraries and agent runs are stored in PostgreSQL; users only access their own data
O6	Industry-standard engineering practice	Automated backend and frontend tests, coverage gate, linting, GitHub Actions CI/CD, branch-based workflow

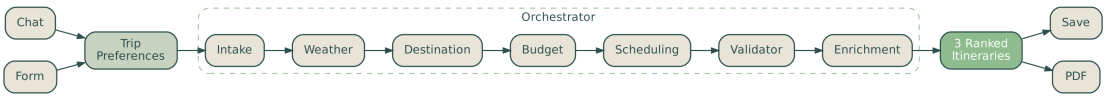


Figure 3.2: Iteration 1: personalized itinerary planning

Explanation of Figure 3.2:

- The trip is described by chat or by the form; both fill the same trip preferences.

- The Orchestrator runs seven agents in order: Intake, Weather, Destination, Budget, Scheduling, Validator and Enrichment.
- Three ranked itineraries are produced, which can be saved or exported as PDF.

Table 3.4: Iteration 1 features and deliverables

Feature	**Deliverable**	**Verified by**
User accounts	Register, log in, log out, persistent session, profile	API tests
Conversational intake	Chatbot with conversation manager, preference extractor, clarification handler and optional local language model	Extractor and conversation tests
Quick-select planning	Form with destination, dates, group, budget, interests, pace, needs, visitor type, home city and travel mode	Widget and API tests
Multi-agent planner	Intake, Weather, Destination, Budget, Scheduling, Validator and Enrichment agents coordinated by the Orchestrator	Agent tests, generation trace
Itinerary presentation	Three ranked options, plan detail with timeline, cost breakdown, map, toggles	Widget tests
Itinerary management	Save, unsave, list saved trips, reopen all options of a generation	API tests
PDF export	Downloadable itinerary document	Manual acceptance test
Place catalogue	Database of destinations, places, interests and media loaded by repeatable scripts	Data script tests
CI/CD	Backend CI, frontend CI, deployment and APK release workflows	GitHub Actions workflow runs

3.5 Iteration 2 Plan

Goal. A generated itinerary becomes a living plan that can be changed through conversation.

Table 3.5: Iteration 2 objectives and deliverables

ID	Objective / Feature	Deliverable
O7	Edit an itinerary through chat	Replanning Agent that understands add, remove, move, start-later and change-day requests and rebuilds only the affected days; versioned itineraries.
O8	Weather-triggered replanning	Suggestion to swap outdoor and indoor stops when the forecast changes.
O9	Infrastructure for replanning	Redis cache for sessions and API responses.

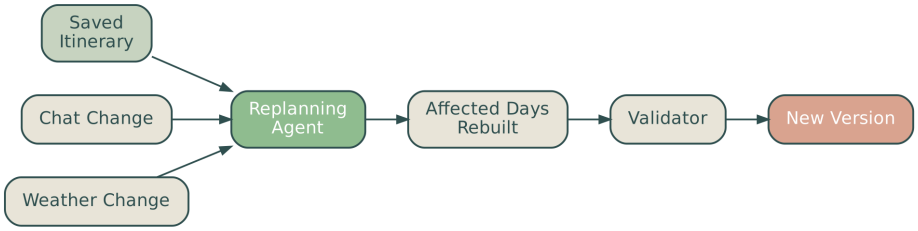


Figure 3.3: Iteration 2: dynamic replanning

3.6 Iteration 3 Plan

Goal. Each selected stop of the itinerary tells its own story in a cinematic narration.

Table 3.6: Iteration 3 objectives and deliverables

ID	Objective / Feature	Deliverable
O10	Cinematic narration	Narration Agent that writes a storytelling script for each stop in a selected persona, grounded in stored source text.
O11	Audio narration	Voice Agent converting scripts to speech with a free text-to-speech engine; in-app player.
O12	Questions about a stop	Stop Q&A Agent using retrieval over source text with a vector search extension in PostgreSQL; answers “not available” when the sources do not contain the answer.

Explanation of Figure 3.4:

Table 3.2: Iteration roadmap

Iteration	Theme	Main deliverables	Status
1	Personalized conversational itinerary planning	Accounts; chatbot and quick-select form; seven planning agents with orchestrator; three ranked itineraries with match score; stop details; saved trips; PDF export; Android and web app; CI/CD	Completed (v0.1.0)
2	Dynamic replanning	Itinerary editing through chat; weather-triggered replanning	Planned
3	Cinematic narration	Persona-based storytelling narration at stops in text and audio; questions about a stop answered from stored sources	Planned
4	Personalized automated travel video	Video templates; caption and voiceover per stop; automatic assembly of images, clips and music; background rendering; final deployment and user evaluation	Planned

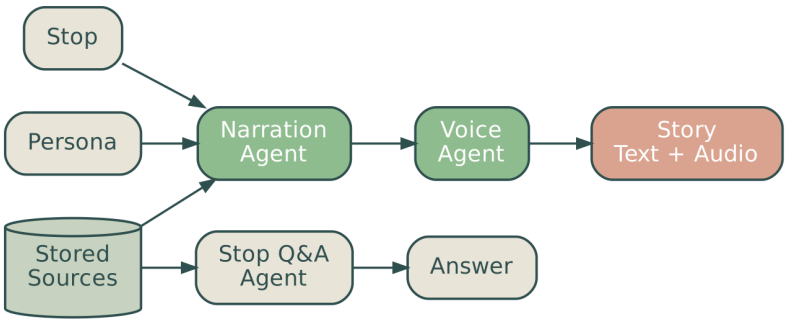


Figure 3.4: Iteration 3: cinematic narration

- The Narration Agent uses the stop, the chosen persona and the stored sources to write a story.
- The Voice Agent turns the story into audio.
- The Stop Q&A Agent answers questions about a stop from the same sources.

3.7 Iteration 4 Plan

Goal. A finished trip is turned into a personalized travel video.

Table 3.7: Iteration 4 objectives and deliverables

ID	Objective / Feature	Deliverable
O13	Video templates	Selectable templates, for example a highlight reel or a day-by-day recap, each with its own pacing and style.
O14	Captions and voiceover	Caption Agent that generates a caption or voiceover line for each stop, consistent with the narration.
O15	Automatic video assembly	Video Agent that assembles images, clips and music of the stops into the chosen template, with rendering as background jobs through a queue.
O16	Final deployment and user evaluation	Deployed system and a usability study with users.

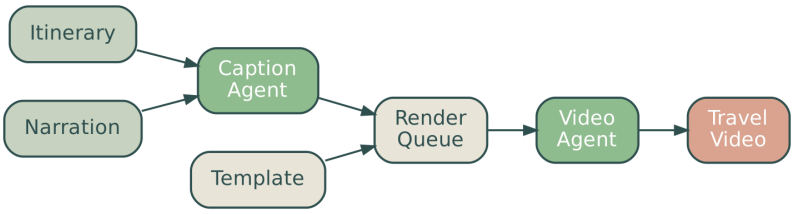


Figure 3.5: Iteration 4: personalized travel video

Explanation of Figure 3.5:

- The Caption Agent writes a caption for each stop from the itinerary and its narration.
- The chosen template and captions go to a render queue.
- The Video Agent assembles the travel video in the background.

3.8 Work Breakdown and Responsibilities

Work is divided by layer so that each member owns a complete part of the system, while testing, reviews and documentation are shared. Every change is reviewed by another member through a pull request.

Table 3.8: Work breakdown and responsibilities

Member	**Role**	**Iteration 1 responsibilities**	**Iterations 2 to 4**
Dania Waseem	Backend and DevOps	REST API, authentication, database schema, ingestion and seed scripts, Docker setup, CI/CD workflows, deployment	Redis, job queue, video rendering worker, deployment
Munaza Tariq	Agents and conversational intelligence	Conversation manager, preference extractor, clarification handler, language model integration, planning agents, orchestrator, agent tests	Replanning, Narration, Q&A, Caption agents; retrieval
Wajeeha Khalid	Mobile and web application	Flutter screens, navigation, state management, maps, cost breakdown, PDF export, responsive layout, widget tests	Narration player, video templates and player, usability study
All members	Shared	Test design, code reviews, documentation, report, supervisor meetings	Same

3.9 Progress Against Plan

All Iteration 1 milestones were achieved, as shown in Table 3.9.

Table 3.9: Iteration 1 milestones

No.	**Milestone**	**Result**
M1	Revised scope approved by supervisor	Achieved
M2	Repository, environment, database and CI set up	Achieved
M3	Place catalogue and ingestion scripts	Achieved
M4	Authentication API and screens	Achieved
M5	Chatbot: extractor, clarification, conversation manager	Achieved
M6	Planning agents and orchestrator	Achieved
M7	Itinerary screens, saving, PDF export	Achieved
M8	Testing, coverage gate, documentation	Achieved

No.	Milestone	Result
M9	Iteration 1 release (v0.1.0) and demonstration	Achieved

3.10 Delays, Deviations and Recovery Plan

Iteration 1 was completed within its planned time box. The following deviations from the proposal were made deliberately, each for a technical reason:

Table 3.10: Deviations from the proposal

No.	Proposal	Change	Reason and impact
1	Separate mobile and web frontends	Single Flutter codebase for Android and web	One user interface and logic layer; halves frontend effort; consistent behaviour on both platforms
2	Hosted language model API	Rule-based extraction plus a free, locally hosted open-source model (Ollama)	Paid APIs are not permitted; local model keeps user messages private and works offline; rules keep the chatbot functional without the model
3	Redis in the first release	Redis moved to Iteration 2	Iteration 1 has no background jobs; chat state is stored in PostgreSQL. Redis is introduced when replanning and rendering jobs need it
4	Review score as a stop attribute	Moved to a later iteration	Not required by the core features; avoids dependence on review platforms
5	Safety alerts, disruption prediction and forecasting	Removed	Not part of the two core features identified by the panel; time reallocated to video generation

Recovery plan. To protect the remaining schedule, each iteration is time-boxed and divided into small issues on the project board. If an iteration risks overrunning, lower-priority items (for example, additional narration personas or extra video templates) are moved to the end of FYP-2 while the core flow of each iteration is delivered first. Weekly supervisor meetings are used to detect slippage early.

Chapter 4

Software Requirements

This chapter defines what QissaGo must do and how well it must do it. Requirements are grouped by the modules of Table 1.2 (Section 1.6). The following techniques are used:

- a use case diagram and an event response table for user interactions (Sections 4.1 and 4.2);
- detailed use cases for the main flows (Section 4.3);
- numbered functional requirements per module (Section 4.4);
- quantitative and verifiable non-functional requirements (Section 4.5);
- acceptance criteria and a traceability matrix (Sections 4.6–4.7).

Each requirement is tagged with the iteration that delivers it: **It. 1** (complete), **It. 2** re-planning, **It. 3** narration, **It. 4** video.

4.1 User Interactions / Use Cases

4.1.1 Actors

Table 4.1: Actors of QissaGo

Actor	**Type**	**Role**
Traveller	Primary	Registered user who plans, views, saves, edits, listens to and turns trips into videos

Actor	Type	Role
Guest	Primary	Visitor who can only register or log in
System (QissaGo Agents)	Internal	Orchestrator and specialised agents that generate itineraries, suggest replanning and render videos
Local Language Model	Internal	Open-source model (Ollama) used for reading free text and writing narration
Weather Service	External	Open-Meteo forecasts and climate data
Map Service	External	OpenStreetMap tiles, map and route links

4.1.2 Use case list

Table 4.2: Use cases

ID	Use case	Module	It.	Actor
UC-01	Register	Authentication	1	Guest
UC-02	Log in / log out	Authentication	1	Guest, Traveller
UC-03	View profile	Authentication	1	Traveller
UC-04	Plan trip by chat	Chatbot	1	Traveller
UC-05	Answer clarifying question	Chatbot	1	Traveller
UC-06	Plan trip by quick-select form	Quick-select form	1	Traveller
UC-07	Compare itinerary options	Itinerary viewing	1	Traveller
UC-08	View itinerary details	Itinerary viewing	1	Traveller
UC-09	Show / hide stop details	Itinerary viewing	1	Traveller
UC-10	Open map and route links	Itinerary viewing	1	Traveller
UC-11	Save / unsave itinerary	Itinerary management	1	Traveller
UC-12	View saved trips	Itinerary management	1	Traveller
UC-13	Export itinerary PDF	Itinerary management	1	Traveller
UC-14	Edit itinerary through chat	Replanning	2	Traveller
UC-15	View itinerary versions	Replanning	2	Traveller
UC-16	Choose narrator persona	Narration	3	Traveller

ID	Use case	Module	It.	Actor
UC-17	Listen to stop narration	Narration	3	Traveller
UC-18	Ask about a stop	Narration	3	Traveller
UC-19	Choose video template	Video	4	Traveller
UC-20	Generate travel video	Video	4	Traveller
UC-21	Download / share video	Video	4	Traveller
UC-22	Generate itineraries	Multi-agent planner	1	System (QissaGo Agents)
UC-23	Suggest weather replanning	Replanning	2	System (QissaGo Agents)
UC-24	Render video in background	Video	4	System (QissaGo Agents)
UC-25	Provide weather forecast	Multi-agent planner	1	Weather Service
UC-26	Provide maps and routes	Itinerary viewing	1	Map Service
UC-27	Interpret text / write narration	Chatbot, Narration	1, 3	Local Language Model

4.1.3 Use-case / event response table

For each user action, Table 4.3 gives the state before the action, the system response and the resulting state.

Table 4.3: Use-case / event response table

User action	Pre-event state	System response	Post-event state
Guest registers	Register screen open	Validates input, checks email, hashes password, issues token	Account created, home screen shown
Traveller logs in	Login screen open	Verifies password, issues signed token	Logged in; token stored on device
Traveller opens chat	Logged in	Creates chat session, greets by name, offers options	Session waiting for first message
Traveller sends trip message	Chat session open	Extracts details (rules, then model), merges, checks contradictions	Next question or summary shown

User action	**Pre-event state**	**System response**	**Post-event state**
Traveller taps a suggestion	Question shown	Applies the selection to the same preferences	Next question or summary shown
Traveller confirms summary	All details known	Starts itinerary generation	Progress screen shown
Traveller submits form	Form completed	Validates preferences, starts generation	Progress screen shown
System generates plans	Valid preferences	Runs seven agents, scores and ranks three plans, stores plans and trace	Three ranked options shown
Traveller opens a plan	Options shown	Loads timeline, stop details, cost breakdown, map	Plan detail shown
Traveller toggles a detail	Plan detail shown	Shows or hides that detail type	Layout updated
Traveller saves plan	Plan open	Marks plan as saved	Plan listed in Saved tab
Traveller exports PDF	Plan open	Builds PDF of summary, costs and days	PDF downloaded or shared
Traveller opens map link	Plan open	Opens place or day route in map service	External map opened
Traveller requests a change (It. 2)	Plan open	Detects change and affected days, re-runs agents for them	New version shown
Forecast changes (It. 2)	Saved plan with outdoor stops	Suggests indoor/outdoor swap	Suggestion awaiting answer
Traveller opens narration (It. 3)	Stop selected	Loads or creates persona narration from stored sources, adds audio	Narration shown and played
Traveller requests video (It. 4)	Plan open, template chosen	Queues rendering job, writes captions, renders video	Video ready for playback
Traveller asks about a stop (It. 3)	Stop selected	Retrieves stored sources and answers, or replies “not available”	Answer shown
System (QissaGo Agents) generates itineraries	Preferences confirmed	Orchestrator runs all agents, requests forecast from Weather Service	Plans and agent runs stored

User action	Pre-event state	System response	Post-event state
Weather Service provides forecast	Forecast requested	Returns daily temperature and rain probability; cached for one hour	Weather used by agents
Server unreachable	Any screen	Shows error with retry	Request can be repeated

4.2 Use Case Diagram

The use case diagram gives an overview of QissaGo's capabilities and shows how each actor interacts with the system. Figure 4.1 covers all four iterations.

Explanation of Figure 4.1:

- **Guest** can only register and log in.
- **Traveller** plans trips by chat or form, views, compares, saves and exports itineraries (Iteration 1), edits them and views versions (Iteration 2), listens to narration and asks about stops (Iteration 3), and creates, downloads and shares travel videos (Iteration 4).
- **System (QissaGo Agents)** performs the automated use cases: generating itineraries, suggesting weather replanning and rendering videos in the background.
- **Generate itineraries** includes **provide weather forecast**, which is supplied by the external **Weather Service**.
- **Map Service** provides maps and routes; the **Local Language Model** interprets chat text and writes narration.

4.3 Detailed Use Cases

Table 4.4: UC-01 Register

Use case	UC-01 Register
Actors	Guest
Pre-conditions	Not logged in

Main flow	1. Opens Register. 2. Enters name, email, password. 3. System validates, checks email is unused, hashes password, returns token. 4. Home screen opens.
Alternative flows	3a. Email in use: error shown. 3b. Invalid input: field highlighted.
Post-conditions	Account exists; user logged in
Iteration	1
Requirements	FR-1.1, FR-1.2, NFR-S1

Table 4.5: UC-04 Plan trip by chat

Use case	UC-04 Plan trip by chat
Actors	Traveller, Local Language Model
Pre-conditions	Logged in
Main flow	1. Opens chat; bot greets by name. 2. Types trip details. 3. System extracts and merges details, acknowledges them. 4. Bot asks the next missing detail with suggestions. 5. Steps 2–4 repeat. 6. Bot shows summary; traveller confirms. 7. UC-22 runs.
Alternative flows	3a. Contradiction found: UC-05. 3b. Unsupported destination: supported list offered. 3c. Not understood: question repeated. 6a. “change budget”: that detail is asked again.
Post-conditions	Complete preferences stored with all messages
Iteration	1
Requirements	FR-2.1 – FR-2.10

Table 4.6: UC-06 Plan trip by quick-select form

Use case	UC-06 Plan trip by quick-select form
Actors	Traveller

Pre-conditions	Logged in
Main flow	1. Opens form (or taps a destination on Home). 2. Selects destination, date, days, group, budget, interests, exclusions, pace, needs, visitor type, home city, travel mode. 3. Presses Create plans. 4. UC-22 runs.
Alternative flows	3a. Required field missing: button disabled.
Post-conditions	Preferences submitted
Iteration	1
Requirements	FR-3.1 – FR-3.3

Table 4.7: UC-22 Generate itineraries

Use case	UC-22 Generate itineraries
Actors	System (QissaGo Agents), Weather Service
Pre-conditions	Complete preferences
Main flow	1. Progress screen opens. 2. Orchestrator runs Intake, Weather, Destination, Budget, Scheduling, Validator, Enrichment. 3. Plans scored and ranked; best labelled. 4. Plans and agent trace stored. 5. Options shown (UC-07).
Alternative flows	2a. Weather unavailable or beyond forecast range: typical climate used. 2b. Over budget after travel: weakest stop dropped, day rebuilt. 1a. Server unreachable: error with retry.
Post-conditions	Three itineraries and agent runs stored
Iteration	1
Requirements	FR-4.1 – FR-4.12

Table 4.8: UC-08 View itinerary details

Use case	UC-08 View itinerary details
Actors	Traveller, Map Service
Pre-conditions	Plan generated or saved

Main flow	1. Opens a plan. 2. Sees score, total, budget left, cost breakdown, timeline per day. 3. Each stop shows time, duration, travel leg, cost, weather note, photo, description, opening hours. 4. Opens day map or links.
Alternative flows	2a. Wide screen: costs beside timeline.
Post-conditions	None
Iteration	1
Requirements	FR-5.1 – FR-5.6

Table 4.9: UC-13 Export itinerary PDF

Use case	UC-13 Export itinerary PDF
Actors	Traveller
Pre-conditions	Plan open
Main flow	1. Presses Export. 2. System builds PDF. 3. File downloaded (web) or shared/printed (Android).
Alternative flows	2a. Image missing: stop printed without photo.
Post-conditions	PDF available
Iteration	1
Requirements	FR-5.9

Table 4.10: UC-14 Edit itinerary through chat

Use case	UC-14 Edit itinerary through chat
Actors	Traveller, Local Language Model
Pre-conditions	Plan open
Main flow	1. Writes a change (remove, add, move, start later). 2. Replanning Agent finds change and affected days. 3. Only those days are rebuilt and validated. 4. New version stored and shown.
Alternative flows	2a. Unclear: bot asks. 3a. Breaks budget or hours: bot explains and offers alternatives.

Post-conditions	New version; old versions kept
Iteration	2
Requirements	FR-7.1 – FR-7.5

Table 4.11: UC-17 Listen to stop narration

Use case	UC-17 Listen to stop narration
Actors	Traveller, Local Language Model
Pre-conditions	Plan open, persona chosen
Main flow	1. Opens a stop. 2. Narration Agent loads cached script or writes one from stored sources. 3. Voice Agent creates audio. 4. App shows text and plays audio.
Alternative flows	2a. No sources: short description shown.
Post-conditions	Narration cached
Iteration	3
Requirements	FR-8.1 – FR-8.6

Table 4.12: UC-20 Generate travel video

Use case	UC-20 Generate travel video
Actors	Traveller, System (QissaGo Agents)
Pre-conditions	Plan open
Main flow	1. Chooses template (UC-19). 2. System queues job. 3. Caption Agent writes captions; Video Agent selects media and renders. 4. App shows progress, then the video.
Alternative flows	3a. Rendering fails: job marked failed, retry offered.
Post-conditions	Video stored; can be downloaded (UC-21)
Iteration	4
Requirements	FR-9.1 – FR-9.6



Figure 4.1: Use case diagram for QissaGo

4.4 Functional Requirements

Functional requirements are organised by the modules of Section 1.6. Each is numbered so it can be traced to design and tests (Section 4.7).

4.4.1 Module 1: Authentication

Following are the requirements for the Authentication module (It. 1):

FR-1.1 The system shall let a guest register with name, email and password.

FR-1.2 The system shall reject a registration whose email is already registered.

FR-1.3 The system shall let a user log in and issue a signed, expiring token.

FR-1.4 The app shall keep the user logged in across restarts until logout or expiry.

FR-1.5 The system shall let the user view the profile and log out.

FR-1.6 The system shall give each user access to their own chats and itineraries only.

4.4.2 Module 2: Chatbot (Intake)

Following are the requirements for the Chatbot module (It. 1):

FR-2.1 The system shall greet the user by name and offer suggested starting options.

FR-2.2 The system shall extract the destination, including aliases and misspellings.

FR-2.3 The system shall extract dates in relative and absolute forms (“tomorrow”, “next friday”, “20 dec”, “25/12”, ranges).

FR-2.4 The system shall extract the duration in days, nights or weeks (up to seven days).

FR-2.5 The system shall extract budgets in local formats (“50k”, “Rs 40,000”, “1.5 lakh”, “per person”).

FR-2.6 The system shall extract group type and size, interests, exclusions, excluded places, pace and special needs.

FR-2.7 The system shall ask for one missing detail at a time, in a fixed order, with suggested answers.

FR-2.8 The system shall detect past dates, unsupported destinations, group-size conflicts and budgets below the minimum trip cost.

FR-2.9 The system shall generate plans only after the user confirms a summary.

FR-2.10 The system shall use the local model only for details the rules missed and shall validate its output.

FR-2.11 The system shall store every message and the current preferences.

4.4.3 Module 3: Quick-select Form

Following are the requirements for the Quick-select Form module (It. 1):

FR-3.1 The app shall offer a form for all trip details as buttons and fields.

FR-3.2 The form shall include visitor type, home city and travel mode.

FR-3.3 The form shall fill the same preference model as the chatbot.

4.4.4 Module 4: Multi-agent Planner

Following are the requirements for the Multi-agent Planner module (It. 1):

FR-4.1 The system shall generate three itineraries: Balanced Explorer, Interest Deep-Dive and Smart Saver.

FR-4.2 The system shall use the daily forecast within the forecast range and typical climate beyond it.

FR-4.3 The system shall score places by interests, priority and cost and remove excluded interests and places.

FR-4.4 The system shall adapt plans to children, elderly travellers, indoor preference, late start and no-accommodation needs.

FR-4.5 The system shall keep every plan within budget, covering entries, local travel, meals, stay, road trip from the home city and vehicle hire.

FR-4.6 The system shall group places into days and order them by distance and best visiting time.

FR-4.7 The system shall schedule visits inside opening hours and finish every day by a fixed time.

FR-4.8 The system shall respect the stops per day and start time of the selected pace.

FR-4.9 The system shall validate every plan and remove invalid items before showing it.

FR-4.10 The system shall give each plan a match score out of 100 and rank the plans.

FR-4.11 The system shall record each agent's message and duration and show progress while planning.

FR-4.12 The system shall store generated itineraries for the user.

4.4.5 Module 5: Itinerary Viewing and Management

Following are the requirements for the Itinerary Viewing and Management module (It. 1):

FR-5.1 The app shall list the three options with score, total cost and statistics.

FR-5.2 The app shall show a timeline per day with time slot, duration and travel leg.

FR-5.3 The app shall show each stop's cost, weather note, photo, description and opening hours.

FR-5.4 The app shall let the user show or hide each type of stop detail.

FR-5.5 The app shall show a cost breakdown: entries, local travel, meals, stay, road trip, vehicle hire.

FR-5.6 The app shall show a day map and give map and route links.

FR-5.7 The user shall be able to save and unsave an itinerary.

FR-5.8 The app shall list saved itineraries and reopen all options generated together.

FR-5.9 The app shall export an itinerary as a PDF.

FR-5.10 The app shall show upcoming features (narration, video) in the navigation.

4.4.6 Module 6: Content and Data

Following are the requirements for the Content and Data module (It. 1):

FR-6.1 The system shall store destinations, places, interests and their links in the database.

FR-6.2 Ingestion scripts shall import descriptions, coordinates and openly licensed images once and store them locally.

FR-6.3 Each image shall be stored with its author and licence, and credits shall be shown.

FR-6.4 A seed script shall load the catalogue repeatably.

FR-6.5 Weather responses shall be cached.

4.4.7 Module 7: Replanning

Following are the requirements for the Replanning module (It. 2):

FR-7.1 The system shall accept change requests in chat: add, remove, move, start later, change day.

FR-7.2 The system shall rebuild only the affected days and keep other days unchanged.

FR-7.3 The changed plan shall still respect budget, opening hours and pace.

FR-7.4 The system shall store every version of an itinerary.

FR-7.5 The system shall suggest changes when the forecast makes an outdoor stop unsuitable.

4.4.8 Module 8: Narration

Following are the requirements for the Narration module (It. 3):

FR-8.1 The system shall turn stored source text about a stop into a storytelling narration script.

FR-8.2 The user shall be able to choose between at least two narrator personas.

FR-8.3 The narration shall use only stored source material.

FR-8.4 The narration shall be shown and played as audio at the relevant stop.

FR-8.5 Narrations and audio shall be cached for reuse.

FR-8.6 The system shall answer questions about a stop from stored sources and reply “not available” when the sources do not contain the answer.

4.4.9 Module 9: Video

Following are the requirements for the Video module (It. 4):

FR-9.1 The app shall offer at least two video templates (e.g. highlight reel, day-by-day recap) with their own pacing and style.

FR-9.2 The system shall write a caption for each stop, consistent with the narration.

FR-9.3 The system shall assemble images, clips, captions, voiceover and music into the chosen template.

FR-9.4 Rendering shall run as a background job with visible progress.

FR-9.5 The user shall be able to play, download and share the video.

FR-9.6 The video shall follow the user's own itinerary (stops, order, days).

4.5 Non-Functional Requirements

Non-functional requirements are specific, measurable and verifiable. The verification method is given in brackets.

4.5.1 Reliability

NFR-R1 The chatbot shall remain fully usable when the language model is unavailable. (tests with model disabled)

NFR-R2 Planning shall continue when the weather service is unreachable, using typical climate. (fallback test)

NFR-R3 No plan shall be shown with overlapping visits, visits outside opening hours or a total above budget. (Validator, agent tests)

NFR-R4 The same preferences and weather shall always produce the same plans. (deterministic agents)

NFR-R5 The app shall show an error with a retry option when the server is unreachable. (TC-M8)

NFR-R6 All errors shall be logged on the server. (log review)

4.5.2 Usability

NFR-U1 A first-time user shall create an itinerary without instructions. (usability study, It. 4)

NFR-U2 Every chatbot question shall offer tappable suggested answers. (conversation tests)

NFR-U3 A plan shall be created in at most 3 screens from Home using the form. (manual check)

NFR-U4 Every screen shall offer a way back and to Home. (navigation tests)

NFR-U5 The interface shall use short labels and imagery, no emojis or instructional text. (design review)

NFR-U6 The System Usability Scale score shall be at least 68. (usability study, It. 4)

4.5.3 Performance

NFR-P1 Generating three itineraries shall take at most 3 seconds on the reference machine. (agent trace durations)

NFR-P2 A chatbot reply shall take at most 1 second with rules only and at most 8 seconds with the local model. (timed API calls)

NFR-P3 Weather for the same location shall be cached for one hour. (weather tests)

NFR-P4 Place images shall be served locally and cached by the app. (static files, image cache)

NFR-P5 A cached narration shall load in at most 2 seconds. (It. 3, timing tests)

NFR-P6 A one-minute video shall render in at most 5 minutes in the background. (It. 4, render tests)

4.5.4 Security

NFR-S1 Passwords shall be stored only as salted bcrypt hashes. (code review)

NFR-S2 All protected endpoints shall require a valid, signed token that expires after 72 hours. (API tests)

NFR-S3 A user shall not read or change another user's data. (API test returns 404)

NFR-S4 All input shall be validated for type, length and range. (schema validation)

NFR-S5 Secrets shall be kept in environment files excluded from version control. (repository check)

NFR-S6 User messages shall not be sent to third-party language model services. (local model)

NFR-S7 Data between app and server shall travel over HTTPS in deployment. (deployment check)

4.5.5 Scalability

NFR-SC1 New destinations and places shall be added through seed files without code changes.

NFR-SC2 New agents shall be added to the pipeline without changing existing agents.

NFR-SC3 Long-running work (video) shall run in separate workers that can be scaled independently.

4.5.6 Compatibility

NFR-C1 The app shall run on Android 8.0 or later.

NFR-C2 The web app shall run in current Chrome, Edge and Firefox.

NFR-C3 The layout shall adapt from 320 px phone width to wide desktop screens.

4.5.7 Maintainability

NFR-M1 Backend test coverage shall stay above 70%. (CI gate)

NFR-M2 Code shall pass linting, formatting and static analysis before merging. (CI)

NFR-M3 Each agent shall be testable without a database. (agent unit tests)

NFR-M4 The API shall have generated interactive documentation. (`/docs`)

NFR-M5 Only free and open services and tools shall be used. (dependency review)

4.6 Acceptance Criteria

Table 4.13: Acceptance criteria

ID	Acceptance check	Req.	Test
AC-01	Duplicate email is rejected; new email logs the user in	FR-1.1, 1.2	TC-36
AC-02	User is still logged in after reopening the app	FR-1.4	TC-M1
AC-03	One message with destination, days, budget and interests fills all four	FR-2.2–2.6	TC-01–16
AC-04	“1.5 lakh” is read as 150,000, never as a date	FR-2.5	TC-06, TC-11
AC-05	Past date and too-low budget are rejected with a reason	FR-2.8	TC-22, TC-23
AC-06	Plans are generated only after confirmation	FR-2.9	TC-21
AC-07	All chatbot tests pass with the model disabled	NFR-R1	TC-18
AC-08	Three plans returned, ranked, best labelled	FR-4.1, 4.10	TC-28
AC-09	For all trip profiles: within budget, no overlaps, within opening hours and pace	FR-4.5–4.9	TC-28
AC-10	Excluded interest or place never appears	FR-4.3	TC-30
AC-11	With late start, no stop before 11:00	FR-4.4	TC-33
AC-12	Cost parts include road trip and vehicle hire and add up to the total	FR-4.5, 5.5	TC-49–53, TC-61
AC-13	Saved plan appears in Saved tab and disappears after unsave	FR-5.7, 5.8	TC-39, TC-64
AC-14	Another user’s itinerary returns “not found”	FR-1.6, NFR-S3	TC-42
AC-15	PDF contains summary, costs and all days	FR-5.9	TC-M7
AC-16	Coverage above 70% and all CI checks green	NFR-M1, M2	CI run
AC-17	After an edit, unaffected days are identical (It. 2)	FR-7.2	planned
AC-18	Narration contains no facts absent from the sources (It. 3)	FR-8.3	planned
AC-19	Video shows the plan’s stops in order with captions (It. 4)	FR-9.6	planned

4.7 Requirements Traceability

Table 4.14: Requirements traceability matrix

Req.	Module	Design element	Implementation	Tests
FR-1.1–1.3	Authentication	Auth API, users table	`api/auth.py`, `core/security.py`	TC-36, 37
FR-1.4	Authentication	App state	`core/app_state.dart`	TC-M1
FR-1.6	Authentication	Auth dependency	`api/deps.py`	TC-42
FR-2.2–2.6	Chatbot	Preference extractor	`agents/intake/extractor.py`	TC-01–18
FR-2.7–2.9	Chatbot	Clarifier, conversation manager	`intake/clarifier.py`, `intake/conversation.py`	TC-19–27
FR-2.10	Chatbot	LLM client	`services/llm.py`	TC-17, 18
FR-3.1–3.3	Form	Quick-select screen	`screens/quick_plan_screen.dart`	TC-40, 62
FR-4.1, 4.10	Planner	Orchestrator	`agents/orchestrator.py`	TC-28, 29
FR-4.2	Planner	Weather Agent	`agents/weather.py`, `services/weather.py`	TC-28
FR-4.3, 4.4	Planner	Destination Agent	`agents/destination.py`	TC-30, 31, 33
FR-4.5	Planner	Budget Agent, cost rules	`agents/budget.py`, `agents/costs.py`	TC-32, 49–53
FR-4.6–4.8	Planner	Scheduling Agent	`agents/scheduling.py`	TC-28, 33
FR-4.9	Planner	Validator Agent	`agents/validator.py`	TC-28
FR-5.1–5.6	Itinerary	Plan screens	`screens/plans_screen.dart`, `plan_detail_screen.dart`	TC-45–48, 60, 61
FR-5.7, 5.8	Itinerary	Itinerary API	`api/itineraries.py`	TC-39, 64
FR-5.9	Itinerary	PDF builder	Flutter PDF export	TC-M7

Req.	**Module**	**Design element**	**Implementation**	**Tests**
FR-6.1–6.4	Content	Catalogue, scripts	`scripts/fetch_wikipedia.py`, `seed_db.py`	TC-43, 44, 54–56
FR-7.x	Replanning	Replanning Agent	planned (It. 2)	planned
FR-8.x	Narration	Narration, Voice, Q&A agents	planned (It. 3)	planned
FR-9.x	Video	Caption, Video agents	planned (It. 4)	planned

Chapter 5

System Design

QissaGo is a conversational travel planner built around the complete travel experience. The design covers the whole system across all four iterations: a single Flutter application for Android and the web, a FastAPI backend, a chatbot that turns natural language into structured trip preferences, a team of specialised agents coordinated by an Orchestrator that builds, checks and later changes itineraries, narration and video agents for the story-telling features, and a PostgreSQL database. Only free and open services are used. Exact quantities such as money, time, distance and opening hours are computed by algorithms, while the local language model is used only for language.

5.1 System Architecture

The architecture follows a four-tier, layered client–server design with a multi-agent core. Each tier has a clear responsibility and communicates with neighbouring tiers through defined interfaces, so components can be developed, tested and deployed independently.

5.1.1 Box and Line Diagram

The box and line diagram gives a simplified, high-level view of QissaGo's major subsystems and how they are connected.

As shown in Figure 5.1, the system is organised around seven core components:

- **Flutter App:** the entry point for all users on Android and the web; provides the chat, the quick-select form, itinerary views, maps, PDF export and, later, the narration and video players.
- **REST API:** receives every request, validates input, checks the login token and

passes the request to the right component.

- **Chatbot (Intake):** reads the traveller's messages and turns them into structured trip preferences, asking for anything missing or contradictory.
- **Multi-Agent Planner:** the Orchestrator and the planning agents that build, check and rank three itineraries.
- **Replanning, Narration and Video Agents:** change itineraries, tell the story of each stop and assemble the travel video (Iterations 2–4).
- **Database and Media Storage:** store users, chats, places, itineraries, agent runs, photos, audio and videos.
- **External Services:** the weather service, the map service and the locally hosted language model.

The arrows show data flow. The app talks only to the API; the API calls the chatbot, the planner and the later agents; every component reads and writes through the database.

5.1.2 Four-Tier Architecture

QissaGo's architecture is structured into four tiers: Presentation, Application, Integration and Data.

5.1.2.1 Presentation Layer

The Presentation Layer is the user-facing Flutter application. It is responsible for:

- **User Interaction:** trip input through chat messages, suggestion chips and the quick-select form.
- **Response Rendering:** ranked itinerary options, day timelines, cost breakdowns, maps, PDF export, and later narration and videos.
- **Navigation:** a shell with Home, Plan, Saved, Narration and Video tabs on phones and wide screens.
- **Session Management:** storing the login token on the device and sending it with every request.

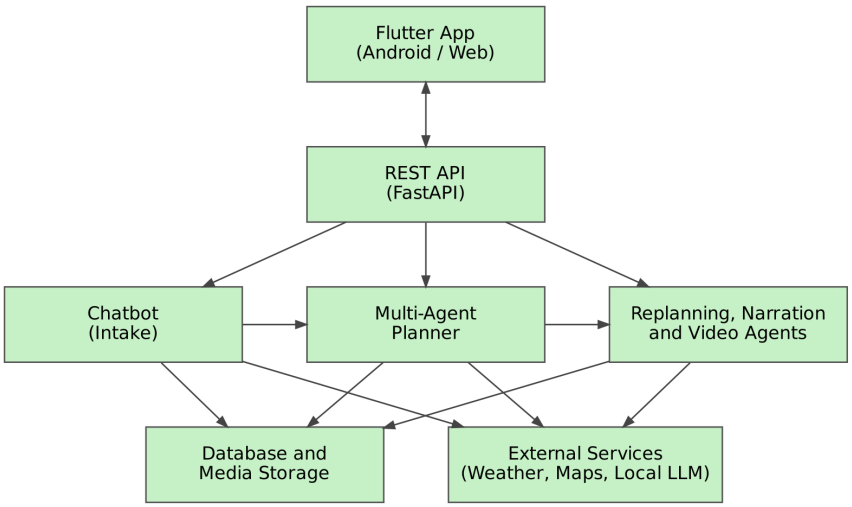


Figure 5.1: Box and Line Diagram of QissaGo

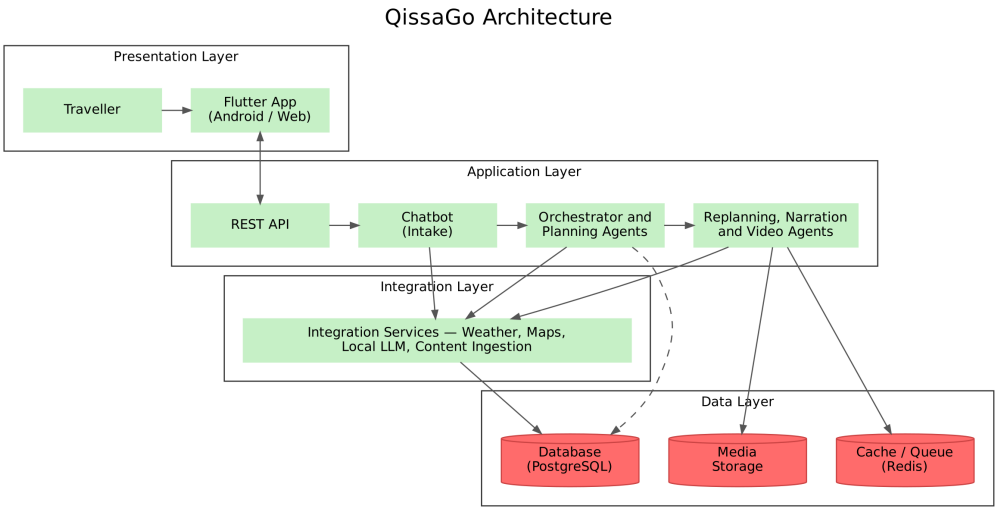


Figure 5.2: Four-Tier Architecture of QissaGo

5.1.2.2 Application Layer

The Application Layer contains the core logic:

- **REST API (FastAPI):** routes for authentication, planning options, chat and itineraries; later replanning, narration and video; input validation and token checks.
- **Chatbot (Intake):** conversation manager, rule-based preference extractor with optional local model, and clarification handler.
- **Orchestrator and Planning Agents:** Intake, Weather, Destination, Budget, Scheduling, Validator and Enrichment agents that produce three ranked itineraries.
- **Replanning, Narration and Video Agents:** Replanning Agent (Iteration 2); Narration, Voice and Stop Q&A agents (Iteration 3); Caption and Video agents (Iteration 4).

5.1.2.3 Integration Layer

The Integration Layer connects the application to external and local services:

- **Weather:** daily forecasts and climate data from Open-Meteo, cached for one hour, with a climate fallback.
- **Maps:** OpenStreetMap tiles, place links and route links.
- **Local Language Model:** Ollama serving an open-source model for reading free text and writing narration.
- **Content Ingestion:** one-time import of descriptions, coordinates and openly licensed images from Wikipedia, Wikidata and Wikimedia Commons.
- **Media Rendering (Iteration 4):** FFmpeg-based video assembly in a background worker.

5.1.2.4 Data Layer

- **Database (PostgreSQL):** users, catalogue of destinations and places, chats, itineraries and agent runs; JSON columns for flexible plan documents; a vector extension for narration sources later.
- **Media Storage:** place photos, narration audio and rendered videos.
- **Cache and Queue (Redis, Iteration 2 onward):** caching and the job queue for background rendering.

5.1.3 Architecture Benefits

- **Modularity:** each tier and each agent can be developed and tested on its own.
- **Correctness:** budget, time, distance and opening hours are computed by code, not by the language model.
- **Extensibility:** new agents are added to the pipeline without changing existing ones.
- **Privacy:** the language model runs locally, so user messages stay on the server.
- **Cost:** only free and open services are used.
- **Portability:** one codebase for Android and web; all server parts run in Docker.

5.2 Architectural Components

Table 5.1: Architectural components

No.	Component	Responsibility	Iteration
1	Flutter App	Screens, navigation, state, API client, maps, PDF export	1–4
2	REST API	Routing, validation, authentication, static media, API documentation	1–4
3	Orchestrator Agent	Runs the agents in order, builds three variants, scores and ranks them, records the trace	1
4	Intake Agent	Conversation manager, extractor, clarifier; sets the daily rhythm	1
5	Weather Agent	Forecast or typical climate; marks rainy and hot days	1
6	Destination Agent	Scores and filters places for each variant	1
7	Budget Agent	Reserves meals, stay and road trip; selects places within budget	1
8	Scheduling Agent	Groups places into days, orders them and assigns time slots within opening hours	1
9	Validator Agent	Checks overlaps, opening hours, day end, pace and budget; repairs plans	1

No.	Component	Responsibility	Iteration
10	Enrichment Agent	Adds group prices, weather notes, map links, photos and day totals	1
11	Replanning Agent	Applies change requests and weather changes to the affected days	2
12	Narration Agent	Writes a persona-based story for a stop from stored sources	3
13	Voice Agent	Converts narration scripts to audio	3
14	Stop Q&A Agent	Answers questions about a stop from stored sources	3
15	Caption Agent	Writes one caption per stop, matched to the template timing	4
16	Video Agent	Selects media, applies the template and renders the video	4
17	Services	Catalogue, distances, weather, language model client; later retrieval and rendering	1–4
18	Persistence	PostgreSQL through SQLAlchemy; Redis from Iteration 2	1–4
19	Ingestion Pipeline	Fetch and seed scripts for the place catalogue and media	1

Explanation of Figure 5.3:

- The Flutter App depends only on the REST API over HTTPS and JSON.
- The agent layer uses the services (catalogue, geography, weather, language model) and never accesses the database directly, so every agent can be tested on plain data.
- The ingestion pipeline is separate and runs only when the catalogue is loaded or refreshed.

Explanation of Figure 5.4:

- The Orchestrator coordinates all agents through one shared state.
- The seven planning agents of Iteration 1 run in a fixed order.
- The Replanning Agent re-runs the planning agents only for the affected days.
- The Narration Agent passes its script to the Voice Agent; the Stop Q&A Agent uses the same stored sources.

- The Caption Agent prepares captions that the Video Agent uses to assemble the video.

5.3 Interfaces and Dependencies

5.3.1 REST Interface

All endpoints use the prefix `/api/v1` and exchange JSON. Every endpoint except register, login, planning options and health requires a bearer token.

Table 5.2: REST endpoints

Method	**Endpoint**	**Purpose**	**It.**
POST	`/auth/register`	Create account	1
POST	`/auth/login`	Log in	1
GET	`/auth/me`	Current user	1
GET	`/meta`	Planning form options	1
GET	`/cities/{id}/places`	Places of a destination	1
POST	`/chat/sessions`	Start chat	1
GET	`/chat/sessions/{id}`	Chat history	1
POST	`/chat/sessions/{id}/messages`	Send text or selection	1
POST	`/itineraries/generate`	Three plans and agent trace	1
GET	`/itineraries`	Saved itineraries	1
GET	`/itineraries/batch/{batchId}`	Options generated together	1
GET	`/itineraries/{id}`	One itinerary	1
POST, DELETE	`/itineraries/{id}/save`	Save, unsave	1
GET	`/health`	Health check	1
POST	`/itineraries/{id}/replan`	Apply a change	2
GET	`/itineraries/{id}/versions`	Version history	2
GET	`/stops/{placeId}/narration`	Narration text and audio	3
POST	`/stops/{placeId}/ask`	Question about a stop	3
GET	`/video/templates`	Video templates	4
POST	`/itineraries/{id}/video`	Start rendering job	4

Method	Endpoint	Purpose	It.
GET	`/video/jobs/{id}`	Job status and download	4

5.3.2 External Dependencies

Table 5.3: External dependencies

Dependency	Used for	Access	If unavailable
Open-Meteo	Forecast and climate	Free API	Climate fallback, cache
OpenStreetMap	Map tiles and links	Free tiles	Links still work
Wikipedia, Wikidata, Wikimedia Commons	Descriptions, coordinates, images	Ingestion scripts	Item skipped
Ollama with Qwen 2.5	Reading free text; narration	Local server	Rules continue alone
PostgreSQL	All persistent data	Docker container	Health check, restart
Redis	Cache and job queue (It. 2+)	Docker container	Jobs retried
Text-to-speech engine	Narration audio (It. 3)	Local	Text narration shown
FFmpeg / MoviePy	Video rendering (It. 4)	Local worker	Job marked failed, retry

5.4 Architecture Decisions and Rationale

Table 5.4: Architecture decisions

ID	Decision	Alternative	Rationale
D1	One Flutter codebase	Separate web and Android apps	Half the frontend effort; same behaviour on both platforms
D2	Python and FastAPI	Node.js, Django	Strong ecosystem for language models, retrieval and media; automatic API documentation; simple testing

ID	Decision	Alternative	Rationale
D3	PostgreSQL	MongoDB	Relational integrity plus JSON plan documents; vector extension later; same engine locally, in CI and in deployment
D4	Language model for language, algorithms for numbers	Planning entirely by a language model	Budget, time and opening hours must be exact and repeatable; algorithms are testable
D5	Rules first, model second	Model only	Fast, free and deterministic in tests; model output is validated
D6	Agents over a shared state	One large planner function	Single responsibility, unit-testable agents; new agents added without changes
D7	Content imported once and stored locally	Live content calls	Faster, works offline, licences stored with media
D8	Free services only	Paid APIs	Project constraint; privacy; no running cost
D9	Background jobs for video	Rendering inside the request	API stays responsive; progress can be shown; failed jobs retried
D10	Narration grounded in stored sources	Free text generation	Reduces invented facts

5.5 Design Evolution

The design changed after the proposal review, mainly to reduce the scope and to strengthen the core features.

Table 5.5: Design changes since the proposal

No.	Proposal design	Current design	Justification
1	Many loosely connected modules	Core modules mapped to four iterations	Panel: idea vague, scope too large
2	Itinerary produced by a single generator	Seven specialised agents with validation and match score	Stronger technical contribution; constraints always respected
3	Video as a secondary feature	Dedicated Iteration 4 with Caption and Video agents	Panel: personalized video must be a core feature
4	Safety and alert module	Removed	Not part of the core features

No.	Proposal design	Current design	Justification
5	Predictive disruption forecasting	Removed	Not part of the core features
6	Personalized recommendation features	Removed	Scope reduction
7	Hosted language model	Rules plus a local open-source model	No paid APIs; privacy; offline operation
8	Separate web and mobile clients	One Flutter codebase	Less effort, consistent behaviour
9	Redis from the first release	Redis from Iteration 2	Introduced when background jobs are needed
10	Free-form narration	Narration grounded in stored sources	Factual reliability

5.6 Domain Model

Explanation of Figure 5.5:

- A **User** has many **ChatSessions**; each session has many **ChatMessages** and one **TripPreferences**.
- A **City** has many **Places**; places and **Interests** are related many-to-many.
- An **Itinerary** has **Days**, each with **Stops** that refer to places; it has one **CostBreakdown** and many **AgentRuns**.
- Later iterations add **ItineraryVersion** (Iteration 2), **Narration** (Iteration 3), and **VideoJob** with **Captions** (Iteration 4).

Table 5.6: Domain entities

Entity	Description	It.
User	Registered traveller	1
TripPreferences	Trip details shared by chat and form	1
ChatSession, ChatMessage	Conversation state and history	1
City, Place, Interest	Catalogue of destinations and places	1
Itinerary, Day, Stop	Generated plan and its parts	1

Entity	Description	It.
CostBreakdown	Entries, local travel, meals, stay, road trip, vehicle hire	1
AgentRun	One agent's message and duration	1
ItineraryVersion	Stored version after a change	2
SourceDocument, Persona, Narration	Source text, narrator style, script and audio	3
VideoTemplate, VideoJob, Caption	Video generation	4

5.7 Data Design

Database schemas are clearest when shown in their raw, textual form, so each table is presented below as its SQL definition. The QissaGo schema is implemented in PostgreSQL through SQLAlchemy. Catalogue tables use readable text identifiers that match the seed files, user data uses generated identifiers, and flexible documents such as trip preferences and itinerary plans are stored in JSON columns. Figure 5.6 shows the relationships between all tables of the complete system.

Explanation of Figure 5.6:

- Tables with a green header are implemented in Iteration 1.
- Light green headers are added in Iterations 2 and 3; orange headers in Iteration 4.
- Crow's feet mark the "many" side of each relationship.

5.7.1 Users Table

The `users` table stores registered travellers. The `email` column is unique, so each email has one account, and only a bcrypt hash of the password is kept. It is referenced by chat sessions and itineraries.

```
CREATE TABLE IF NOT EXISTS users (
  id            SERIAL PRIMARY KEY,
  name          VARCHAR(120) NOT NULL,
  email         VARCHAR(200) NOT NULL UNIQUE,
  password_hash VARCHAR(200) NOT NULL,
  created_at    TIMESTAMPTZ  NOT NULL DEFAULT NOW()
```

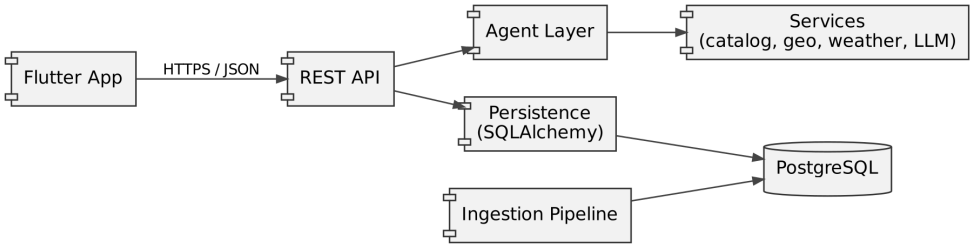


Figure 5.3: Component diagram of QissaGo

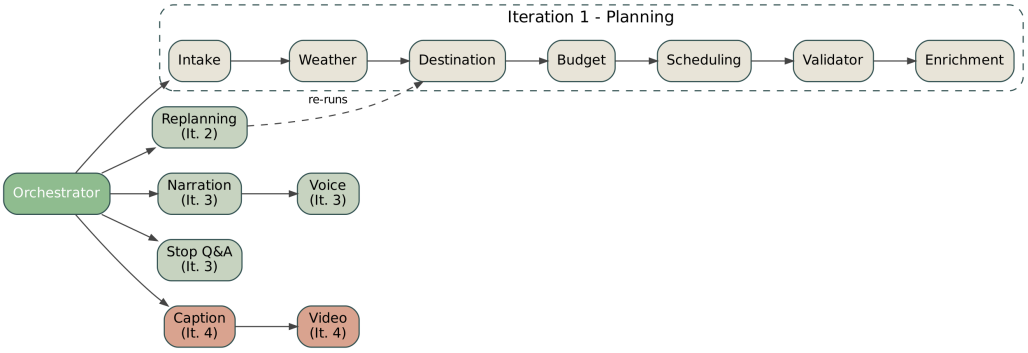


Figure 5.4: Multi-agent architecture of the complete system

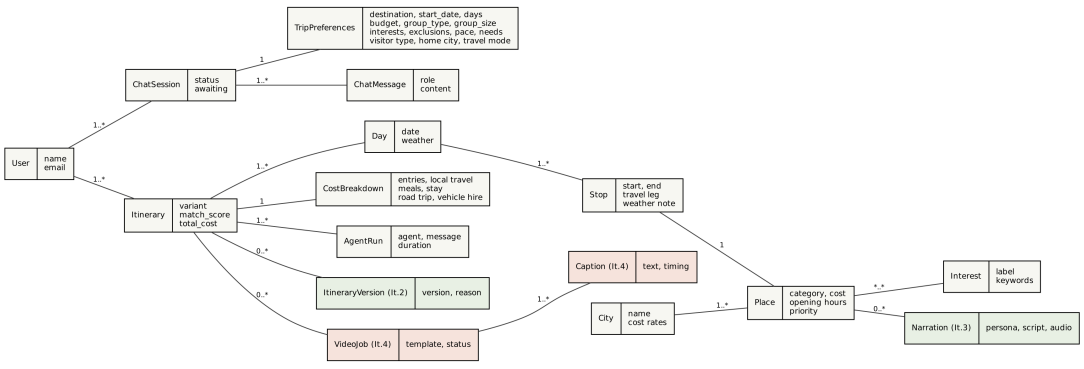


Figure 5.5: Domain model of QissaGo

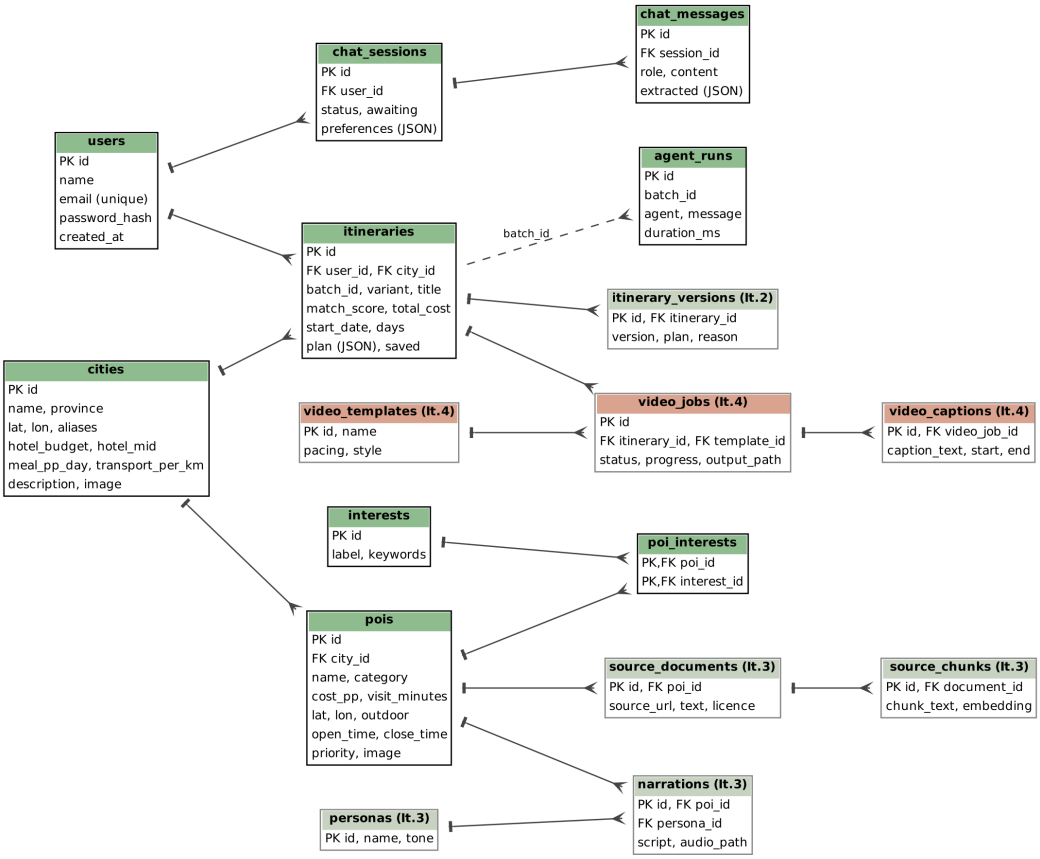


Figure 5.6: Entity relationship diagram of QissaGo (all iterations)

```
);
CREATE INDEX IF NOT EXISTS ix_users_email ON users(email);
```

5.7.2 Cities Table

The `cities` table holds the supported destinations. Besides the name and coordinates, it stores other spellings for the chatbot (`aliases`) and the cost rates used by the Budget Agent, together with the description and image imported by the ingestion scripts.

```
CREATE TABLE IF NOT EXISTS cities (
  id               VARCHAR(40) PRIMARY KEY,
  name             VARCHAR(80) NOT NULL,
  province         VARCHAR(80),
  lat              DOUBLE PRECISION NOT NULL,
  lon              DOUBLE PRECISION NOT NULL,
  aliases          VARCHAR(200),
  hotel_budget     INTEGER NOT NULL,   -- PKR per room per night
  hotel_mid        INTEGER NOT NULL,
  meal_pp_day      INTEGER NOT NULL,   -- PKR per person per day
  transport_per_km INTEGER NOT NULL,   -- PKR per car per km
  wiki_title       VARCHAR(200),
  description      TEXT,
  image            VARCHAR(300),
  image_credit     VARCHAR(300)
);
```

5.7.3 Interests Table

The `interests` table lists the interest categories, such as history, food or nature, and the keywords the chatbot recognises for each.

```
CREATE TABLE IF NOT EXISTS interests (
  id       VARCHAR(40) PRIMARY KEY,
  label    VARCHAR(80) NOT NULL,
  keywords VARCHAR(400)
);
```

5.7.4 Places (POIs) Table

The `pois` table is the central catalogue of places a traveller can visit. It stores the cost per person, visit duration, coordinates, whether the place is outdoors, its best visiting time,

opening hours and priority (3 = must-see), together with the licensed image and its credit. These columns drive the Destination, Budget and Scheduling agents.

```
CREATE TABLE IF NOT EXISTS pois (
  id            VARCHAR(60) PRIMARY KEY,
  city_id       VARCHAR(40) NOT NULL REFERENCES cities(id),
  name          VARCHAR(160) NOT NULL,
  category      VARCHAR(40)  NOT NULL,
  cost_pp       INTEGER DEFAULT 0,
  visit_minutes INTEGER DEFAULT 60,
  lat           DOUBLE PRECISION NOT NULL,
  lon           DOUBLE PRECISION NOT NULL,
  outdoor       BOOLEAN DEFAULT TRUE,
  best_time     VARCHAR(10) DEFAULT 'any',
               -- morning|midday|evening|any
  open_time     VARCHAR(5)  DEFAULT '09:00',
  close_time    VARCHAR(5)  DEFAULT '21:00',
  priority      INTEGER DEFAULT 2,           -- 3 = must-see
  wiki_title    VARCHAR(200),
  wiki_url      VARCHAR(300),
  description   TEXT,
  image         VARCHAR(300),
  image_credit  VARCHAR(300),
  image_license VARCHAR(120)
);
CREATE INDEX IF NOT EXISTS ix_pois_city_id ON pois(city_id);
```

5.7.5 Place Interests Table

The `poi_interests` table links places to interests (many-to-many). The composite primary key prevents duplicate links.

```
CREATE TABLE IF NOT EXISTS poi_interests (
  poi_id      VARCHAR(60) REFERENCES pois(id) ON DELETE CASCADE,
  interest_id VARCHAR(40)
               REFERENCES interests(id) ON DELETE CASCADE,
  PRIMARY KEY (poi_id, interest_id)
);
```

5.7.6 Chat Sessions and Chat Messages Tables

A chat session holds the current trip preferences as JSON, its status (collecting, ready or planned) and the field the chatbot asked about last, so that short answers are understood in context. Every message is stored with the details the extractor found in it.

```
CREATE TABLE IF NOT EXISTS chat_sessions (
  id          VARCHAR(32) PRIMARY KEY,
  user_id     INTEGER NOT NULL
               REFERENCES users(id) ON DELETE CASCADE,
  status      VARCHAR(20) DEFAULT 'collecting',
  preferences JSON,
  awaiting    VARCHAR(40),
  created_at  TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  updated_at  TIMESTAMPTZ NOT NULL DEFAULT NOW()
);
CREATE INDEX IF NOT EXISTS ix_chat_sessions_user_id
  ON chat_sessions(user_id);

CREATE TABLE IF NOT EXISTS chat_messages (
  id         SERIAL PRIMARY KEY,
  session_id VARCHAR(32) NOT NULL
             REFERENCES chat_sessions(id) ON DELETE CASCADE,
  role       VARCHAR(10) NOT NULL,       -- user | bot
  content    TEXT NOT NULL,
  extracted  JSON,
  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);
CREATE INDEX IF NOT EXISTS ix_chat_messages_session_id
  ON chat_messages(session_id);
```

5.7.7 Itineraries Table

The `itineraries` table stores one row per generated plan. The `batch_id` groups the three options generated together, `variant` identifies the strategy (balanced, deep-dive or saver), and the `plan` JSON column holds the days, stops, cost breakdown and statistics. The `saved` flag drives the Saved tab.

```
CREATE TABLE IF NOT EXISTS itineraries (
  id          VARCHAR(32) PRIMARY KEY,
  user_id     INTEGER NOT NULL
               REFERENCES users(id) ON DELETE CASCADE,
```

```
  session_id  VARCHAR(32),
  batch_id    VARCHAR(32) NOT NULL,
  city_id     VARCHAR(40) NOT NULL REFERENCES cities(id),
  variant     VARCHAR(20) NOT NULL,
  title       VARCHAR(120) NOT NULL,
  match_score INTEGER NOT NULL,
  total_cost  INTEGER NOT NULL,
  start_date  DATE NOT NULL,
  days        INTEGER NOT NULL,
  preferences JSON NOT NULL,
  plan        JSON NOT NULL,
  saved       BOOLEAN DEFAULT FALSE,
  created_at  TIMESTAMPTZ NOT NULL DEFAULT NOW()
);
CREATE INDEX IF NOT EXISTS ix_itineraries_user_id
  ON itineraries(user_id);
CREATE INDEX IF NOT EXISTS ix_itineraries_batch_id
  ON itineraries(batch_id);
```

5.7.8 Agent Runs Table

The `agent_runs` table records one row per agent per generation, with the agent's message and its duration. It is the stored evidence of the multi-agent pipeline and is shown on the planning progress screen.

```
CREATE TABLE IF NOT EXISTS agent_runs (
  id          SERIAL PRIMARY KEY,
  batch_id    VARCHAR(32) NOT NULL,
  agent       VARCHAR(60) NOT NULL,
  message     TEXT NOT NULL,
  duration_ms INTEGER NOT NULL,
  created_at  TIMESTAMPTZ NOT NULL DEFAULT NOW()
);
CREATE INDEX IF NOT EXISTS ix_agent_runs_batch_id
  ON agent_runs(batch_id);
```

5.7.9 Itinerary Versions and Replan Requests Tables (Iteration 2)

Each accepted change creates a new row in `itinerary_versions`, so earlier versions are kept. The `replan_requests` table records the change text, the parsed change and its status.

```
CREATE TABLE IF NOT EXISTS itinerary_versions (
  id           SERIAL PRIMARY KEY,
  itinerary_id VARCHAR(32) NOT NULL
               REFERENCES itineraries(id) ON DELETE CASCADE,
  version      INTEGER NOT NULL,
  plan         JSON NOT NULL,
  reason       TEXT,
  created_at   TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  UNIQUE (itinerary_id, version)
);

CREATE TABLE IF NOT EXISTS replan_requests (
  id            SERIAL PRIMARY KEY,
  itinerary_id  VARCHAR(32) NOT NULL
               REFERENCES itineraries(id) ON DELETE CASCADE,
  user_id       INTEGER NOT NULL
               REFERENCES users(id) ON DELETE CASCADE,
  request_text  TEXT NOT NULL,
  parsed_change JSON,
  status        VARCHAR(20) DEFAULT 'pending',
               -- pending|applied|rejected
  created_at    TIMESTAMPTZ NOT NULL DEFAULT NOW()
);
```

5.7.10 Source, Persona and Narration Tables (Iteration 3)

Narration is grounded in stored sources. `source_documents` keeps the text and licence for each place, `source_chunks` stores searchable pieces with their embeddings, `personas` defines narrator styles, and `narrations` caches each script and its audio file.

```
CREATE TABLE IF NOT EXISTS source_documents (
  id           SERIAL PRIMARY KEY,
  poi_id       VARCHAR(60) NOT NULL
               REFERENCES pois(id) ON DELETE CASCADE,
  source_url   VARCHAR(300),
  title        VARCHAR(200),
  text         TEXT NOT NULL,
  licence      VARCHAR(120),
  retrieved_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

CREATE TABLE IF NOT EXISTS source_chunks (
```

```
  id          SERIAL PRIMARY KEY,
  document_id INTEGER NOT NULL
               REFERENCES source_documents(id) ON DELETE CASCADE,
  chunk_text  TEXT NOT NULL,
  embedding   VECTOR(384)
);

CREATE TABLE IF NOT EXISTS personas (
  id           SERIAL PRIMARY KEY,
  name         VARCHAR(60) NOT NULL UNIQUE,
  tone         VARCHAR(120),
  style_prompt TEXT
);

CREATE TABLE IF NOT EXISTS narrations (
  id         SERIAL PRIMARY KEY,
  poi_id     VARCHAR(60) NOT NULL
               REFERENCES pois(id) ON DELETE CASCADE,
  persona_id INTEGER NOT NULL REFERENCES personas(id),
  script     TEXT NOT NULL,
  audio_path VARCHAR(300),
  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  UNIQUE (poi_id, persona_id)
);
```

5.7.11 Video Tables (Iteration 4)

`video_templates` defines the pacing and style of each template, `media_assets` lists the photos, clips and music available for a place, `video_jobs` tracks each background rendering job, and `video_captions` stores the caption shown for each stop with its timing.

```
CREATE TABLE IF NOT EXISTS video_templates (
  id          SERIAL PRIMARY KEY,
  name        VARCHAR(60) NOT NULL UNIQUE,
  description TEXT,
  pacing      JSON,
  style       JSON,
  music_track VARCHAR(300)
);

CREATE TABLE IF NOT EXISTS media_assets (
  id       SERIAL PRIMARY KEY,
```

```
  poi_id   VARCHAR(60) REFERENCES pois(id) ON DELETE CASCADE,
  type     VARCHAR(10) NOT NULL,      -- image|clip|music
  path     VARCHAR(300) NOT NULL,
  credit   VARCHAR(300),
  licence  VARCHAR(120),
  duration REAL
);

CREATE TABLE IF NOT EXISTS video_jobs (
  id           VARCHAR(32) PRIMARY KEY,
  itinerary_id VARCHAR(32) NOT NULL
               REFERENCES itineraries(id) ON DELETE CASCADE,
  user_id      INTEGER NOT NULL
               REFERENCES users(id) ON DELETE CASCADE,
  template_id  INTEGER NOT NULL REFERENCES video_templates(id),
  status       VARCHAR(20) DEFAULT 'queued',
               -- queued|rendering|completed|failed
  progress     INTEGER DEFAULT 0,
  output_path  VARCHAR(300),
  error        TEXT,
  created_at   TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  finished_at  TIMESTAMPTZ
);

CREATE TABLE IF NOT EXISTS video_captions (
  id           SERIAL PRIMARY KEY,
  video_job_id VARCHAR(32) NOT NULL
               REFERENCES video_jobs(id) ON DELETE CASCADE,
  poi_id       VARCHAR(60) REFERENCES pois(id),
  caption_text TEXT NOT NULL,
  start_sec    REAL NOT NULL,
  end_sec      REAL NOT NULL
);
```

5.7.12 Relational Integrity

- Deleting a user deletes their chat sessions, messages, itineraries, versions and video jobs (`ON DELETE CASCADE`).
- Deleting a place deletes its interest links, source documents and narrations.
- Itineraries reference a city, so a city cannot be removed while plans use it.

- Unique constraints prevent duplicate accounts, duplicate interest links, duplicate versions and duplicate cached narrations.

- Every API query filters by the logged-in user's id, so users only see their own data.

5.8 Process / Behavior Design

5.8.1 Activity Diagram

The activity diagram shows the dynamic flow of operations and the decision points across the system. It covers the complete user journey from describing a trip to saving and exporting an itinerary. The diagram has four swim lanes: Traveller, QissaGo System, Agents and Database.

5.8.1.1 Trip Description Flow

- The traveller opens the chat or the planning form and enters the trip details.

- The system extracts the preferences using rules first and the local language model only for what the rules miss.

- If details are missing or contradictory, the traveller answers a follow-up question and the details are extracted again.

- When all details are complete, the system shows a summary and the traveller confirms it.

5.8.1.2 Itinerary Generation Flow

- The Intake and Weather agents prepare the plan settings and the daily weather.

- The Destination and Budget agents select places, using places and cost rates loaded from the database.

- The Scheduling agent builds the days. If the plan is over budget after real travel costs, the weakest stop is dropped and the days are rebuilt.

- The Validator and Enrichment agents check and complete the plans; the three plans are ranked and stored with the agent runs.

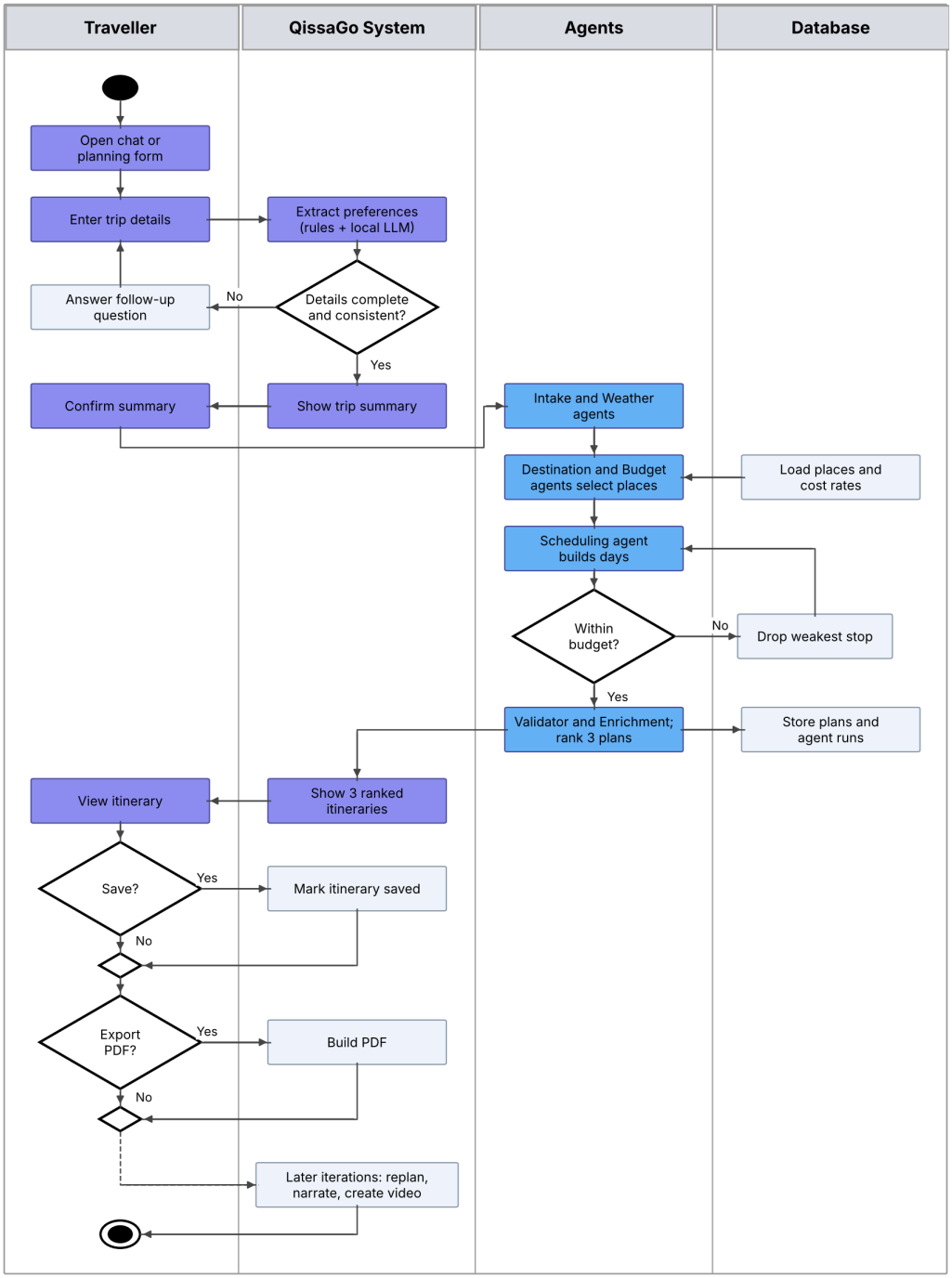


Figure 5.7: Activity diagram of QissaGo

5.8.1.3 Itinerary Use Flow

- The traveller views the ranked itineraries and opens one.
- The traveller may save it (stored as saved) and may export it as a PDF.
- In later iterations the traveller can also edit the itinerary, listen to narration and create a travel video.

5.8.1.4 Key Decision Points

1. **Details complete and consistent?** – decides whether to ask a follow-up question or show the summary.
2. **Within budget?** – decides whether the weakest stop must be dropped.
3. **Save?** – decides whether the itinerary is added to the Saved tab.
4. **Export PDF?** – decides whether a PDF is built for download.

5.8.2 Data Flow Diagrams

Data flow diagrams show how data moves between processes, data stores and external entities, from the context diagram (Level 0) to detailed Level 2 diagrams for the main processes.

5.8.2.1 DFD Level 0 – Context Diagram

The context diagram shows QissaGo as a single process interacting with four external entities:

- **Traveller:** sends credentials, chat messages, form selections, change requests and video template choices; receives replies, itineraries, PDFs, narration and videos.
- **Weather Service:** receives the location and dates; returns the forecast.
- **Map Service:** receives coordinates; returns map tiles and routes.
- **Local Language Model:** receives text and prompts; returns structured details and narration text.

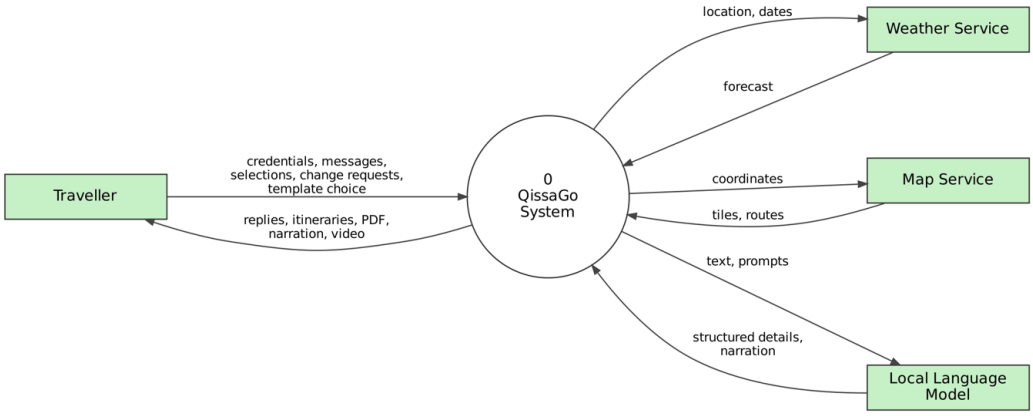


Figure 5.8: Context diagram of QissaGo

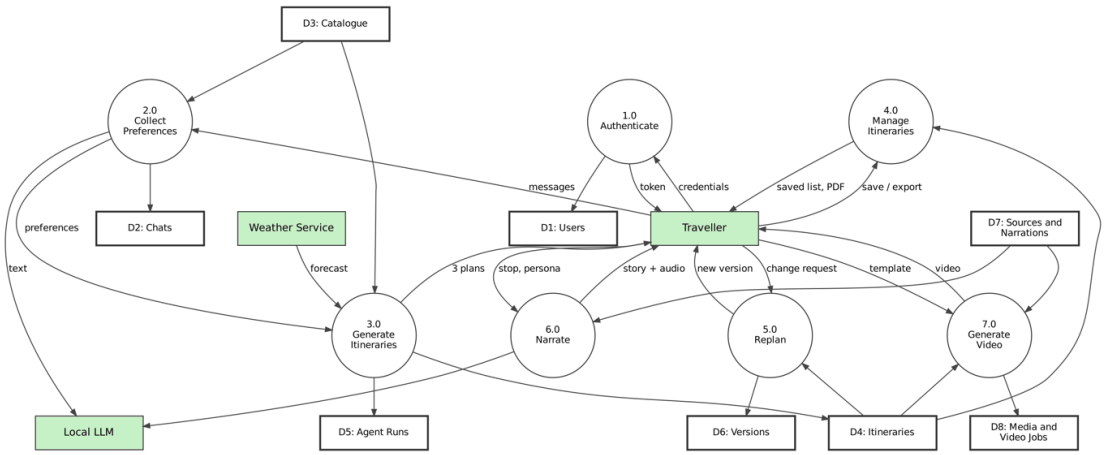


Figure 5.9: DFD Level 1 diagram of QissaGo

5.8.2.2 DFD Level 1

The Level 1 DFD decomposes QissaGo into seven major processes:

Process 1.0 – Authenticate:

- **Input:** Credentials from the Traveller
- **Processing:** Validate input, hash or verify the password, issue a token
- **Output:** Token to the Traveller
- **Data Store Interaction:** Stores and reads D1: Users

Process 2.0 – Collect Preferences:

- **Input:** Messages and selections from the Traveller
- **Processing:** Rule extraction, local model extraction, merging, conflict checks, next question
- **Output:** Complete preferences to Process 3.0
- **Data Store Interaction:** Stores messages in D2: Chats; reads names and interests from D3: Catalogue

Process 3.0 – Generate Itineraries:

- **Input:** Preferences from Process 2.0; forecast from the Weather Service
- **Processing:** Seven planning agents, scoring and ranking
- **Output:** Three ranked plans to the Traveller
- **Data Store Interaction:** Reads D3: Catalogue; stores D4: Itineraries and D5: Agent Runs

Process 4.0 – Manage Itineraries:

- **Input:** Save, list and export requests
- **Processing:** Update the saved flag, list saved plans, build PDF
- **Output:** Saved list and PDF to the Traveller
- **Data Store Interaction:** Reads and updates D4: Itineraries

Process 5.0 – Replan (Iteration 2):

- **Input:** Change request from the Traveller or a forecast change
- **Processing:** Find affected days, re-run planning agents for them
- **Output:** New version to the Traveller
- **Data Store Interaction:** Reads D4: Itineraries; stores D6: Versions

Process 6.0 – Narrate (Iteration 3):

- **Input:** Stop and persona, or a question, from the Traveller
- **Processing:** Retrieve sources, write script, create audio, answer questions
- **Output:** Story text and audio, or an answer
- **Data Store Interaction:** Reads and stores D7: Sources and Narrations

Process 7.0 – Generate Video (Iteration 4):

- **Input:** Itinerary and template from the Traveller
- **Processing:** Captions, media selection, background rendering
- **Output:** Video to the Traveller
- **Data Store Interaction:** Reads D4 and D7; stores D8: Media and Video Jobs

Data Stores:

- **D1: Users** – accounts and password hashes.
- **D2: Chats** – chat sessions, messages and current preferences.
- **D3: Catalogue** – destinations, places, interests and cost rates.
- **D4: Itineraries** – generated and saved plans.
- **D5: Agent Runs** – trace of every agent per generation.
- **D6: Versions** – itinerary versions after changes.
- **D7: Sources and Narrations** – source text, chunks, personas and narrations.
- **D8: Media and Video Jobs** – photos, audio, videos and rendering jobs.

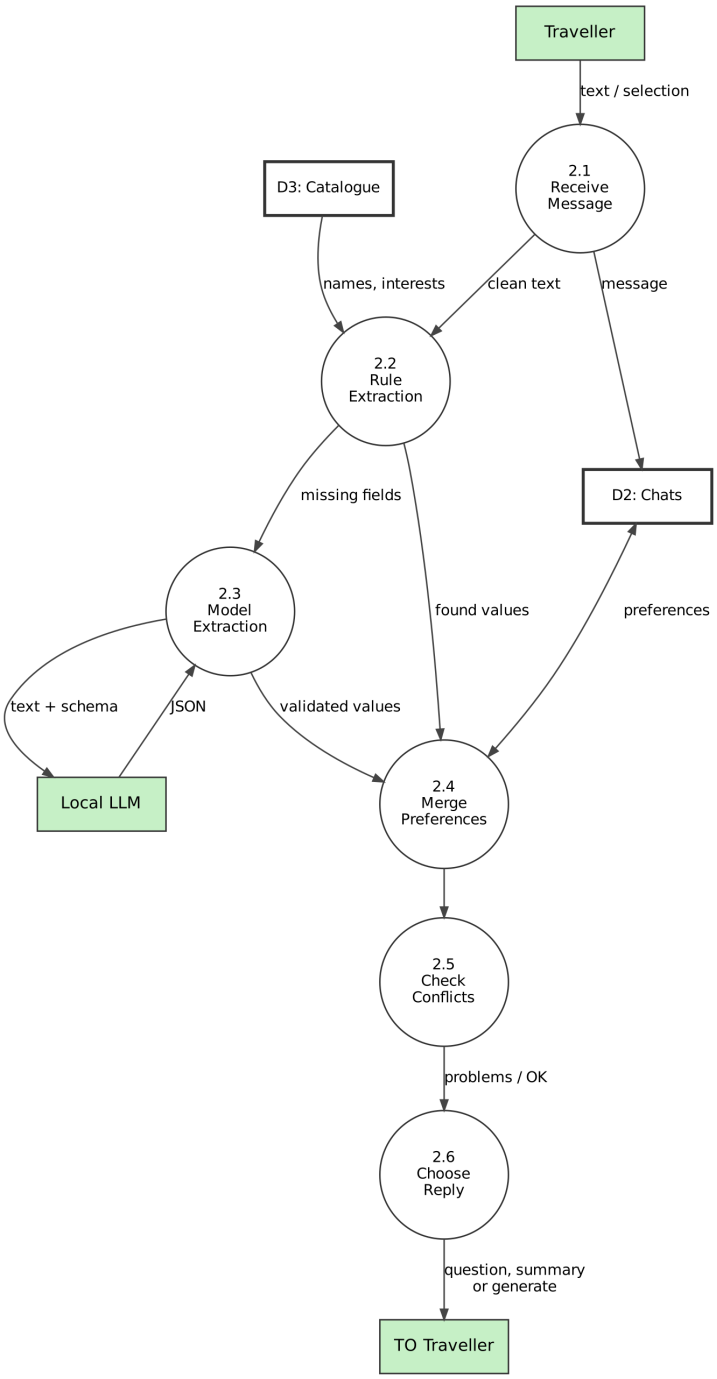


Figure 5.10: DFD Level 2 diagram of Collect Preferences

5.8.2.3 DFD Level 2 – Collect Preferences

This Level 2 diagram breaks down Process 2.0 into its steps.

Process 2.1 – Receive Message:

- **Input:** Text or selection from the Traveller
- **Processing:** Normalise text, store the message
- **Output:** Clean text to Process 2.2
- **Data Store Interaction:** Stores in D2: Chats

Process 2.2 – Rule Extraction:

- **Input:** Clean text; place names and interests
- **Processing:** Extract money, dates, days, group, destination, interests, exclusions, pace and needs
- **Output:** Found values to Process 2.4; missing fields to Process 2.3
- **Data Store Interaction:** Reads D3: Catalogue

Process 2.3 – Model Extraction:

- **Input:** Text and missing fields
- **Processing:** Ask the local model for JSON that follows a schema; validate the values
- **Output:** Validated values to Process 2.4

Process 2.4 – Merge Preferences:

- **Input:** Values from Processes 2.2 and 2.3
- **Processing:** Merge into the stored preferences using the last question asked
- **Output:** Updated preferences to Process 2.5
- **Data Store Interaction:** Reads and updates D2: Chats

Process 2.5 – Check Conflicts:

- **Input:** Updated preferences, today's date

- **Processing:** Past date, unsupported destination, group size, minimum trip cost
- **Output:** Problems or OK to Process 2.6

Process 2.6 – Choose Reply:

- **Input:** Preferences and problems
- **Processing:** Pick the next question with suggestions, the summary, or the generate action
- **Output:** Reply to the Traveller

5.8.2.4 DFD Level 2 – Generate Itineraries

This Level 2 diagram breaks down Process 3.0, the core of the multi-agent planner.

Process 3.1 – Intake:

- **Input:** Preferences
- **Processing:** Validate preferences; set stops per day and start time from pace and needs
- **Output:** Planning settings to Process 3.2

Process 3.2 – Weather:

- **Input:** Destination and dates; forecast
- **Processing:** Daily temperature and rain; mark rainy and hot days
- **Output:** Daily weather to Process 3.3
- **Data Store Interaction:** Reads and updates the weather cache

Process 3.3 – Destination Scoring:

- **Input:** Places, interests, needs, weather
- **Processing:** Score each place for three variants; remove exclusions
- **Output:** Ranked places to Process 3.4
- **Data Store Interaction:** Reads D3: Catalogue

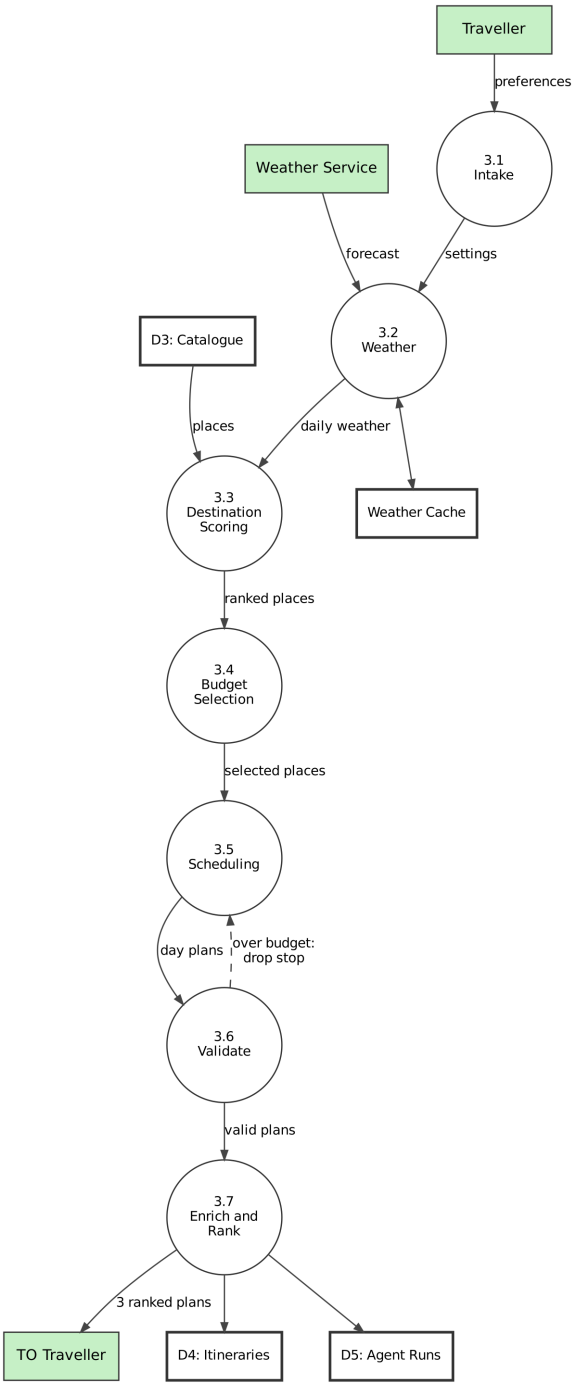


Figure 5.11: DFD Level 2 diagram of Generate Itineraries

Process 3.4 – Budget Selection:

- **Input:** Ranked places and budget
- **Processing:** Reserve meals, stay and road trip; select places that fit
- **Output:** Selected places to Process 3.5

Process 3.5 – Scheduling:

- **Input:** Selected places
- **Processing:** Group into days; order by distance and best time; assign slots within opening hours
- **Output:** Day plans to Process 3.6

Process 3.6 – Validate:

- **Input:** Day plans
- **Processing:** Check overlaps, opening hours, day end, pace and budget; drop the weakest stop if over budget
- **Output:** Valid plans to Process 3.7

Process 3.7 – Enrich and Rank:

- **Input:** Valid plans
- **Processing:** Add prices, weather notes and links; compute match score; rank
- **Output:** Three ranked plans to the Traveller
- **Data Store Interaction:** Stores D4: Itineraries and D5: Agent Runs

5.8.2.5 DFD Level 2 – Replan (Iteration 2)

Process 5.1 – Receive Change:

- **Input:** Change request or forecast change
- **Processing:** Parse the change (add, remove, move, start later, change day)
- **Output:** Parsed change to Process 5.2

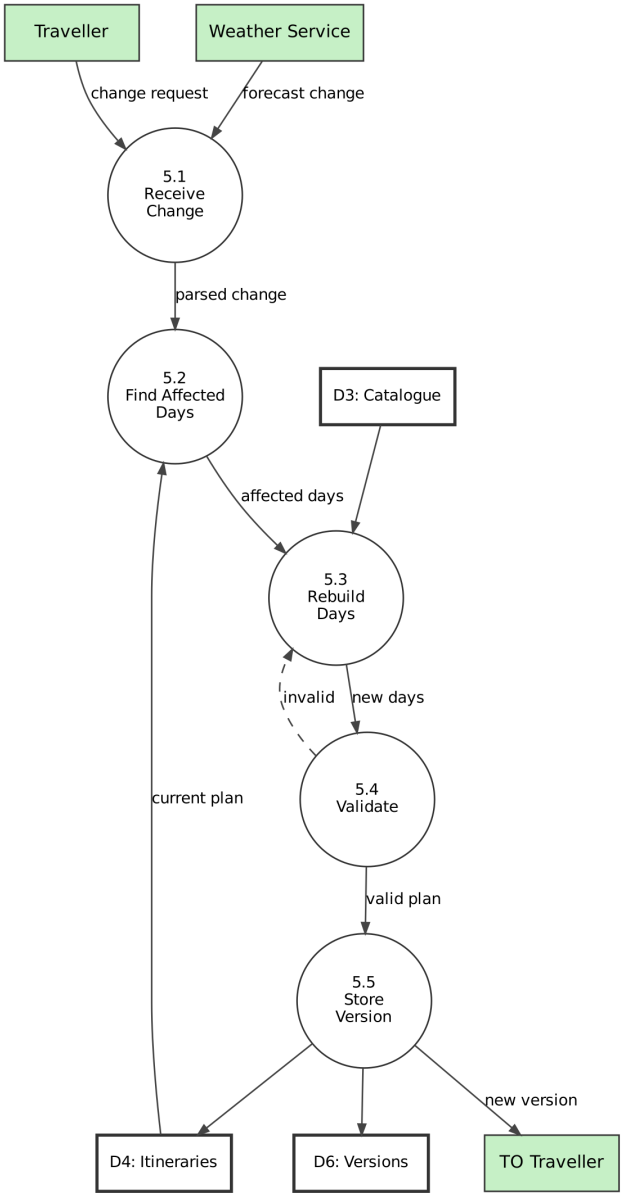


Figure 5.12: DFD Level 2 diagram of Replan

Process 5.2 – Find Affected Days:

- **Input:** Parsed change and current plan
- **Processing:** Identify the days that must change
- **Output:** Affected days to Process 5.3
- **Data Store Interaction:** Reads D4: Itineraries

Process 5.3 – Rebuild Days:

- **Input:** Affected days
- **Processing:** Re-run Destination, Budget and Scheduling for those days only
- **Output:** New days to Process 5.4
- **Data Store Interaction:** Reads D3: Catalogue

Process 5.4 – Validate:

- **Input:** New days
- **Processing:** Check budget, opening hours and pace
- **Output:** Valid plan to Process 5.5

Process 5.5 – Store Version:

- **Input:** Valid plan
- **Processing:** Save as a new version
- **Output:** New version to the Traveller
- **Data Store Interaction:** Stores D6: Versions; updates D4: Itineraries

5.8.2.6 DFD Level 2 – Narrate (Iteration 3)

Process 6.1 – Check Cache:

- **Input:** Stop and persona
- **Processing:** Look for a stored narration

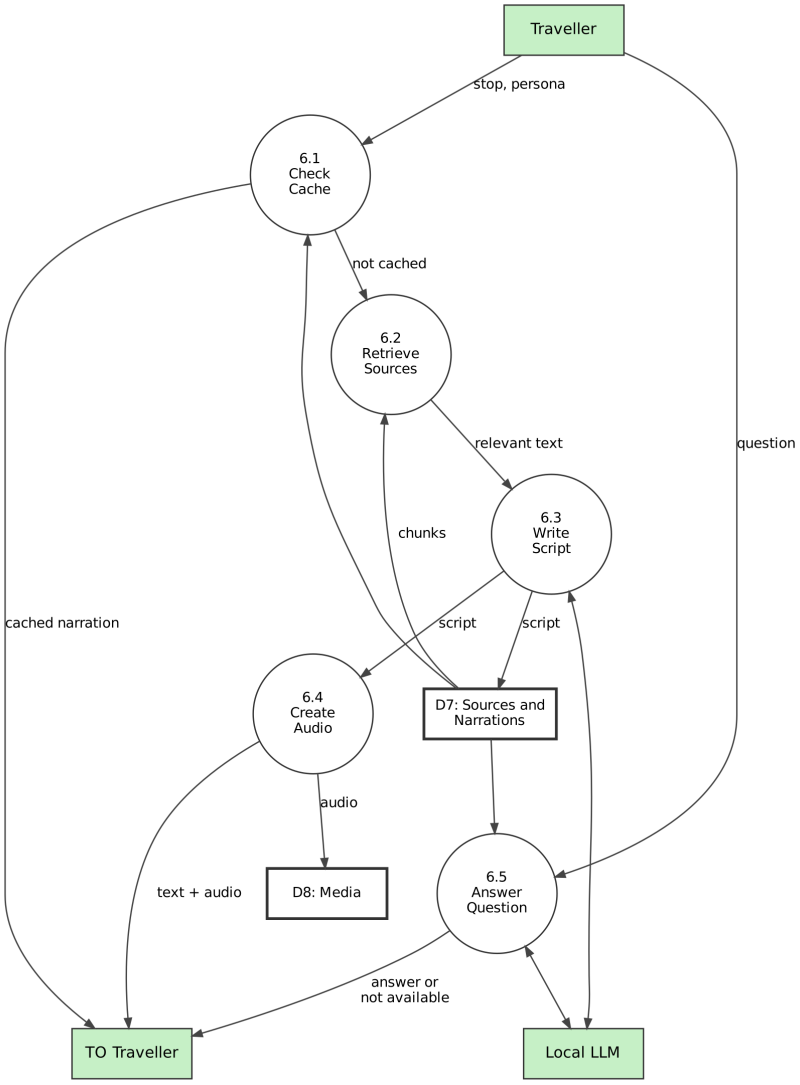


Figure 5.13: DFD Level 2 diagram of Narrate

- **Output:** Cached narration to the Traveller, or a request to Process 6.2
- **Data Store Interaction:** Reads D7

Process 6.2 – Retrieve Sources:

- **Input:** Stop
- **Processing:** Find the most relevant source chunks
- **Output:** Relevant text to Process 6.3
- **Data Store Interaction:** Reads D7

Process 6.3 – Write Script:

- **Input:** Relevant text and persona
- **Processing:** Local model writes a story using only the given text
- **Output:** Script to Process 6.4
- **Data Store Interaction:** Stores the script in D7

Process 6.4 – Create Audio:

- **Input:** Script
- **Processing:** Text-to-speech conversion
- **Output:** Text and audio to the Traveller
- **Data Store Interaction:** Stores audio in D8: Media

Process 6.5 – Answer Question:

- **Input:** Question about a stop
- **Processing:** Retrieve chunks; answer only from them
- **Output:** Answer or “not available” to the Traveller
- **Data Store Interaction:** Reads D7

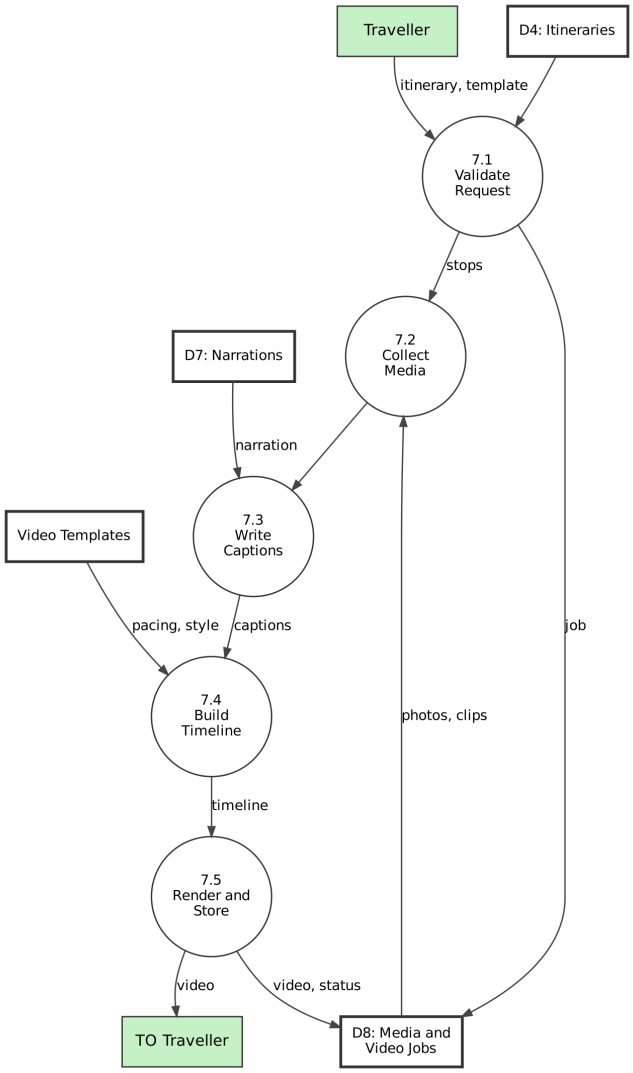


Figure 5.14: DFD Level 2 diagram of Generate Video

5.8.2.7 DFD Level 2 – Generate Video (Iteration 4)

Process 7.1 – Validate Request:

- **Input:** Itinerary and template
- **Processing:** Check ownership and template; create a job
- **Output:** Stops to Process 7.2
- **Data Store Interaction:** Stores the job in D8

Process 7.2 – Collect Media:

- **Input:** Stops
- **Processing:** Gather photos, clips and credits
- **Output:** Media list to Process 7.3
- **Data Store Interaction:** Reads D8: Media

Process 7.3 – Write Captions:

- **Input:** Stops and narration
- **Processing:** One caption per stop, sized to the template
- **Output:** Captions to Process 7.4
- **Data Store Interaction:** Reads D7: Narrations

Process 7.4 – Build Timeline:

- **Input:** Media, captions, template
- **Processing:** Order, durations, transitions and music
- **Output:** Timeline to Process 7.5
- **Data Store Interaction:** Reads video templates

Process 7.5 – Render and Store:

- **Input:** Timeline
- **Processing:** Render and compress the video; update the job status
- **Output:** Video to the Traveller
- **Data Store Interaction:** Stores in D8: Media and Video Jobs

5.8.3 System-level Sequence Diagrams

This section contains the system sequence diagrams of QissaGo. Only the inputs and outputs are shown: the system is treated as a black box, following the rules of system sequence diagrams.

5.8.4 State Diagrams

Explanation of Figure 5.32:

- **Collecting:** details are being gathered; a missing detail or a conflict keeps the session in this state.
- **Ready:** all details are known and the summary is shown; a change request returns to Collecting.
- **Planned:** the itineraries have been generated.

Explanation of Figure 5.33:

- A generated itinerary can be saved and unsaved.
- From Iteration 2 a saved itinerary can be changed, creating a new version.
- From Iteration 4 a video can be requested for a saved itinerary.

Explanation of Figure 5.34:

- A job is Queued, then Rendering with progress updates.
- It ends as Completed or Failed; a failed job can be retried.

5.8.5 Class Diagram

Explanation of Figure 5.35:

- The abstract class `Agent` defines `name`, `role` and `run(state)`; calling an agent times it and records a trace entjpeg
- The seven planning agents inherit from `Agent`.
- The `Orchestrator` uses the agents, the `Catalog` and `TripPreferences`.
- The `LLM` interface is implemented by `OllamaLLM` and by `NoLLM`, which lets the chatbot work without the model.

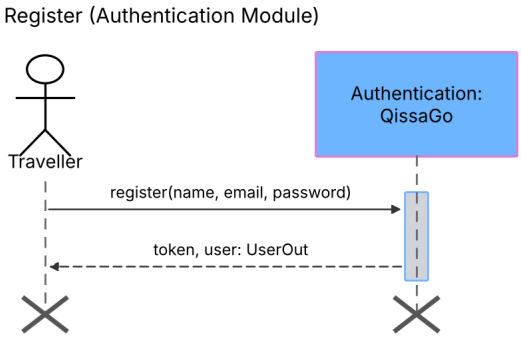


Figure 5.15: System Sequence Diagram for Register function (Authentication Module)

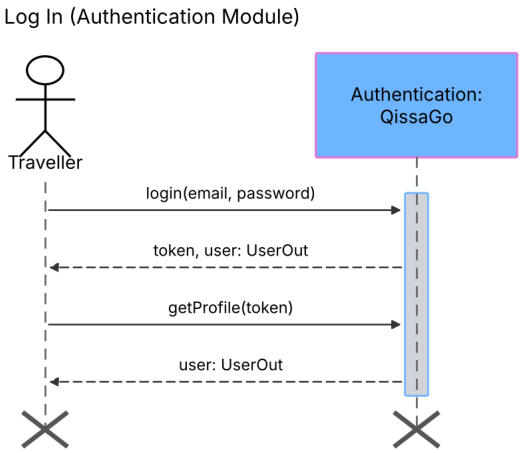


Figure 5.16: System Sequence Diagram for Log In function (Authentication Module)

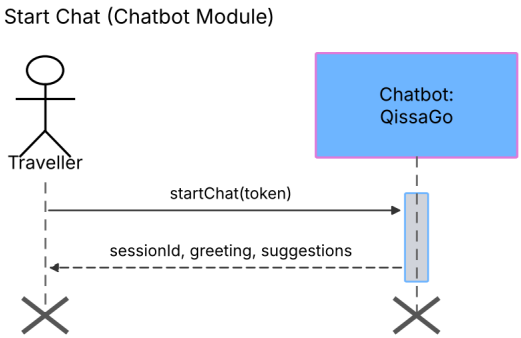


Figure 5.17: System Sequence Diagram for Start Chat function (Chatbot Module)

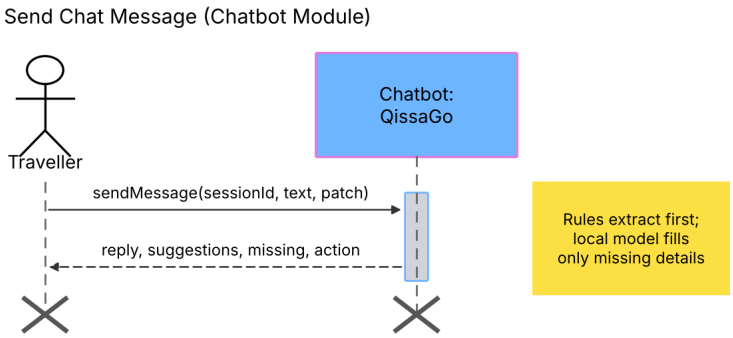


Figure 5.18: System Sequence Diagram for Send Chat Message function (Chatbot Module)

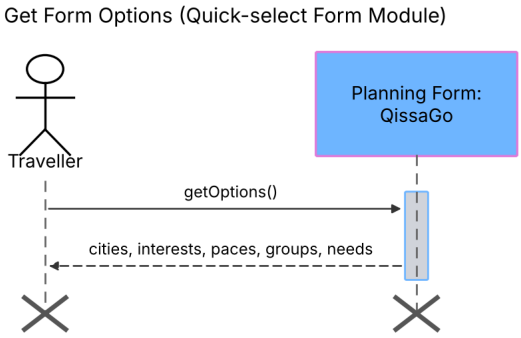


Figure 5.19: System Sequence Diagram for Get Form Options function (Quick-select Form Module)

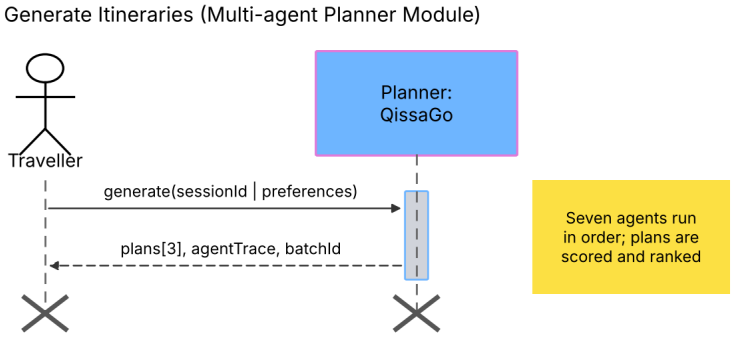


Figure 5.20: System Sequence Diagram for Generate Itineraries function (Multi-agent Planner Module)

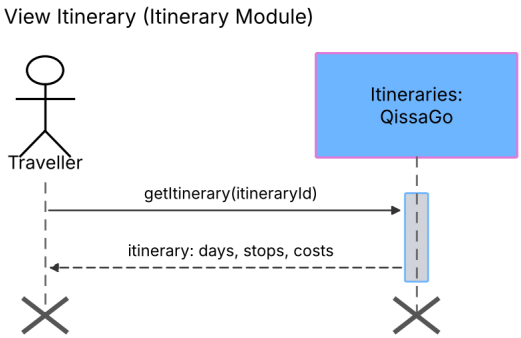


Figure 5.21: System Sequence Diagram for View Itinerary function (Itinerary Module)

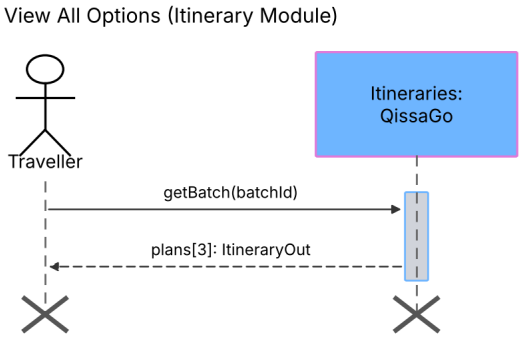


Figure 5.22: System Sequence Diagram for View All Options function (Itinerary Module)

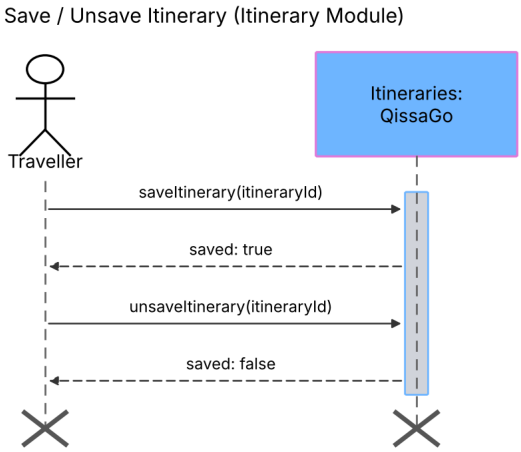


Figure 5.23: System Sequence Diagram for Save / Unsave Itinerary function (Itinerary Module)

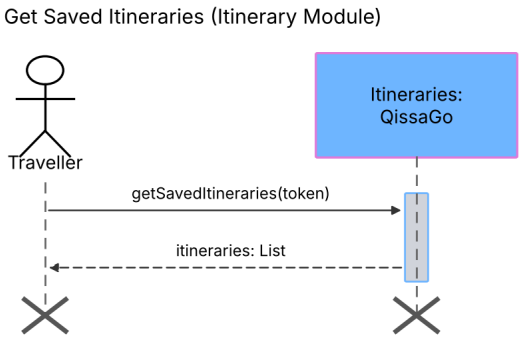


Figure 5.24: System Sequence Diagram for Get Saved Itineraries function (Itinerary Module)

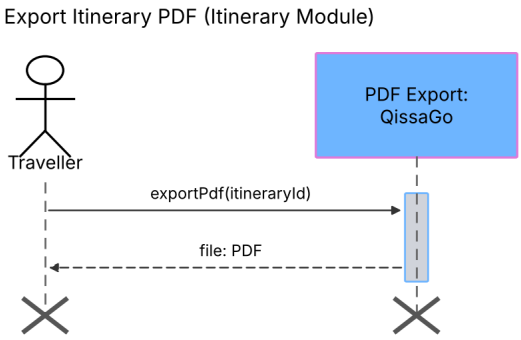


Figure 5.25: System Sequence Diagram for Export Itinerary PDF function (Itinerary Module)

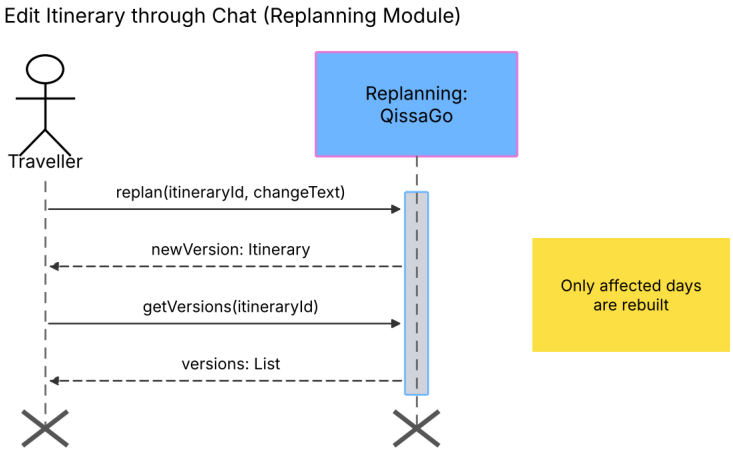


Figure 5.26: System Sequence Diagram for Edit Itinerary function (Replanning Module)

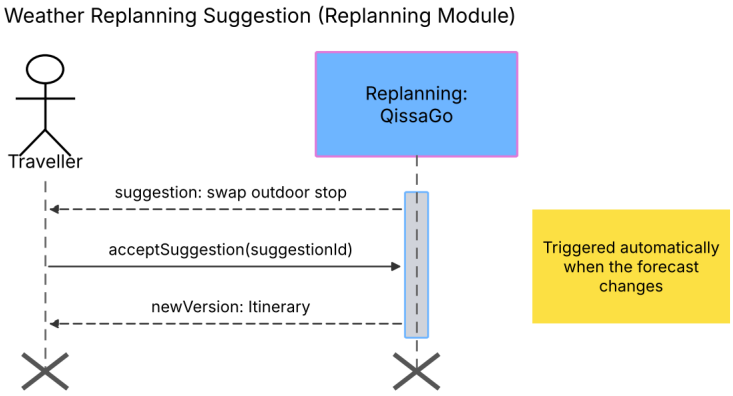


Figure 5.27: System Sequence Diagram for Weather Replanning function (Replanning Module)

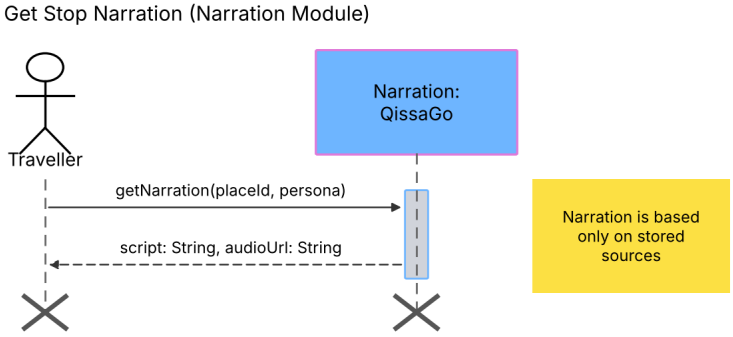


Figure 5.28: System Sequence Diagram for Get Stop Narration function (Narration Module)

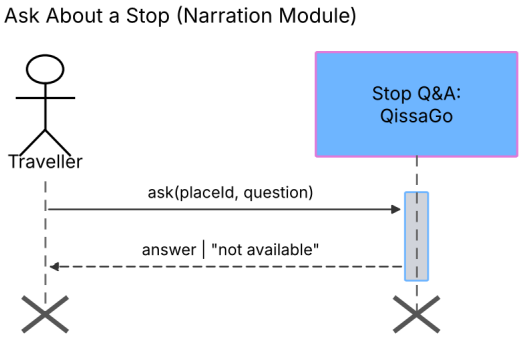


Figure 5.29: System Sequence Diagram for Ask About a Stop function (Narration Module)

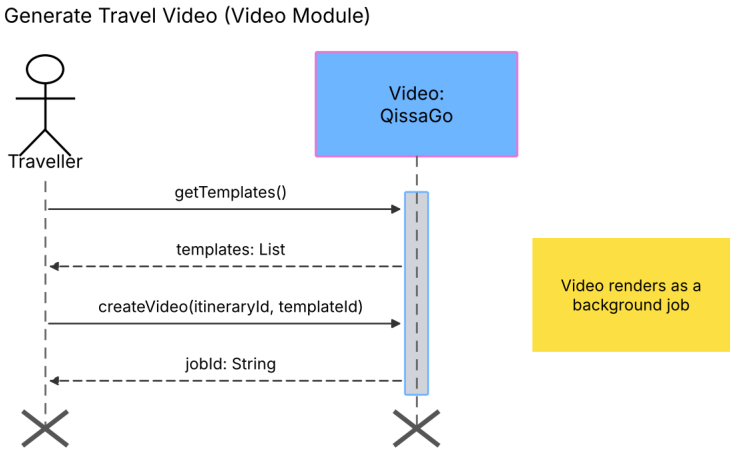


Figure 5.30: System Sequence Diagram for Generate Travel Video function (Video Module)

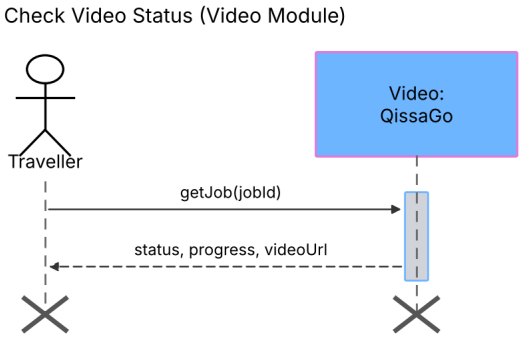


Figure 5.31: System Sequence Diagram for Check Video Status function (Video Module)

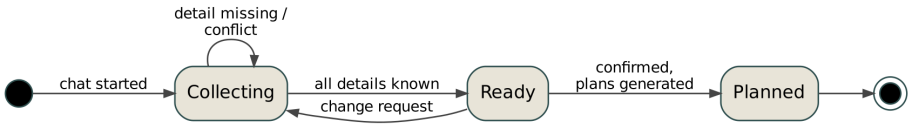


Figure 5.32: State diagram of a chat session

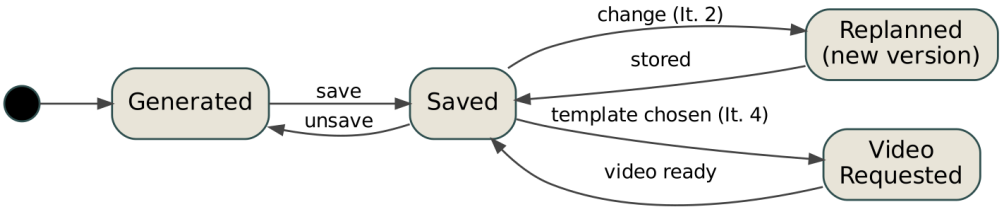


Figure 5.33: State diagram of an itinerary

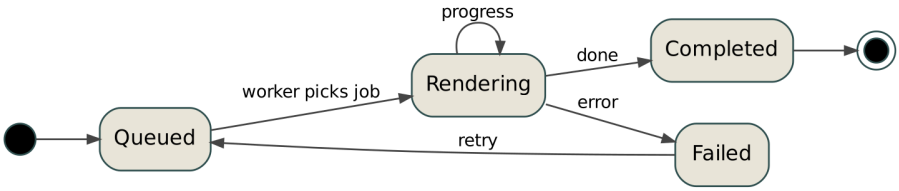


Figure 5.34: State diagram of a video job (Iteration 4)

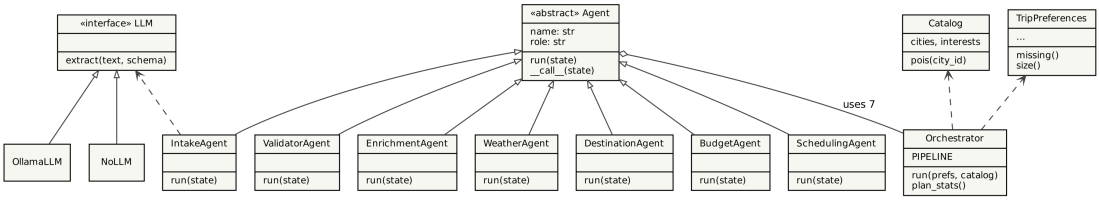


Figure 5.35: Class diagram of the agents and services

5.8.6 Deployment Diagram

Explanation of Figure 5.36:

- The Android app and the web app (published on GitHub Pages) connect to the backend over HTTPS.
- The server runs Docker containers for the FastAPI backend, PostgreSQL and Ollama; Redis and the render worker are added in later iterations.
- The backend calls Open-Meteo for weather; the apps load OpenStreetMap tiles directly.

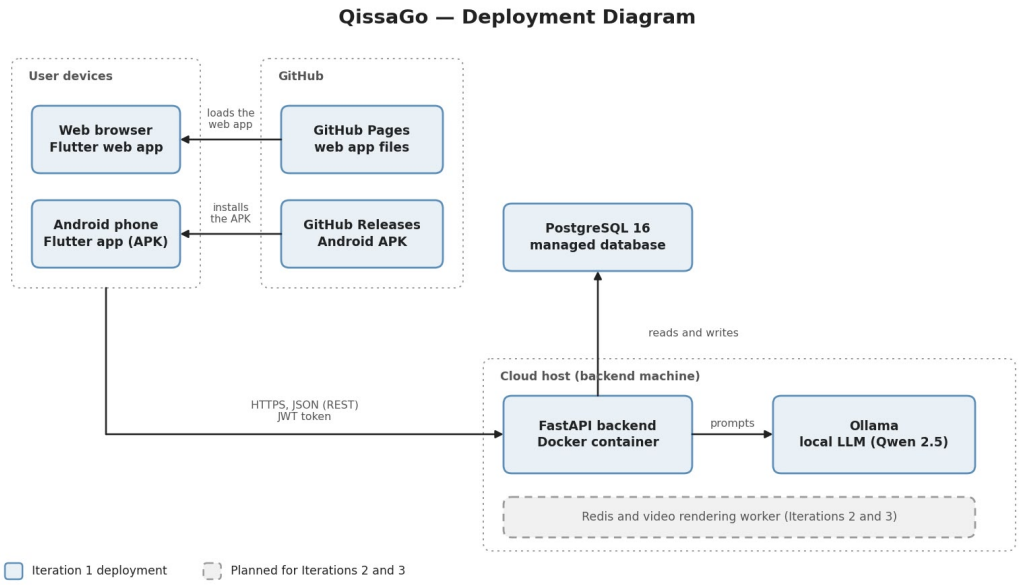


Figure 5.36: Deployment diagram of QissaGo

Chapter 6

Iteration 1: Design and Implementation

6.1 Iteration Objectives

Iteration 1 delivers the first core feature: personalized conversational itinerary planning. Its objectives (Table 3.3) are:

- **O1:** a user can describe a trip in natural English or with buttons.
- **O2:** missing or contradictory details are detected and clarified with targeted questions.
- **O3:** specialised agents produce three feasible, budget-respecting, weather-aware and ranked itineraries.
- **O4:** every stop shows its details, and each detail can be switched on or off.
- **O5:** conversations and itineraries are stored persistently for each user.
- **O6:** development follows industry practice: automated tests, CI/CD and a branch-based workflow.

6.2 Selected Use Cases / Features

Table 6.1: Use cases and features of Iteration 1

Use cases	**Feature**	**Scope delivered**
UC-01, UC-02, UC-03	Accounts	Register, log in, persistent session, profile, log out
UC-04, UC-05	Chatbot	Extraction of all trip details, one question at a time with suggestions, contradiction checks, confirmation
UC-06	Quick-select form	All trip details by buttons, including visitor type, home city and travel mode
UC-22, UC-25	Multi-agent planning	Seven agents and the Orchestrator, three variants, progress screen, weather forecast
UC-07 – UC-10, UC-26	Itinerary viewing	Ranked options, timeline, detail toggles, cost breakdown, day map, map and route links
UC-11 – UC-13	Itinerary management	Save, unsave, saved trips, all options of a generation, PDF export
UC-27	Local language model	Reading free text that the rules cannot understand

6.3 Design

The Iteration 1 design follows the architecture of Chapter 5. Its main design elements are:

6.3.1 Agent Pipeline

- One shared state object is passed through the agents in a fixed order: Intake → Weather → Destination → Budget → Scheduling → Validator → Enrichment.
- From the Destination Agent onward, three variants are built side by side: Balanced Explorer, Interest Deep-Dive and Smart Saver.
- Every agent adds a trace entry (name, message, duration) that is shown on the progress screen and stored in the `agent_runs` table.
- The Orchestrator computes statistics and the match score, ranks the plans and labels the best one.

6.3.2 Chatbot Design

- **Conversation manager:** keeps one preference object per chat and remembers the question asked last, so short answers such as "3" are understood in context; merges typed text and button selections into the same state.
- **Preference extractor:** a rule layer (patterns, aliases, fuzzy matching) runs first; the local language model is called with a JSON schema only for details the rules missed, and its values are validated.
- **Clarification handler:** picks the next missing detail in a fixed order, phrases the question with tappable suggestions and detects contradictions.

6.3.3 User Interface Design

- **Navigation shell:** bottom bar on phones (Home, Plan, Saved, Narration, Video; the last two marked as upcoming) and a top bar on wide screens that also shows Profile.
- **Plan flow:** chat or form → progress screen → ranked options → itinerary detail.
- **Itinerary detail:** summary, cost breakdown, day timeline, stop cards with detail toggles, day map; on wide screens the costs appear beside the timeline.
- **Back behaviour:** back first returns inside the current tab, then to Home, then closes the app.
- **Visual style:** short labels, imagery and a consistent colour palette; no emojis or instructional text.

6.4 Implementation

The system is implemented as one repository with a Python backend and a Flutter frontend.

```
qissago/
  backend/
    app/agents/          base, orchestrator, weather, destination,
                         budget, costs, scheduling, validator, enrichment
    app/agents/intake/   agent, conversation, extractor, clarifier
    app/api/             auth, meta, chat, itineraries, deps
    app/core/            config, security
    app/db/              models, session
```

```
  app/schemas/          preferences, api
  app/services/         catalog, geo, distances, weather, llm
  data/seed/            catalogue seed files
  scripts/              fetch_wikipedia, seed_db
  static/images/        locally stored place photos
  tests/                pytest test suite
frontend/
  lib/core/             config, models, api client, app state, format
  lib/screens/          auth, home, chat, quick plan, generating,
                        plans, plan detail, saved, profile
  lib/widgets/          shared widgets
  test/                 flutter test suite
.github/                workflows, templates, dependabot
docker-compose.yml
```

- **Backend:** FastAPI application with routers under `/api/v1`; SQLAlchemy models; agents written as plain classes that work on dictionaries, so they can be tested without a database.
- **Frontend:** one Flutter codebase compiled for Android and the web; state managed with Provider; API calls through a single client class.
- **Database:** PostgreSQL in Docker, with a separate test database used by the automated tests.
- **Content:** ingestion scripts import descriptions, coordinates and openly licensed images once; a seed script loads them into the database.

6.5 Completed Features / Module

Table 6.2: Completed modules of Iteration 1

Module	**Completed features**	**Status**
Authentication	Register, login, token sessions, persistent login, profile, logout, data isolation	Complete
Chatbot (Intake)	Conversation manager, rule extractor, local model with fallback, clarifier, contradiction checks, confirmation, chat history	Complete

Module	Completed features	Status
Quick-select form	Shared preference model; visitor type, home city and travel mode	Complete
Multi-agent planner	Seven agents, Orchestrator, three variants, match score, agent trace	Complete
Itinerary viewing and management	Ranked options, timeline, detail toggles, cost breakdown with road trip and vehicle hire, day map, links, save, saved trips, PDF export	Complete
Content and data	Database schema, ingestion of text, coordinates and licensed images, repeatable seeding, weather cache	Complete
Engineering	Automated tests, coverage gate, linting, CI/CD workflows, Docker setup	Complete

6.6 Technical Implementation Details

Table 6.3: Important files, classes and functions

File	Class / function	Purpose
`app/main.py`	`app`	FastAPI application, CORS, static media, routers
`app/core/config.py`	`Settings`	Database URL, token secret, model settings, allowed origins from the environment
`app/core/security.py`	`hash_password`, `verify_password`, `create_token`, `read_token`	bcrypt hashing; signed tokens valid for 72 hours
`app/api/deps.py`	`current_user`, `catalog`, `llm`	Logged-in user, cached catalogue, language model
`app/api/chat.py`	`start`, `send`, `get_session`	Chat endpoints; every message stored
`app/api/itineraries.py`	`generate`, `list_saved`, `batch`, `save`, `unsave`	Runs the planner; stores and lists plans
`app/schemas/preferences.py`	`TripPreferences`, `missing()`	Shared preference model; asking order

File	Class / function	Purpose
app/agents/intake/ extractor.py	extract_rules, clean_llm, extract	Rule layer, model validation, merge
app/agents/intake/ clarifier.py	next_question, check_conflicts, summary	Questions, contradictions, summary
app/agents/intake/ conversation.py	greeting, handle_turn	One chatbot turn
app/agents/base.py	Agent	Timing and trace entry for every agent
app/agents/ orchestrator.py	Orchestrator, plan_stats	Pipeline, statistics, match score, ranking
app/agents/ destination.py	VARIANTS, score	Variant weights; place scoring
app/agents/budget.py	BudgetAgent, trip_totals	Selection within budget; totals
app/agents/costs.py	meals, stay, rooms, cars, minimum_trip_cost	Shared cost rules
app/agents/ scheduling.py	SchedulingAgent, fit_cost	Days, order, time slots, opening hours
app/agents/validator.py	ValidatorAgent, problems	Plan checks and repair
app/agents/ enrichment.py	EnrichmentAgent, weather_note	Prices, weather notes, links, day totals
app/services/geo.py	haversine_km, leg, osm_link, directions_link	Distance, travel time, map links
app/services/ distances.py	–	Inter-city road distances for the road trip
app/services/weather.py	get_forecast, seasonal_day	Forecast, climate fallback, cache
app/services/llm.py	OllamaLLM, NoLLM, FakeLLM	Language model client with JSON schema
scripts/ fetch_wikipedia.py	–	One-time import of text, coordinates and images with credits

File	Class / function	Purpose
`scripts/seed_db.py`	–	Repeatable loading of the catalogue
`frontend/lib/core/api.dart`	`ApiClient`	HTTP client, bearer token, error messages
`frontend/lib/core/app_state.dart`	`AppState`	Login state and current user
`frontend/lib/screens/`	Screens	All application screens

Configuration

- `backend/.env`: database URL, token secret, model provider and name, live weather switch, allowed origins; excluded from version control.
- `docker-compose.yml`: PostgreSQL 16 with a health check and a separate test database.
- `pyproject.toml`: pytest and Ruff settings; `analysis_options.yaml`: Dart lint rules.
- `.github/workflows/`: backend CI, frontend CI, deployment and APK release.

6.7 Algorithms and Technical Decisions

6.7.1 Travel Distance and Time

$$d = 2R \arcsin\sqrt{\sin^2\left(\frac{\varphi_2-\varphi_1}{2}\right) + \cos\varphi_1 \cos\varphi_2 \sin^2\left(\frac{\lambda_2-\lambda_1}{2}\right)}, \qquad R = 6371 \text{ km} \tag{6.1}$$

- Great-circle (haversine) distance d between two places.
- Road distance $= 1.35 \times d$; travel time $= \max(8,\ 60 \times \text{road distance}/24)$ minutes.

6.7.2 Cost Model

For a group of n people and a trip of D days:

$$C_{total} = C_{entries} + C_{local} + C_{meals} + C_{stay} + C_{roadtrip} + C_{hire} \tag{6.2}$$

- rooms $= \lceil n/2 \rceil$, cars $= \lceil n/4 \rceil$, nights $= D - 1$.
- C_{meals} = meal rate $\times D \times n$; C_{stay} = hotel rate $\times$ nights $\times$ rooms (zero when no accommodation is needed).
- C_{local} = road kilometres of all legs $\times$ rate per km $\times$ cars.
- $C_{entries}$ = entry fee per person (by visitor type) $\times n$ for each stop.
- $C_{roadtrip}$ = travel from the home city and back by the chosen travel mode (fuel when travelling by own car).
- C_{hire} = 4×4 vehicle hire for stops that can only be reached by jeep.
- The minimum trip cost (meals, budget stay and a minimum daily local travel) is used by the chatbot to reject budgets that are too low.

6.7.3 Place Scoring

$$s_v(p) = w^v_{int} I(p) + w^v_{pri} \frac{\text{priority}(p) - 1}{2} + w^v_{cost} \left(1 - \frac{\min(e_p, 3000)}{3000}\right) \tag{6.3}$$

where $I(p)$ is the share of the traveller's interests found in the place's tags and e_p is the entry cost per person.

Table 6.4: Variant weights

Variant	w_{int}	w_{pri}	w_{cost}	**Hotel**
Balanced Explorer	0.50	0.30	0.20	mid
Interest Deep-Dive	0.75	0.20	0.05	mid
Smart Saver	0.35	0.20	0.45	budget

- A place with an excluded tag has its score multiplied by 0.4.
- Family places gain 0.15 when travelling with children; adventure places are halved with elderly travellers.
- Outdoor places are reduced for an indoor preference and on rainy or very hot days.

6.7.4 Daily Rhythm

Table 6.5: Pace and scheduling rules

Rule	**Value**
Relaxed / Balanced / Packed	3 stops from 10:00 / 4 stops from 09:00 / 5 stops from 08:30
Children or elderly	At most 3 stops per day
Late start	First stop not before 11:00
Lunch break	12:30 for 45 minutes
End of day	All visits finish by 21:30
Rainy day / hot day	Rain probability $\geq$ 50% / maximum temperature $\geq$ 36 °C
Hot days	Outdoor places scheduled in the evening where possible

6.7.5 Budget-Constrained Selection and Scheduling

For each variant the Budget and Scheduling agents follow these steps:

1. Start with the budget and subtract meals, accommodation and the road trip.
2. If the remainder is negative with a mid-range hotel, switch to a budget hotel and recompute.
3. Sort places by their score for this variant, with a small novelty bonus so the three plans differ.
4. Take places in order while their cost and expected travel still fit the remaining budget, until the number of stops for the pace is reached.
5. Group the selected places into days by location; put outdoor places on dry days.
6. Order each day by best visiting time and nearest neighbour; assign time slots with travel, lunch and opening hours.
7. While the total with real travel costs is above the budget, drop the weakest stop and rebuild that day.

6.7.6 Match Score

$$\text{Score} = 35F_{int} + 20F_{budget} + 15F_{full} + 10F_{weather} + 10F_{travel} + 10F_{must} \tag{6.4}$$

- F_{int}: share of the traveller's interests covered.
- F_{budget}: how well the plan fits the budget.
- F_{full}: stops scheduled divided by stops expected for the pace.
- $F_{weather}$: outdoor stops placed on good-weather days.
- F_{travel}: travel efficiency, $\max(0, \min(1, (15 - \bar{d})/12))$ for average leg length $\bar{d}$ km.
- F_{must}: share of must-see places included.

6.7.7 Technical Decisions

- **Algorithms for numbers:** money, time, distance and opening hours are computed by code, so results are exact, fast and testable.
- **Rules before the model:** the rule layer handles most messages; the local model (Qwen 2.5, 3B, through Ollama) fills only what is missing, with a JSON schema, temperature 0 and a timeout.
- **Validated model output:** any value from the model is accepted only if it matches the catalogue and allowed ranges.
- **Deterministic agents:** the same preferences and weather always produce the same plans.
- **Local content:** place text and images are imported once and stored locally with their licences.

6.8 Tools, Frameworks and Technologies

Table 6.6: Tools, frameworks and technologies

Area	**Technology**	**Purpose**
Frontend	Flutter, Dart, Material 3	Android and web application
Frontend packages	provider, http, shared_preferences, cached_network_image, flutter_map, url_launcher, google_fonts, intl, pdf, printing	State, API calls, storage, images, maps, links, fonts, formatting, PDF

Area	Technology	Purpose
Backend	Python 3.12, FastAPI, Uvicorn, Pydantic v2	REST API and validation
Database	PostgreSQL 16, SQLAlchemy 2, psycopg 3	Persistence
Security	bcrypt, PyJWT	Password hashing, tokens
Language model	Ollama, Qwen 2.5 (3B)	Reading free text
Data import	requests, Pillow	Ingestion, image resizing
External services	Open-Meteo, OpenStreetMap, Wikipedia, Wikidata, Wikimedia Commons	Weather, maps, content
Testing	pytest, pytest-cov, FastAPI TestClient, flutter_test, http MockClient	Automated tests and coverage
Code quality	Ruff, dart format, flutter analyze	Linting, formatting, static analysis
DevOps	Git, GitHub, GitHub Actions, Dependabot, Docker, Docker Compose	Version control, CI/CD, containers
Development	Visual Studio Code, Android Studio, Android Emulator, Chrome	Development and testing

6.9 Demonstration Evidence

Iteration 1 was demonstrated as a running application on the web and on Android. The evidence is:

Table 6.7: Demonstration evidence

No.	Demonstrated	Where it can be verified
1	Register, log in, stay logged in	Running app; API tests
2	Plan a trip by chat with follow-up questions	Running app; conversation tests
3	Plan a trip by the quick-select form	Running app; widget tests
4	Progress screen with all seven agents	Running app; `agent_runs` table
5	Three ranked itineraries with match score	Running app; agent tests

No.	Demonstrated	Where it can be verified
6	Itinerary detail, toggles, cost breakdown, map	Running app; widget tests
7	Save, saved trips, PDF export	Running app; API tests
8	Interactive API documentation	`/docs` page of the backend
9	Automated checks on every change	GitHub Actions workflow runs
10	Released version	GitHub release v0.1.0 with Android APK

6.10 Iteration Outcome

Table 6.8: Iteration 1 outcome

Obj.	Result	Verified by
O1	Chat and form fill one shared preference model	TC-01 – TC-16, TC-40, TC-62
O2	Missing and contradictory details are clarified	TC-19 – TC-27
O3	Three valid, ranked plans for every tested trip profile	TC-28 – TC-34
O4	All stop details shown and switchable	TC-M4
O5	Data stored per user and isolated	TC-39, TC-42
O6	103 backend tests (93% coverage), 50 app tests, CI/CD green	Chapter 7

- Iteration 1 is complete and released as `v0.1.0`.
- The agent design allows the Replanning Agent of Iteration 2 to be added without changing the existing agents.

Chapter 7

Testing and Early Validation

7.1 Testing Strategy

Testing was planned together with the design, and every behaviour added in Iteration 1 is covered by an automated test. The strategy follows these principles:

- **Exact tests for deterministic parts.** The agents, cost rules and chatbot rules have no randomness, so their output is checked against exact expected values.
- **External services replaced in automated tests.** The language model is replaced by a fake model and network responses of the ingestion script are mocked, so results are repeatable. The real model is checked during manual testing.
- **Integration tests on a real database.** API tests run against a separate PostgreSQL test database that is recreated and seeded for every run.
- **Property checks over many trip profiles.** Agent tests generate plans for solo, couple, family with children, friends, elderly travellers, small and large budgets, and check rules that must hold for every plan.
- **Continuous testing.** All tests, linting and static analysis run in GitHub Actions on every push and pull request; a failing check blocks merging.
- **Coverage gate.** Backend coverage must stay above 70%.

7.2 Test Levels and Techniques

Table 7.1: Test levels and techniques

Level	**Scope**	**Tool**	**Technique**
Unit	Preference extractor	pytest (parametrised)	Equivalence partitioning, boundary values
Unit	Conversation manager, clarification handler	pytest	Scenario-based, state transitions
Unit	Planning agents	pytest	Property checks over trip profiles
Unit	Cost rules	pytest	Boundary values, expected totals
Unit	Catalogue and seed data	pytest	Data validation rules
Integration	API with database	pytest, FastAPI TestClient, PostgreSQL	End-to-end request flows
Integration	Ingestion script	pytest, mocked HTTP	Fake external responses
Unit	App models, formatting, API client	flutter_test, http MockClient	Expected output, error handling
Widget	Screens and navigation	flutter_test	Rendering and interaction on phone and wide layouts
Static	Whole codebase	Ruff, flutter analyze, dart format	Lint and format rules
Acceptance	Complete app on web and Android	Manual	Scenario tests TC-M1 – TC-M8

7.3 Test Cases

`<city>` denotes any supported destination and `<place>` any place in the catalogue.

7.3.1 Chatbot Test Cases

Table 7.2: Chatbot test cases

ID	Input / action	Expected result	Result
TC-01	"3 days in <city>"	Destination found; 3 days	Pass
TC-02	"trip to <city> for a week"	7 days	Pass
TC-03	"from <city A> to <city B>"	Destination is city B	Pass
TC-04	Misspelt destination name	Correct destination	Pass
TC-05	"2 nights in <city>"	3 days	Pass
TC-06	"budget 50k", "Rs 40,000", "1.5 lakh", "75000 rupees", "PKR 2 lakh"	50000, 40000, 150000, 75000, 200000	Pass
TC-07	"20k per person"	Budget multiplied by group size	Pass
TC-08	"3" when asked for days	Days = 3, not a budget	Pass
TC-09	tomorrow, next friday, this weekend, 20 dec, 25/12, in 2 weeks	Correct dates; past dates move to next year	Pass
TC-10	"20-22 december"	Start date and 3 days	Pass
TC-11	"1.5 lakh"	Budget, never a date	Pass
TC-12	just me / with my wife / family of 5 / 2 adults and 3 kids / 6 friends	solo-1 / couple-2 / family-5 / family-5 / friends-6	Pass
TC-13	"history and food but no shopping"	Two interests and one exclusion	Pass
TC-14	"Family fun" option	Interest only; group type unchanged	Pass
TC-15	"skip <place>"	Only that place excluded	Pass
TC-16	Relaxed pace, grandparents, prefer indoors	Pace and special needs set	Pass
TC-17	Model answer conflicts with rules; unknown values	Rules win; invalid values dropped	Pass
TC-18	Language model unavailable	Rules still extract all details	Pass
TC-19	Start a chat	Greeting with the user's name and options	Pass
TC-20	Answer step by step	Questions asked in the defined order	Pass
TC-21	One detailed message, then "Create plans"	Ready; generation action returned	Pass
TC-22	Budget below the minimum trip cost	Minimum explained; budget asked again	Pass

ID	Input / action	Expected result	Result
TC-23	Start date in the past	Date asked again	Pass
TC-24	Unsupported destination	Supported destinations offered	Pass
TC-25	Button selection followed by text	One shared preference state	Pass
TC-26	“change budget” at the summary	Budget asked again	Pass
TC-27	Unclear message	Same question asked politely again	Pass

7.3.2 Planner and Cost Test Cases

Table 7.3: Planner and cost test cases

ID	Input / action	Expected result	Result
TC-28	Profiles: family with kids, couple, friends, solo, elderly, low and high budget	3 plans; within budget; no overlaps; within opening hours and pace; no duplicate places; ranked	Pass
TC-29	Generation trace	All agents ran in order with messages and durations	Pass
TC-30	Excluded category and excluded place	Never included in any plan	Pass
TC-31	Specific interests	Deep-Dive plan has the highest interest coverage	Pass
TC-32	Compare the three variants	Smart Saver has the lowest total	Pass
TC-33	Late start	No stop before 11:00	Pass
TC-34	Incomplete preferences	Rejected by the Intake Agent	Pass
TC-49	Rooms, cars and nights for 1, 2, 4, 5 people and 1 day	Correct counts at the boundaries	Pass
TC-50	Minimum trip cost; no accommodation needed	Correct minimum; stay cost is zero	Pass
TC-51	Road trip from the home city by own car	Road-trip cost calculated as fuel	Pass
TC-52	Stop reachable only by jeep	Vehicle hire added; legs marked	Pass
TC-53	Plan totals	Parts add up to the total; amount left correct	Pass

7.3.3 API, Data and Ingestion Test Cases

Table 7.4: API, data and ingestion test cases

ID	Input / action	Expected result	Result
TC-35	Health check	200 OK	Pass
TC-36	Register; duplicate email; wrong password; login; profile	201 / 409 / 401 / 200 / profile	Pass
TC-37	Missing or invalid token	401	Pass
TC-38	Planning form options	Destinations, interests, paces, groups, needs	Pass
TC-39	Chat → generate → save → list → options → unsave	Full flow works; messages, plans and trace stored	Pass
TC-40	Generate from form preferences	Correct plans	Pass
TC-41	Generate with missing details	400 listing the missing fields	Pass
TC-42	Request another user's itinerary	404	Pass
TC-43	Import with a mocked source	Image resized; credit and licence stored	Pass
TC-44	Missing source page	Skipped without failure	Pass
TC-54	Seed files	Unique ids; every reference valid	Pass
TC-55	Place fields	Valid coordinates, opening hours, categories and priorities	Pass
TC-56	Seed script run twice	Same database contents	Pass

7.3.4 Application (Frontend) Test Cases

Table 7.5: Application test cases

ID	Input / action	Expected result	Result
TC-45	Itinerary and chat JSON	Parsed correctly	Pass
TC-46	Currency, duration and date formatting	Correct output	Pass
TC-47	Login request; server error; token	JSON sent and token read; message shown; bearer header sent	Pass

ID	Input / action	Expected result	Result
TC-48	Score ring, labels, image placeholder, retry button	Render and respond	Pass
TC-57	Navigation at phone width	Five tabs; Narration and Video marked as upcoming	Pass
TC-58	Navigation on a wide screen	All six tabs in the top bar	Pass
TC-59	System back button	Back inside the tab, then Home, then app closes	Pass
TC-60	Itinerary detail on a wide screen	Costs beside the timeline	Pass
TC-61	Plans with road trip and jeep hire; car travel; older plans	Own labels; fuel label; older plans still shown; jeep legs marked	Pass
TC-62	Submit the planning form	Visitor type, home city and travel mode sent	Pass
TC-63	Open Profile	Name, email, back and log out; returns to previous tab	Pass
TC-64	Saved tab before and after saving	Empty state, then the saved plan	Pass
TC-65	Plan options screen	Back to chat and Home actions	Pass
TC-66	Chat button on Home	Opens the Plan tab in chat mode	Pass
TC-67	Choose a destination on Home	Form opens with that destination	Pass

7.4 Test Results

7.4.1 Backend

All 103 backend tests passed in 10.6 seconds with 93% statement coverage, well above the 70% gate.

Table 7.6: Backend test results by file

Test file	Covers	Tests	Passed
`test_extractor.py`	Preference extractor	34	34
`test_conversation.py`	Conversation manager, clarification	22	22
`test_agents.py`	Planning agents, orchestrator	15	15
`test_data_scripts.py`	Seed data and scripts	11	11

test_api.py	API with database	10	10
test_costs.py	Cost rules	9	9
test_fetch_wikipedia.py	Ingestion script	2	2
Total		**103**	**103**

Table 7.7: Backend coverage by module

Module	Statements	Missed	Coverage
agents/base.py	15	0	100%
agents/budget.py	69	4	94%
agents/costs.py	49	0	100%
agents/destination.py	51	4	92%
agents/enrichment.py	42	7	83%
agents/intake/agent.py	26	1	96%
agents/intake/clarifier.py	74	7	91%
agents/intake/conversation.py	156	19	88%
agents/intake/extractor.py	358	24	93%
agents/orchestrator.py	55	0	100%
agents/scheduling.py	117	4	97%
agents/validator.py	50	4	92%
agents/weather.py	15	0	100%
api/auth.py	29	0	100%
api/chat.py	48	2	96%
api/deps.py	20	1	95%
api/itineraries.py	94	7	93%
api/meta.py	11	1	91%
core/config.py	23	0	100%
core/security.py	24	2	92%
db/models.py	112	0	100%
db/session.py	22	0	100%
main.py	24	0	100%
schemas/api.py	62	0	100%

Module	Statements	Missed	Coverage
schemas/preferences.py	54	0	100%
services/catalog.py	24	0	100%
services/distances.py	26	5	81%
services/geo.py	29	0	100%
services/llm.py	44	17	61%
services/weather.py	65	23	65%
Total	**1788**	**132**	**93%**

- The lower coverage of services/llm.py and services/weather.py is expected: those lines make the real calls to the local model server and the live weather service, which are replaced by fakes in automated tests. Both paths were checked manually.

7.4.2 Application

Table 7.8: Application test results

Test file	Covers
api_test.dart	Login request and token, server error messages, bearer header
budget_ui_test.dart	Cost-part labels (road trip, jeep hire, fuel), older plans, jeep legs, planning form fields
shell_test.dart	Navigation on phone and wide screens, upcoming tabs, profile, back behaviour, saved tab, chat shortcut, form reset
design_test.dart	Itinerary detail layout on wide screens
widget_test.dart	Score ring and shared widgets
Result	**50 tests, all passed; static analysis: no issues found**

7.5 Acceptance Testing / Validation

The complete application was tested manually on the web (Chrome) and on Android (emulator) against the acceptance criteria of Table 4.13.

Table 7.9: Manual acceptance tests

ID	Scenario	Expected result	Web	Android
TC-M1	Register, log in, close and reopen the app	Still logged in	Pass	Pass
TC-M2	Plan a trip by chat from start to end	Three ranked plans shown	Pass	Pass
TC-M3	Plan a trip by the form from start to end	Three ranked plans shown	Pass	Pass
TC-M4	Show and hide each stop detail	Detail appears and disappears	Pass	Pass
TC-M5	Open map, route and source links	Correct external page opens	Pass	Pass
TC-M6	Save, reopen, view all options, unsave	Saved list updates	Pass	Pass
TC-M7	Export an itinerary as PDF	PDF with summary, costs and all days	Pass	Pass
TC-M8	Stop the server and use the app	Error message with retry	Pass	Pass

Table 7.10: Acceptance criteria status

Criteria	Area	Status
AC-01 – AC-02	Accounts and persistent login	Met
AC-03 – AC-07	Chatbot extraction, contradictions, confirmation, work without the model	Met
AC-08 – AC-12	Planner: three ranked plans, constraints, exclusions, late start, cost parts	Met
AC-13 – AC-15	Saving, data isolation, PDF export	Met
AC-16	Coverage and CI checks	Met
AC-17 – AC-19	Replanning, narration and video	Planned (Iterations 2–4)

7.6 Edge and Failure Cases

Table 7.11: Edge and failure cases covered

No.	Case	System behaviour
1	Unsupported destination	Supported destinations offered as suggestions
2	Misspelt destination or place name	Matched by aliases and fuzzy matching
3	Start date in the past; date already passed this year	Asked again; date moved to next year
4	Budget below the minimum trip cost	Minimum explained; budget asked again
5	Group size contradicts group type	Correction requested
6	Number given in the wrong context (“3” when asked for days)	Read using the question asked last
7	Negated interests; excluding a single place	Only that interest or place is excluded
8	Language model unavailable or returns invalid values	Rules used; invalid values discarded
9	Weather service unreachable; trip beyond forecast range	Typical climate used
10	Place closed at the planned time	Moved or removed by Scheduling and Validator
11	Day running too late	Last stops removed by the Validator
12	Plan over budget after real travel costs	Weakest stop dropped and day rebuilt
13	Plans saved before new cost fields existed	Still displayed correctly
14	Missing source page or image during import	Item skipped; placeholder image in the app
15	Expired or invalid token	401; user sent to login
16	Access to another user’s data	404
17	Empty message	400 with a message

7.7 Defects and Corrective Actions

Table 7.12: Defects found and corrective actions

ID	Defect	Corrective action	Regr. test
D-01	“1.5 lakh” was read as a date	Money parsing runs before date parsing; “.” no longer a date separator	Yes
D-02	Excluding a specific place also excluded a whole interest	Place names removed from the text before interest detection	Yes
D-03	The “Family fun” option set the group type to family	Group pattern corrected	Yes
D-04	The bot confirmed a group size the user had not given	Only values given by the user are acknowledged	Yes
D-05	Meal costs counted twice for food places, leaving one plan with too few stops	Cost model corrected	Yes
D-06	Match score did not penalise nearly empty plans	Fullness factor added to the score	Yes
D-07	Unclear input could trigger generation at the confirmation step	Generation only on explicit confirmation without changes	Yes
D-08	Chat API response dropped the action field	Response schema extended	Yes
D-09	Local database service conflicted with the container port during setup	Setup procedure updated	–

7.8 Current Validation Status

Table 7.13: Validation status at the end of Iteration 1

Item	Status
Backend automated tests	103 of 103 passed
Backend coverage	93% (gate: 70%)
Application automated tests	50 of 50 passed
Static analysis	No issues
Manual acceptance tests	8 of 8 passed on web and Android

Acceptance criteria AC-01 – AC-16	All met
Open defects	None

- All Iteration 1 requirements are validated.
- A wider evaluation (a chatbot accuracy set of realistic messages and a usability study) is planned for the remaining iterations (Section 9.5).

Chapter 8

Project Management and Development Evidence

The project follows an iterative model. Each iteration is time-boxed, split into GitHub issues, built on feature branches, reviewed through pull requests and released with a version tag. Changes are merged into `develop` only after one review and passing automated checks, and `develop` is merged into `main` for each release.

8.1 Work Completed

Table 8.1: Work completed in Iteration 1

No.	Work item	Output	Status
1	Scope revision after the proposal	Four iterations; reduced, focused modules	Done
2	Research	Existing solutions compared; design decisions	Done
3	Requirements	Use cases, functional and non-functional requirements, acceptance criteria	Done
4	System design	Architecture, data design, process models	Done
5	Environment and repository	Docker setup, repository structure, branch rules, CI	Done
6	Database and content	Schema, ingestion and seed scripts	Done
7	Authentication	API, screens, token handling	Done

No.	Work item	Output	Status
8	Chatbot	Extractor, clarifier, conversation manager, model client	Done
9	Planner	Seven agents, Orchestrator, match score	Done
10	Application screens	Home, chat, form, progress, options, detail, saved, profile	Done
11	Cost breakdown	Road trip, vehicle hire, visitor type	Done
12	Saving and PDF export	Save, saved trips, PDF	Done
13	Testing	103 backend tests, 50 app tests, manual acceptance tests	Done
14	Release	`v0.1.0`, Android APK, web build	Done

8.2 Work Remaining

Table 8.2: Work remaining

It.	Work	Main output
2	Replanning Agent, change requests in chat, itinerary versions, weather suggestions, Redis cache	Editable itineraries
3	Source documents and retrieval, Narration Agent with personas, Voice Agent, Stop Q&A Agent, narration player	Narrated stops
4	Video templates, Caption and Video agents, render worker and queue, video player, deployment, usability study	Travel video, final release

8.3 Individual Responsibilities

Table 8.3: Individual responsibilities

Member	**Role**	**Iteration 1**	**Iterations 2 to 4**
Dania Waseem	Backend and DevOps	REST API, authentication, database schema, ingestion and seed scripts, Docker, CI/CD workflows	Redis, job queue, video rendering worker, deployment
Munaza Tariq	Agents and conversational intelligence	Conversation manager, preference extractor, clarification handler, language model integration, planning agents, Orchestrator	Replanning, Narration, Voice, Stop Q&A, Caption and Video agents; retrieval
Wajeeha Khalid	Mobile and web application	Flutter screens, navigation, state management, maps, cost breakdown, PDF export, responsive layout	Narration player, video templates and player, usability study
All members	Shared	Test design, code reviews, documentation, report, supervisor meetings	Same

8.4 Contribution of Each Team Member

Table 8.4: Contribution of each team member in Iteration 1

Member	**Main contributions**	**Tests and reviews**
Dania Waseem	REST API and routers; authentication and token security; database models; ingestion and seed scripts; Docker Compose; GitHub Actions workflows; release builds	API, data script and ingestion tests; reviews of agent and app changes
Munaza Tariq	Preference extractor; clarification handler; conversation manager; local model client; Intake, Weather, Destination, Budget, Scheduling, Validator and Enrichment agents; Orchestrator and match score; cost rules	Extractor, conversation, agent and cost tests; reviews of API changes

Member	Main contributions	Tests and reviews
Wajeeha Khalid	Flutter screens and navigation shell; chat and form; progress screen; itinerary options and detail; cost breakdown; maps; saved trips; PDF export; responsive layout	Widget, navigation and API client tests; manual acceptance tests; reviews of backend changes

- Every pull request was reviewed by another member before merging.
- All members took part in requirements, design, testing and the report.

8.5 Tools and Tool-Assisted Development

Table 8.5: Development tools

Tool	How it was used
Visual Studio Code	Main editor with Python, Dart/Flutter, Docker, SQLTools and GitLens extensions; debug configurations
Android Studio and Emulator	Android SDK and device testing
Docker and Docker Compose	PostgreSQL with a test database; the same setup for all members
FastAPI interactive documentation	Testing endpoints during development
Ruff, dart format, flutter analyze	Automatic linting and formatting
pytest-cov, flutter test	Coverage reports
GitHub Actions	Automatic tests and builds on every change
Dependabot	Automatic dependency update pull requests
GitHub Issues and Projects	Task board for each iteration
Git and GitHub	Version control, branches, pull requests, releases
LaTeX	Report writing

8.6 External Components and Dependencies

Table 8.6: External components and licences

Component	Purpose	Licence / terms
Flutter, Dart	Application framework	BSD-3-Clause
FastAPI, Pydantic, Uvicorn	API	MIT / BSD
SQLAlchemy, psycopg	Database access	MIT / LGPL
PostgreSQL	Database	PostgreSQL Licence
Ollama	Local model server	MIT
Qwen 2.5 (3B)	Language model	Qwen research licence
bcrypt, PyJWT	Security	Apache-2.0 / MIT
flutter_map	Maps in the app	BSD-3-Clause
pdf, printing	PDF export	Apache-2.0
Open-Meteo	Weather	Free for non-commercial use; data CC BY 4.0
OpenStreetMap	Map tiles	ODbL; attribution shown
Wikipedia, Wikimedia Commons	Text and images	CC BY-SA and per-image licences; credits stored

8.7 Risks and Mitigation

Table 8.7: Risks and mitigation

No.	Risk	Prob.	Impact	Mitigation
R1	Paid APIs not allowed	High	High	Local model, rules and free services
R2	Language model wrong, slow or offline	Medium	Medium	Rules first, validated output, fallback, small model
R3	Prices and opening hours change	Medium	Medium	Ingestion scripts re-run; sources recorded

No.	Risk	Prob.	Impact	Mitigation
R4	Weather service unavailable	Low	Low	Climate fallback and cache
R5	Image licensing	Low	Medium	Openly licensed images only; credits stored and shown
R6	Narration invents facts (It. 3)	Medium	High	Narration grounded in stored sources
R7	Slow video rendering (It. 4)	Medium	Medium	Background jobs, short templates, compressed media
R8	Free hosting limits	Medium	Medium	Complete local Docker setup for demonstrations
R9	Scope growth	Medium	High	Fixed iteration boundaries
R10	A member unavailable	Low	Medium	Shared reviews and documented code

8.8 Security and Privacy Considerations

- Passwords are stored only as bcrypt hashes.
- Signed tokens with a 72-hour expiry protect every private endpoint.
- Users can access only their own chats and itineraries (tested).
- Secrets are kept in an environment file excluded from the repository.
- All input is validated for type, length, range and email format.
- The language model runs locally, so user messages never leave the server.
- Only a name, an email address, chats and itineraries are stored for each user.
- Image credits and licences are recorded and shown.

8.9 Current Limitations

- The system works in English only.
- Destinations are limited to those in the database.
- Travel times are calculated from distance, not live traffic.

- Prices are not live.
- Itineraries cannot yet be edited after generation (Iteration 2).
- Weather beyond 16 days uses typical climate data.
- Travel to the destination is by road only.

Chapter 9

Remaining FYP Plan

9.1 Remaining Scope

- **Iteration 2 – Dynamic replanning:** edit an itinerary through chat, rebuild only the affected days, and suggest changes when the weather changes.
- **Iteration 3 – Cinematic narration:** storytelling narration at selected stops from stored sources, in several personas, as text and audio; questions about a stop answered from the same sources.
- **Iteration 4 – Personalized travel video:** video templates, captions, automatic assembly and rendering; final deployment and user evaluation.
- **Out of scope:** safety alerts, predictive disruption forecasting and personalized recommendation features, removed after the proposal review.

9.2 Remaining Modules and Features

Table 9.1: Remaining modules and features

Module	Features	It.	Requirements
Replanning	Replanning Agent; add, remove, move, start later, change day; itinerary versions; weather suggestions; Redis cache	2	FR-7.1 – FR-7.5
Narration	Source documents and retrieval; Narration Agent with personas; Voice Agent; Stop Q&A Agent; narration player; caching	3	FR-8.1 – FR-8.6

Module	Features	It.	Requirements
Video	Video templates; Caption Agent; Video Agent; render worker and queue; progress; play, download and share	4	FR-9.1 – FR-9.6
Release	Production deployment, performance tuning, usability study	4	NFR-U1, NFR-U6

9.3 Remaining Iterations

Table 9.2: Remaining iterations

It.	Objective	Deliverables	Exit criteria
2	Editable, weather-aware itineraries	Replanning Agent, versions, chat editing, Redis; release `v0.2.0`	Only affected days change; budget, hours and pace still respected; all tests pass
3	A story at every selected stop	Sources, retrieval, Narration, Voice and Stop Q&A agents, player; release `v0.3.0`	No facts outside the sources in review; at least two personas
4	Personalized travel video and final product	Templates, Caption and Video agents, worker, player, deployment, study; release `v1.0.0`	Every template renders; SUS score of at least 68

9.4 Implementation Plan

9.4.1 Iteration 2 – Replanning

- Add chat intents for add, remove, move, start later and change day, read by the rules and the local model.
- Compare the old and new plan inputs to find the affected days, and re-run the planning agents only for those days.
- Store each accepted result in `itinerary_versions` and show the differences in the app.
- Check the forecast for saved trips every day and suggest indoor and outdoor swaps.

9.4.2 Iteration 3 – Narration

- Store source documents for each place; split them into chunks; embed the chunks with a free sentence-embedding model; store them with the pgvector extension.
- Retrieve the most relevant chunks and ask the local model to write a story in the chosen persona using only that text.
- Convert scripts to audio with a free text-to-speech engine; cache scripts and audio.
- Answer questions about a stop from the retrieved chunks, or reply “not available”.

9.4.3 Iteration 4 – Video

- Define templates as data: pacing, transitions, text style and music.
- The Caption Agent writes one line per stop from the narration.
- The Video Agent builds a timeline from photos, clips, captions, voiceover and music, and renders it with FFmpeg and MoviePy.
- Rendering jobs are queued in Redis and processed by a worker; progress is shown in the app.

9.5 Testing and Validation Plan

Table 9.3: Testing and validation plan

Area	**Method**	**Target**
All new code	Unit and integration tests in CI	All pass; coverage above 70%
Chatbot	About 50 realistic messages; accuracy per field with and without the model	At least 90% field accuracy
Replanning	Tests for each change type; unaffected days compared	Unaffected days identical
Narration	Scripts checked against the sources; persona review	No facts outside the sources
Stop Q&A	Question set per stop; rates of correct, “not available” and wrong answers	Wrong answers below 10%

Area	Method	Target
Video	Render test for each template; duration and file-size checks	All templates render
Usability	5–10 users: task success, time and System Usability Scale	SUS of at least 68
Performance	Planning, chat, narration and rendering times	NFR-P1 – NFR-P6

9.6 Deployment Plan

- **Backend:** Docker containers (API, PostgreSQL, Ollama, Redis, render worker) on a cloud host.
- **Web app:** published to GitHub Pages by the deployment workflow on every merge into `main`.
- **Android app:** APK attached to each GitHub release by the release workflow.
- **Configuration:** secrets supplied as environment variables; the catalogue seeded on deployment.
- **Fallback:** a complete local setup with Docker Compose for demonstrations.

9.7 Tentative Timeline

Table 9.4: Tentative timeline

Task	Nov	Dec	Jan	Feb	Mar	Apr	May	Jun
It. 2: Replanning	■	■						
FYP-1 final evaluation			■					
It. 3: Sources and retrieval			■	■				
It. 3: Narration, voice, player				■	■			
It. 4: Templates and captions					■	■		
It. 4: Video agent and worker						■	■	
Usability study, tuning							■	
Deployment, release `v1.0.0`							■	■

Task	Nov	Dec	Jan	Feb	Mar	Apr	May	Jun
Final report and demonstration							■	■

9.8 Expected Final Deliverables

Table 9.5: Expected final deliverables

No.	Deliverable	Description
1	Android application	APK with planning, replanning, narration and video
2	Web application	Published web build
3	Backend API	Deployed service with interactive documentation
4	Database and content pipeline	Schema, ingestion and seed scripts
5	Source code repository	Full history, CI/CD workflows and tests
6	Evaluation reports	Test results, chatbot and narration evaluation, usability study
7	User and setup guide	Installation and use
8	Final report	FYP-2 report
9	Demonstration video	Walk-through of the final system

Chapter 10

Conclusion

- **Problem.** Planning a trip needs many decisions that depend on each other: places, days, travel, costs, opening hours and weather. Existing tools cover only parts of the journey, and none connects planning, changes, stories and a travel video in one system.
- **Research findings.** The comparison of existing solutions showed this gap. The main design finding is that a language model should be used for language, while exact algorithms handle money, time, distance and opening hours.
- **Revised scope.** After the proposal review the scope was reduced and reorganised into four iterations, with the personalized itinerary and the personalized travel video as the core features. Safety alerts, disruption forecasting and recommendation features were removed.
- **Requirements.** Use cases, numbered functional requirements for every module, measurable non-functional requirements, acceptance criteria and a traceability matrix define what the system must do (Chapter 4).
- **Design.** A four-tier architecture with a multi-agent core, a PostgreSQL data design for the complete system, and activity, data flow, sequence, state, class and deployment models describe how it works (Chapter 5).
- **Implementation.** Iteration 1 is complete: accounts, a chatbot that understands natural English and asks targeted follow-up questions, a quick-select form, seven planning agents coordinated by an Orchestrator that produce three ranked itineraries with a full cost breakdown, maps, saved trips and PDF export, on Android and the web (Chapter 6).
- **Testing and validation.** 103 backend tests pass with 93% coverage, 50 application tests pass, static analysis reports no issues, and all manual acceptance tests pass on

web and Android (Chapter 7).

- **Risks.** The main remaining risks are narration accuracy, video rendering time and free hosting limits. They are addressed by grounding narration in stored sources, rendering videos as background jobs and keeping a complete local deployment for demonstrations (Section 8.7).

- **Continuation into FYP-2.** Iteration 2 adds dynamic replanning, Iteration 3 cinematic narration and Iteration 4 the personalized travel video, followed by deployment and user evaluation (Chapter 9). Because every agent is independent and tested, the new agents can be added to the existing pipeline without changing the work already completed.

Bibliography

[1] Vacation and Travel Planner App LLC, "Wanderlog – travel planner," Apple App Store, accessed: Oct. 15, 2026. [Online]. Available: https://apps.apple.com/ZA/app/id1476732439

[2] "Expedia group acquires layla, accelerating its AI-powered trip planning and booking strategy," Hospitality Net, 2026, accessed: Oct. 15, 2026. [Online]. Available: https://www.hospitalitynet.org/news/4133792/expedia-group-acquires-layla-accelerating-its-ai-powere

[3] "Wego launches full travel services in Pakistan," Zameen News, 2025, accessed: Oct. 15, 2026. [Online]. Available: https://www.zameen.com/news/wego-launches-full-travel-services-in-pakistan.html

[4] "AI-powered tourism platform launched," The News International, Aug. 2026, accessed: Oct. 15, 2026. [Online]. Available: https://www.thenews.pk/print/1432365-ai-powered-tourism-platform-launched

[5] 3 Musafir, "Hunza.ai: An intuitive travel itinerary generator," Google AI Developer Competition, accessed: Oct. 15, 2026. [Online]. Available: https://ai.google.dev/competition/projects/hunzaai

[6] "Trango trip – find your next adventure, plan your trip to Pakistan," Product Hunt, accessed: Oct. 15, 2026. [Online]. Available: https://www.producthunt.com/p/trango-trip/trango-trip

[7] Z. Siddique, "Pakrism – Pakistan's #1 travel platform," Apple App Store, accessed: Oct. 15, 2026. [Online]. Available: https://apps.apple.com/us/app/-/id1496630730

[8] "Travel better with FindMyAdventure, one trip at a time," ProPakistani, accessed: Oct. 15, 2026. [Online]. Available: https://propakistani.pk/lens/travel-better-with-findmyadventure-one-trip-at-a-time/